كلية العلوم والتقنيات ببني ملال
ⵜⴰⵖⵡⵍⵉⵜ ⵏ ⵜⵓⵙⵙⵓⵔⵉⵏ ⴰ ⵜⵓⵏⵖⵎⵉⵙⵜⵉⵏ ⵏ ⴰⵣⵜ ⵅⵏⵉⵍⴰ
Faculté des Sciences et Techniques de Béni Mellal

N° .. /

Rapport de projet

Module : Internet des objets

Titre :

SmartCar Ops : système IoT de détection et de priorisation des obstacles routiers par vision artificielle, du prototype au monde réel

---

Réalisé par :

AGUELLOUL Mouad
DAHIBI Wissal
ELAALGI Hatim
JOUNDI Amin
JOUNDI Ayman

Encadré par :

Pr. BASLAM Mohamed

Soutenu le 26 octobre 2026

Année Universitaire 2025/2026

Résumé

Ce projet, réalisé dans le cadre du module *Internet des objets*, porte sur la conception d'un système intelligent de supervision d'un véhicule capable de détecter et de prendre en compte les obstacles présents sur son trajet. Le système associe trois briques complémentaires.

La première est un **prototype matériel** : une voiture miniature construite autour d'une carte Arduino UNO, d'un pont en H L298N, de deux moteurs à courant continu, de deux capteurs infrarouges qui suivent une piste tracée au ruban noir et d'un capteur ultrasonique HC-SR04. Une caméra ESP32-CAM autonome, alimentée par une batterie externe, diffuse en Wi-Fi les images prises du point de vue de la voiture.

La deuxième brique est un ensemble de **modèles de vision par ordinateur** de la famille YOLO26 (Ultralytics). Pour l'environnement réel, un modèle préentraîné sur COCO reconnaît les usagers de la route (piétons, cyclistes, véhicules, animaux) et un second modèle, affiné pendant huit heures sur 7 894 images de caméras embarquées, détecte les anomalies routières (nids-de-poule, cônes, barrières, panneaux de chantier). Pour le prototype, un modèle YOLO26n a été entraîné sur les images de notre propre piste, annotées automatiquement par suivi vidéo avec SAM 2, afin de reconnaître les deux obstacles du circuit.

La troisième brique est l'application web **SmartCar Ops**, développée avec Dash et Plotly et déployée avec Docker. Elle exécute les modèles en temps réel (ONNX Runtime), estime la distance et la gravité de chaque obstacle, lui attribue un niveau de priorité, le place sur une carte et en tient compte pour proposer l'itinéraire le plus sûr. Huit pages (vue d'ensemble, suivi, caméra, planification, obstacles, statistiques, modèles d'IA, périphériques et journaux) permettent de superviser deux environnements : un environnement *monde réel* (flotte simulée sur le réseau routier du Quartier Latin à Paris, avec de vraies vidéos de route analysées par les modèles) et l'environnement *prototype* de la voiture miniature.

Sur l'ensemble de test, le détecteur d'anomalies routières atteint un mAP@50 de 0,53 en temps réel (0,57 au meilleur réglage), et le modèle du prototype détecte tous les obstacles des images de test (mAP@50 de 0,995), résultat à nuancer par la petite taille de cet ensemble.

Mots-clés : Internet des objets, Arduino, ESP32-CAM, vision par ordinateur, détection d'objets, YOLO26, Ultralytics, SAM 2, Dash, Plotly, Docker, planification d'itinéraire.

Table des matières

Résumé **i**

Liste des abréviations **ix**

Introduction générale **1**

1 Contexte général et étude du projet **2**
1.1 Introduction . . . 2
1.2 Généralités sur l'Internet des Objets (IoT) . . . 2
1.3 IoT et Intelligence Artificielle . . . 3
1.4 Présentation du projet . . . 4
1.5 Problématique . . . 5
1.6 Objectifs du projet . . . 5
1.7 Cahier des charges . . . 6
1.8 Architecture générale du système . . . 8
Conclusion du chapitre . . . 9

2 Conception et réalisation matérielle **10**
2.1 Introduction . . . 10
2.2 Architecture matérielle du système . . . 10
2.3 Arduino UNO . . . 11
2.4 Module L298N . . . 13
2.5 Moteurs à courant continu . . . 13
2.6 Capteurs infrarouges . . . 14
2.7 Capteur ultrasonique HC-SR04 . . . 15
2.8 ESP32-CAM . . . 15
2.9 Système d'alimentation . . . 16
2.10 Schéma de câblage . . . 17
2.11 Réalisation physique du prototype . . . 18
Conclusion du chapitre . . . 21

3 Conception et développement du modèle d'intelligence artificielle **22**
Introduction du chapitre . . . 22
3.1 Introduction à la vision par ordinateur . . . 22
3.1.1 Principes de la vision par ordinateur . . . 22
3.1.2 Détection d'objets et segmentation d'images . . . 22

3.1.3 Application à la détection des obstacles routiers et des nids-de-poule 23
3.2 Présentation des modèles YOLO26 24
3.2.1 Généralités sur la famille YOLO 24
3.2.2 Présentation de YOLO26 et de son architecture 25
3.2.3 Utilisation des poids préentraînés sur COCO 25
3.2.4 Environnement Ultralytics 26
3.2.5 Adaptation du modèle aux besoins du projet 26
3.3 Préparation des données 27
3.3.1 Présentation des données réelles 27
3.3.2 Présentation des données du prototype 28
3.3.3 Organisation et annotation des images 28
3.3.4 Définition des classes à reconnaître 29
3.3.5 Prétraitement et augmentation des données 30
3.3.6 Division des données en ensembles d'entraînement, de validation et de test . 31
3.4 Développement des deux modèles 31
3.4.1 Modèle destiné aux données réelles 31
3.4.2 Modèle destiné au prototype 32
3.4.3 Configuration des modèles et des poids préentraînés 32
3.4.4 Procédure d'entraînement et de validation 32
3.4.5 Sauvegarde et chargement des modèles 33
3.5 Détection et analyse des obstacles 34
3.5.1 Détection des objets et des anomalies routières 34
3.5.2 Localisation des objets dans les images 34
3.5.3 Exploitation des scores de confiance 34
3.5.4 Identification des nids-de-poule et des obstacles 34
3.5.5 Estimation de la gravité des anomalies 35
3.5.6 Attribution d'un niveau de priorité 35
3.6 Évaluation des performances 35
3.6.1 Méthodologie d'évaluation 35
3.6.2 Précision, rappel et score F1 36
3.6.3 Intersection sur l'union (IoU) et mAP 36
3.6.4 Temps d'inférence et vitesse de traitement 36
3.6.5 Résultats obtenus sur les données réelles 37
3.6.6 Résultats obtenus sur les données du prototype 39
3.6.7 Analyse comparative des deux modèles 41
Conclusion du chapitre 41

4 Conception et développement de l'application SmartCar Ops 42
Introduction du chapitre 42
4.1 Présentation générale de l'application 42
4.1.1 Objectifs et fonctionnalités principales 42
4.1.2 Choix de Dash et de Plotly 42

4.1.3 Architecture logicielle de l'application 43
4.1.4 Organisation de l'interface et navigation 44
4.1.5 Gestion des deux environnements : Real World et Prototype 44
4.2 Architecture et gestion des données 45
4.2.1 Architecture générale des échanges de données 45
4.2.2 Intégration des modèles YOLO26 45
4.2.3 Acquisition et traitement des données du prototype 46
4.2.4 Gestion des données issues du monde réel 46
4.2.5 Organisation et stockage des informations 46
4.2.6 Actualisation des données et affichage en temps réel 47
4.3 Page Overview : tableau de bord général 47
4.3.1 Présentation de la vue d'ensemble 47
4.3.2 Affichage des indicateurs clés 48
4.3.3 Synthèse des véhicules et des obstacles 48
4.3.4 Comparaison des environnements Real World et Prototype 49
4.4 Page Tracking : suivi et localisation des véhicules 49
4.4.1 Visualisation de la position des véhicules 50
4.4.2 Suivi des déplacements 50
4.4.3 Historique des trajectoires 51
4.4.4 Consultation des parcours précédents 51
4.4.5 Gestion des données de localisation 52
4.5 Page Camera : visualisation du flux vidéo 52
4.5.1 Intégration du flux vidéo de l'ESP32-CAM 52
4.5.2 Affichage des images en direct 53
4.5.3 Intégration des résultats de l'intelligence artificielle 53
4.5.4 Visualisation des détections sur les images 54
4.5.5 Comparaison entre les vues Real World et Prototype 55
4.6 Page Route Planner : planification des itinéraires 55
4.6.1 Présentation de l'interface cartographique 55
4.6.2 Sélection des points de départ et d'arrivée 56
4.6.3 Représentation des obstacles sur la carte 56
4.6.4 Calcul des itinéraires possibles 56
4.6.5 Prise en compte de la distance, des obstacles et de leur gravité 56
4.6.6 Sélection et affichage de l'itinéraire recommandé 57
4.6.7 Comparaison des itinéraires proposés 57
4.7 Page Obstacles : gestion et priorisation des anomalies 58
4.7.1 Affichage des obstacles détectés 59
4.7.2 Identification des nids-de-poule et autres anomalies 59
4.7.3 Classification des obstacles par catégorie 59
4.7.4 Évaluation de la gravité 60
4.7.5 Attribution des niveaux de priorité 61
4.7.6 Localisation et consultation des anomalies 62

4.7.7 Filtrage et tri des obstacles 62
4.8 Page Live Stats : statistiques et indicateurs 62
4.8.1 Présentation des indicateurs statistiques 63
4.8.2 Nombre et répartition des obstacles détectés 63
4.8.3 Distribution des anomalies par gravité et priorité 63
4.8.4 Analyse des données de suivi des véhicules 64
4.8.5 Visualisation graphique des statistiques 64
4.8.6 Comparaison des résultats entre les deux environnements 64
4.9 Page AI Models : gestion des modèles d'intelligence artificielle 65
4.9.1 Présentation des modèles utilisés 65
4.9.2 Modèles YOLO26 pour les données réelles 66
4.9.3 Modèle YOLO26n pour le prototype 66
4.9.4 Configuration et chargement des modèles 67
4.9.5 Affichage des informations et des performances 67
4.9.6 Comparaison des résultats des modèles 67
4.10 Page Devices & Logs : périphériques et journaux du système 67
4.10.1 Présentation des périphériques connectés 68
4.10.2 Suivi de l'état des composants et des dispositifs 68
4.10.3 Consultation des journaux d'exécution 68
4.10.4 Gestion des événements et des erreurs 69
4.10.5 Diagnostic et surveillance du système 69
4.11 Synthèse des fonctionnalités de l'application 70
4.11.1 Interconnexion des différentes pages 70
4.11.2 Circulation des données entre les composants 71
4.11.3 Apports de l'application au système intelligent 71
Conclusion du chapitre 72

Conclusion et perspectives **73**

Bibliographie **74**

Table des figures

1.1 Architecture en couches d'un système IoT et correspondance avec notre projet . . . 3
1.2 Le prototype, l'application *SmartCar Ops* et ses deux environnements 5
1.3 Diagramme UML de cas d'utilisation de *SmartCar Ops* 8
1.4 Architecture globale du système 9

2.1 Architecture matérielle du prototype 11
2.2 Carte Arduino UNO du prototype et ses broches principales 12
2.3 Module L298N : bornes d'alimentation, entrées de commande et sorties moteurs . . 13
2.4 Les deux moteurs à courant continu et les éléments de roulement 14
2.5 Capteurs infrarouges gauche et droit (deux modules identiques) 14
2.6 Capteur ultrasonique HC-SR04 et ses quatre broches 15
2.7 Module ESP32-CAM et son support ESP32-CAM-MB 16
2.8 Éléments du circuit d'alimentation des moteurs 17
2.9 Schéma de câblage du prototype 18
2.10 Éléments mécaniques et de liaison 19
2.11 Prototype assemblé 19
2.12 Piste d'essai et obstacles du prototype 21

3.1 Chaîne de traitement d'un système de vision par ordinateur 22
3.2 Image d'origine, segmentation (masque produit par SAM 2) et détection (boîte englobante déduite du masque) pour les deux obstacles de la piste 23
3.3 Détections des modèles routiers (usagers de la route et anomalies) sur des images de l'ensemble de test, aux seuils de l'application. Les erreurs restent visibles : un nid-de-poule manqué (en haut au centre), un panneau STOP pris pour un panneau de chantier (en bas à droite). 24
3.4 Architecture simplifiée d'un détecteur YOLO26 : backbone, neck multi-échelle et tête de bout en bout sans NMS 25
3.5 Apprentissage par transfert à partir des poids COCO 26
3.6 Flux de travail avec Ultralytics, de la préparation des données à l'intégration 26
3.7 Données réelles : exemples d'images d'entraînement et distribution des annotations 27
3.8 Données du prototype : images extraites de la vidéo (passage devant la colonne, piste vide, boîte) 28
3.9 Annotation automatique par suivi vidéo avec SAM 2 : chaîne de traitement et planche de contrôle d'une séquence de la boîte 29
3.10 Augmentations spécifiques au prototype 30
3.11 Découpage temporel d'une séquence vidéo du prototype 31

3.12 Courbes d'apprentissage : pertes d'entraînement et de validation, précision, rappel et mAP par époque . . . 33
3.13 Intersection sur l'union entre une boîte réelle et une boîte prédite . . . 36
3.14 Résultats du détecteur d'anomalies routières sur l'ensemble de test . . . 39
3.15 Résultats du modèle du prototype sur l'ensemble de test . . . 40
3.16 Test du modèle du prototype sur le flux réel de l'ESP32-CAM . . . 41

4.1 Architecture logicielle de *SmartCar Ops* . . . 43
4.2 Menu latéral complet de l'application . . . 44
4.3 Changement d'environnement : barre supérieure et indicateurs de la page Overview 45
4.4 Flux des données entre les composants de *SmartCar Ops* . . . 45
4.5 Tableau de bord général (page Overview, environnement monde réel) . . . 48
4.6 Synthèse des obstacles et des événements de la page Overview . . . 49
4.7 Carte en direct dans les deux environnements . . . 49
4.8 Carte de suivi avec la position des véhicules et leurs traces . . . 50
4.9 Suivi des véhicules . . . 51
4.10 Historique des trajets et relecture d'un parcours précédent . . . 52
4.11 Panneaux de connexion et de réglage de l'ESP32-CAM . . . 53
4.12 Flux vidéo avec les prédictions des modèles affichées . . . 54
4.13 Détections, modèles et source de la page Camera (monde réel) . . . 55
4.14 Formulaire de planification d'itinéraire . . . 56
4.15 Itinéraire recommandé, obstacles et tronçons à risque . . . 57
4.16 Planification dans l'environnement prototype . . . 58
4.17 Liste des anomalies détectées, triées par priorité . . . 59
4.18 Carte et répartition des obstacles . . . 60
4.19 Niveaux de gravité et priorités : détail de trois obstacles détectés par la caméra embarquée . . . 61
4.20 Indicateurs de la page Obstacles : obstacles ouverts par niveau, revues et faux positifs 61
4.21 Barre de filtres de la page Obstacles . . . 62
4.22 Graphiques et indicateurs statistiques (monde réel) . . . 62
4.23 Indicateurs statistiques dans les deux environnements . . . 63
4.24 Nombre et répartition des obstacles détectés . . . 63
4.25 Gravité des anomalies et données de suivi des véhicules . . . 64
4.26 Page Live stats dans les deux environnements . . . 65
4.27 Page de présentation des modèles : détecteur d'anomalies routières . . . 65
4.28 Carte du modèle des usagers de la route (préentraîné sur COCO) . . . 66
4.29 Carte du modèle du prototype . . . 66
4.30 Comportement des modèles sur la route (7 derniers jours) . . . 67
4.31 Périphériques et services du système . . . 68
4.32 Journal des événements . . . 69
4.33 Réglage des poids de danger des classes . . . 70
4.34 Interactions entre les modules de l'application . . . 71

Liste des tableaux

2.1 Composants du prototype 11
2.2 Caractéristiques principales de l'Arduino UNO 12
2.3 Affectation des broches de l'Arduino UNO 12
2.4 Commande d'un moteur par le L298N (moteur gauche : IN1/IN2 ; moteur droit : IN3/IN4) 13
2.5 Logique de suivi de ligne à deux capteurs 15
2.6 Ordres de grandeur des consommations (valeurs typiques des fiches techniques) . . . 17
2.7 Liaisons du schéma de câblage 18

3.1 Quelques étapes de l'évolution de la famille YOLO 24
3.2 Les trois modèles du projet 27
3.3 Classes du modèle des usagers de la route (regroupement des classes COCO) 29
3.4 Classes des modèles entraînés 30
3.5 Répartition des données 31
3.6 Configuration d'entraînement des deux modèles 32
3.7 Niveaux de priorité 35
3.8 Temps d'inférence mesurés 37
3.9 Résultats du détecteur d'anomalies routières sur l'ensemble de test 37
3.10 Évaluation agnostique de la classe et points de fonctionnement (ensemble de test) . . 38
3.11 Résultats du modèle du prototype sur l'ensemble de test (26 images, 15 obstacles) . . 39
3.12 Matrice de confusion du modèle du prototype (test, seuil 0,40) 40
3.13 Comparaison des deux modèles entraînés 41

Liste des abréviations

Abréviation	Signification
AIoT	*Artificial Intelligence of Things* — intelligence artificielle des objets
AMP	*Automatic Mixed Precision* — calcul en précision mixte
AP	*Average Precision* — précision moyenne
API	*Application Programming Interface*
ArUco	Marqueurs carrés de repérage (Université de Cordoue)
CNN	*Convolutional Neural Network* — réseau de neurones convolutif
COCO	*Common Objects in Context* — jeu de données de référence
CPU	*Central Processing Unit* — processeur
CSS	*Cascading Style Sheets*
DC	*Direct Current* — courant continu
DFL	*Distribution Focal Loss*
ETA	*Estimated Time of Arrival* — durée estimée du trajet
F1	Moyenne harmonique de la précision et du rappel
FN / FP	Faux négatif / faux positif
FPS	*Frames Per Second* — images par seconde
GND	*Ground* — masse électrique
GPU	*Graphics Processing Unit* — processeur graphique
HTTP	*HyperText Transfer Protocol*
IA	Intelligence artificielle
IoT	*Internet of Things* — Internet des objets
IoU	*Intersection over Union* — intersection sur l'union
IR	Infrarouge
JPEG	*Joint Photographic Experts Group* — format d'image compressée
KPI	*Key Performance Indicator* — indicateur clé
mAP	*mean Average Precision* — moyenne des précisions moyennes

Abréviation	Signification
MB	Carte de programmation USB ESP32-CAM-MB
MJPEG	*Motion JPEG* — flux vidéo formé d'images JPEG successives
NMS	*Non-Maximum Suppression* — suppression des non-maxima
ONNX	*Open Neural Network Exchange* — format d'échange de modèles
OSM	OpenStreetMap
PSRAM	*Pseudo-Static RAM* — mémoire externe de l'ESP32
PWM	*Pulse Width Modulation* — modulation de largeur d'impulsion
QVGA / VGA	Résolutions 320×240 / 640×480 pixels
SAM	*Segment Anything Model*
SQL	*Structured Query Language*
TP	Vrai positif (*True Positive*)
TTA	*Test-Time Augmentation* — augmentation au moment de l'inférence
UML	*Unified Modeling Language*
USB	*Universal Serial Bus*
Wi-Fi	Réseau local sans fil IEEE 802.11
YOLO	*You Only Look Once* — famille de détecteurs d'objets en une passe

Introduction générale

Les véhicules connectés et autonomes reposent sur une idée simple : un véhicule qui perçoit son environnement, qui communique ce qu'il perçoit et qui en tire des décisions est plus sûr et plus efficace. Cette idée se trouve au croisement de deux domaines qui progressent rapidement : l'Internet des objets (IoT), qui relie des capteurs et des actionneurs à des services distants, et l'intelligence artificielle, et plus particulièrement la vision par ordinateur, qui permet d'interpréter automatiquement des images.

Sur la route, les obstacles ne sont pas seulement les autres usagers. Les nids-de-poule, les cônes, les barrières et les zones de travaux sont à l'origine de dégâts matériels et d'accidents, et leur signalement reste souvent manuel et tardif. Un système capable de les détecter automatiquement à partir d'une caméra embarquée, de les localiser, d'en évaluer la gravité et de les partager avec les autres véhicules apporterait une aide directe à la conduite et à la maintenance du réseau routier.

C'est dans ce contexte que s'inscrit notre projet, réalisé dans le cadre du module *Internet des objets* à la Faculté des Sciences et Techniques de Béni Mellal. Nous avons conçu une chaîne complète, du capteur jusqu'à l'interface de supervision :

— un **prototype** de voiture miniature qui suit une piste et réagit aux obstacles, équipé d'une caméra Wi-Fi ESP32-CAM ;

— des **modèles de détection d'objets** YOLO26, entraînés d'une part sur des données routières réelles et d'autre part sur des images de notre propre piste ;

— une **application web**, *SmartCar Ops*, qui exécute ces modèles en temps réel, estime la gravité des obstacles, les priorise, les place sur une carte et propose des itinéraires qui les évitent.

L'application gère deux environnements. L'environnement *monde réel* simule une petite flotte de véhicules sur le réseau routier réel d'un quartier de Paris ; la caméra du véhicule CAR-01 y est remplacée par de vraies vidéos de route analysées par nos modèles. L'environnement *prototype* correspond à la voiture miniature et à sa piste. Ce double environnement permet de valider la même chaîne logicielle à deux échelles.

Ce rapport est organisé en quatre chapitres. Le **premier chapitre** présente le contexte général, l'IoT et son association avec l'intelligence artificielle, la problématique, les objectifs, le cahier des charges et l'architecture générale du système. Le **deuxième chapitre** décrit la conception et la réalisation matérielle du prototype. Le **troisième chapitre** est consacré aux modèles d'intelligence artificielle : données, entraînement, évaluation et intégration. Le **quatrième chapitre** présente l'application *SmartCar Ops* page par page. Nous terminons par une conclusion générale et des perspectives.

Chapitre 1 : Contexte général et étude du projet

1.1 Introduction

Ce premier chapitre situe le projet dans son contexte. Nous rappelons d'abord les notions essentielles de l'Internet des objets et la manière dont l'intelligence artificielle l'enrichit. Nous présentons ensuite le projet, la problématique qu'il traite et les objectifs fixés. Nous terminons par le cahier des charges, formalisé par un diagramme de cas d'utilisation, et par l'architecture générale du système.

1.2 Généralités sur l'Internet des Objets (IoT)

L'Internet des objets désigne l'extension d'Internet à des objets physiques dotés de capteurs, d'actionneurs, d'une capacité de calcul et d'une interface de communication. Ces objets collectent des données sur leur environnement, les transmettent à des services distants qui les analysent, et peuvent recevoir en retour des commandes [1, 2]. La valeur d'un système IoT ne vient pas d'un objet isolé, mais de la chaîne complète qui transforme une mesure brute en information utile, puis en décision.

Architecture en couches

On décrit classiquement un système IoT par une architecture en couches (figure 1.1) :

— la **couche perception**, formée des capteurs et des actionneurs (dans notre cas : capteurs infrarouges, capteur ultrasonique, caméra, moteurs) ;

— la **couche réseau**, qui achemine les données (Wi-Fi, Bluetooth, réseaux cellulaires, LoRa. . .) avec des protocoles applicatifs comme HTTP ou MQTT ;

— la **couche traitement**, qui stocke et analyse les données sur un serveur, dans le nuage ou en périphérie de réseau (*edge*) ;

— la **couche application**, qui présente l'information à l'utilisateur et lui permet d'agir.

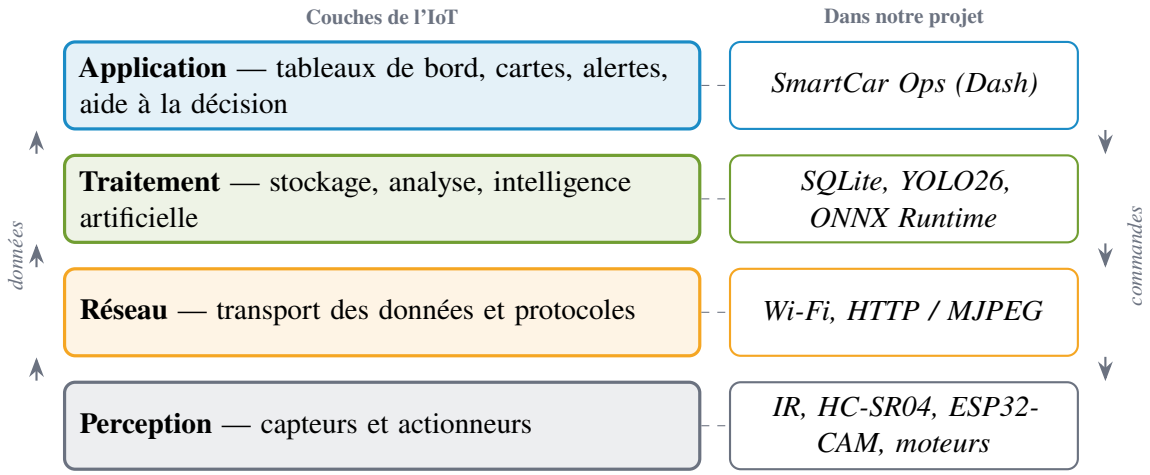


Figure 1.1. Architecture en couches d'un système IoT et correspondance avec notre projet

Domaines d'application et enjeux

L'IoT est présent dans la maison connectée, l'industrie (maintenance prédictive), l'agriculture de précision, la santé et, bien sûr, le transport : véhicules connectés, gestion du trafic, surveillance de l'état des routes. Ce déploiement massif soulève plusieurs enjeux : les contraintes énergétiques des objets, la fiabilité des communications sans fil, le volume des données à traiter, la sécurité et la confidentialité, et l'interopérabilité entre équipements hétérogènes [3].

1.3 IoT et Intelligence Artificielle

Les objets connectés produisent des données en grande quantité, mais ces données n'ont de valeur que si elles sont interprétées. L'association de l'IoT et de l'intelligence artificielle, parfois appelée *AIoT* (*Artificial Intelligence of Things*), consiste à confier cette interprétation à des modèles d'apprentissage automatique [4]. Une caméra connectée ne transmet alors plus seulement des images : elle permet de savoir *quels* objets se trouvent devant le véhicule, *où* ils se trouvent et *à quel point* ils sont dangereux.

Deux stratégies de déploiement coexistent :

— l'**inférence en périphérie** (*edge AI*), sur l'objet lui-même ou à proximité, qui réduit la latence et la bande passante mais est limitée par la puissance de calcul et l'énergie disponibles ;

— l'**inférence sur serveur ou dans le nuage**, qui permet d'utiliser des modèles plus lourds au prix d'une dépendance au réseau.

Dans notre projet, la caméra ESP32-CAM ne dispose ni de la mémoire ni de la puissance nécessaires pour exécuter un détecteur moderne à une cadence utile. Elle se contente donc d'acquérir et de diffuser les images en Wi-Fi ; l'inférence est réalisée sur un serveur local (l'ordinateur portable qui héberge l'application), au plus près du véhicule. Les modèles ont été choisis légers (variantes *nano* et *small* de YOLO26) et exportés au format ONNX afin de rester exécutables en temps réel sur processeur, ce qui préserve la possibilité d'un déploiement embarqué ultérieur.

1.4 Présentation du projet

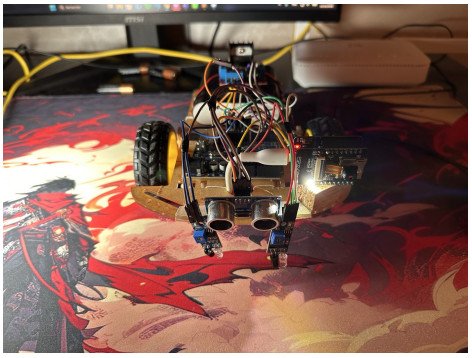
Le projet consiste à concevoir un système intelligent qui détecte les obstacles présents sur le trajet d'un véhicule, en évalue la gravité et met ces informations à disposition d'un opérateur à travers une application de supervision. Il est composé de trois éléments (figure 1.2) :

1. **Le prototype** : une voiture miniature qui suit une piste tracée au ruban adhésif noir grâce à deux capteurs infrarouges, mesure la distance qui la sépare d'un obstacle avec un capteur ultrasonique, et embarque une caméra ESP32-CAM qui filme la route de son point de vue.
2. **Les modèles d'intelligence artificielle** : des détecteurs d'objets YOLO26 qui reconnaissent les obstacles dans les images, d'une part sur des routes réelles, d'autre part sur la piste du prototype.
3. **L'application *SmartCar Ops*** : un tableau de bord web qui exécute les modèles, localise et priorise les obstacles, suit les véhicules, planifie des itinéraires et présente des statistiques.

L'application fonctionne dans **deux environnements**, sélectionnés par un commutateur dans la barre supérieure :

— **Monde réel** : une flotte de trois véhicules simulés circule sur le réseau routier réel du Quartier Latin à Paris (données OpenStreetMap). Le véhicule CAR-01 est équipé d'une « caméra embarquée » virtuelle qui lit de vraies vidéos de route ; nos modèles les analysent en temps réel et les obstacles détectés sont placés sur la carte devant le véhicule.

— **Prototype** : la voiture miniature sur sa piste. Les informations proviennent uniquement de l'image de l'ESP32-CAM (obstacles détectés, marqueurs de points de passage, mouvement), car la carte Arduino n'a pas de liaison sans fil.



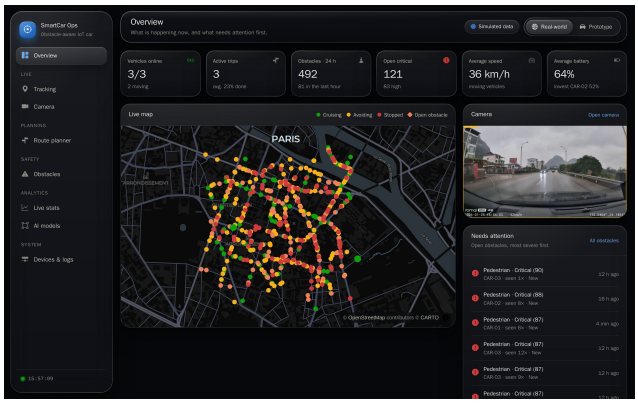
(a) Le prototype



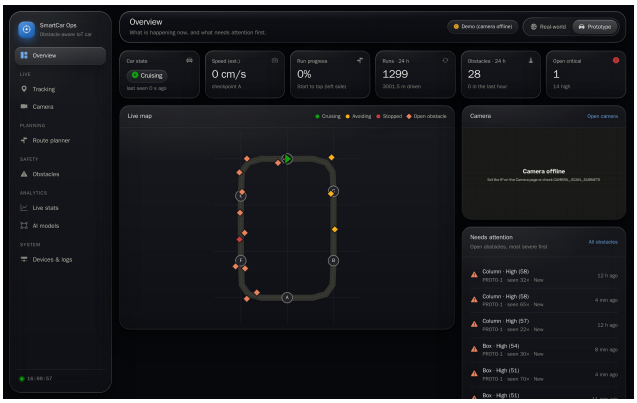
(b) La piste et ses obstacles



(c) Vue de la caméra et détection



(d) Environnement *Monde réel*



(e) Environnement *Prototype*

FIGURE 1.2. Le prototype, l'application *SmartCar Ops* et ses deux environnements

1.5 Problématique

La détection des obstacles routiers pose plusieurs difficultés. Les obstacles sont très variés : usagers mobiles (piétons, cyclistes, véhicules, animaux) et anomalies statiques (nids-de-poule, cônes, barrières, panneaux de chantier). Ils apparaissent dans des conditions changeantes de lumière, de météo et de point de vue. Les nids-de-poule, en particulier, sont plats, de forme irrégulière et souvent peu contrastés.

Détecter ne suffit pas : un opérateur a besoin de savoir *où* se trouve l'obstacle, *à quel point* il est dangereux et *lequel traiter en premier*. Ces informations doivent ensuite servir à la décision, par exemple au choix d'un itinéraire. Enfin, le prototype impose ses propres contraintes : une carte Arduino aux ressources très limitées, une caméra peu coûteuse dont l'adresse IP change à chaque connexion, et aucune liaison de communication entre l'Arduino et la caméra.

La problématique du projet peut donc se formuler ainsi :

> *Comment concevoir, avec des composants à faible coût, un système IoT capable de détecter automatiquement les obstacles sur le trajet d'un véhicule, d'en évaluer la gravité et la priorité, et d'exploiter ces informations en temps réel pour la supervision et la planification d'itinéraires, aussi bien sur des routes réelles que sur un prototype à échelle réduite ?*

1.6 Objectifs du projet

Les objectifs fixés sont les suivants :

1. **Réaliser le prototype** : assembler et câbler une voiture miniature capable de suivre une piste et de réagir à un obstacle, équipée d'une caméra Wi-Fi.
2. **Entraîner des modèles de détection** : choisir des jeux de données réels d'obstacles routiers, entraîner un détecteur dans un environnement reproductible (notebook Jupyter dans un conteneur Docker), puis constituer et annoter un jeu de données propre à notre piste pour entraîner le modèle du prototype.
3. **Évaluer honnêtement les modèles** : mesurer la précision, le rappel, le score F1, le mAP et le temps d'inférence sur des données jamais vues pendant l'entraînement.
4. **Exploiter les détections** : estimer la distance et la gravité des obstacles, leur attribuer une priorité, les localiser et les fusionner lorsqu'ils sont vus plusieurs fois.
5. **Développer l'application *SmartCar Ops*** : suivi des véhicules, flux vidéo annoté, planification d'itinéraires tenant compte des obstacles, gestion des anomalies, statistiques, présentation des modèles, surveillance des périphériques et des journaux, pour les deux environnements.
6. **Assurer un déploiement simple** : l'ensemble du logiciel doit se lancer avec Docker, sans installation manuelle de bibliothèques.

1.7 Cahier des charges

Besoins fonctionnels

— **BF1 — Deux environnements** : basculer à tout moment entre *Monde réel* et *Prototype* ; chaque page s'adapte (unités en mètres ou en centimètres, carte routière ou plan de la piste).

— **BF2 — Vue d'ensemble** : indicateurs clés, carte en direct, obstacles à traiter, derniers événements.

— **BF3 — Suivi** : position des véhicules en temps réel, traces, état, historique des trajets et relecture d'un trajet passé.

— **BF4 — Caméra** : afficher le flux de l'ESP32-CAM malgré une adresse IP dynamique (découverte automatique), ou les vidéos de la caméra embarquée, avec les détections superposées.

— **BF5 — Planification** : choisir un départ et une destination, proposer l'itinéraire le plus court, le plus sûr et un compromis réglable ; éditer la piste du prototype et ses trajectoires.

— **BF6 — Obstacles** : liste triée par priorité, filtres, détail avec instantané, carte, actions (acquitter, résoudre, marquer comme faux positif).

— **BF7 — Statistiques** : répartitions par classe, par gravité, par heure, états des véhicules.

— **BF8 — Modèles d'IA** : métriques d'évaluation, matrice de confusion, comportement sur la route.

— **BF9 — Périphériques et journaux** : état des équipements, journal des événements, réglages (poids de danger des classes, données de démonstration).

Besoins non fonctionnels

— **Temps réel** : rafraîchissement des données chaque seconde ; analyse vidéo sur processeur sans bloquer l'affichage (seule l'image la plus récente est analysée).

— **Fiabilité** : reconnexion automatique de la caméra, base de données robuste (SQLite en mode WAL dans un volume Docker).

— **Ergonomie** : interface claire et cohérente, couleurs de gravité toujours accompagnées d'une icône et d'un libellé.

— **Portabilité et reproductibilité** : application et entraînement conteneurisés (Docker).

— **Évolutivité** : détecteur interchangeable (profils de modèles ONNX), configuration par variables d'environnement.

— **Honnêteté des mesures** : métriques calculées sur des ensembles de test séparés, avec le réglage utilisé clairement indiqué.

Diagramme de cas d'utilisation

Le diagramme de la figure 1.3 résume les interactions entre les acteurs et le système. L'opérateur supervise les véhicules et les obstacles ; l'administrateur, qui est un opérateur avec des droits de configuration, règle la caméra, la piste et les poids de danger. La caméra ESP32-CAM et la caméra embarquée du monde réel sont des acteurs secondaires qui fournissent les images.

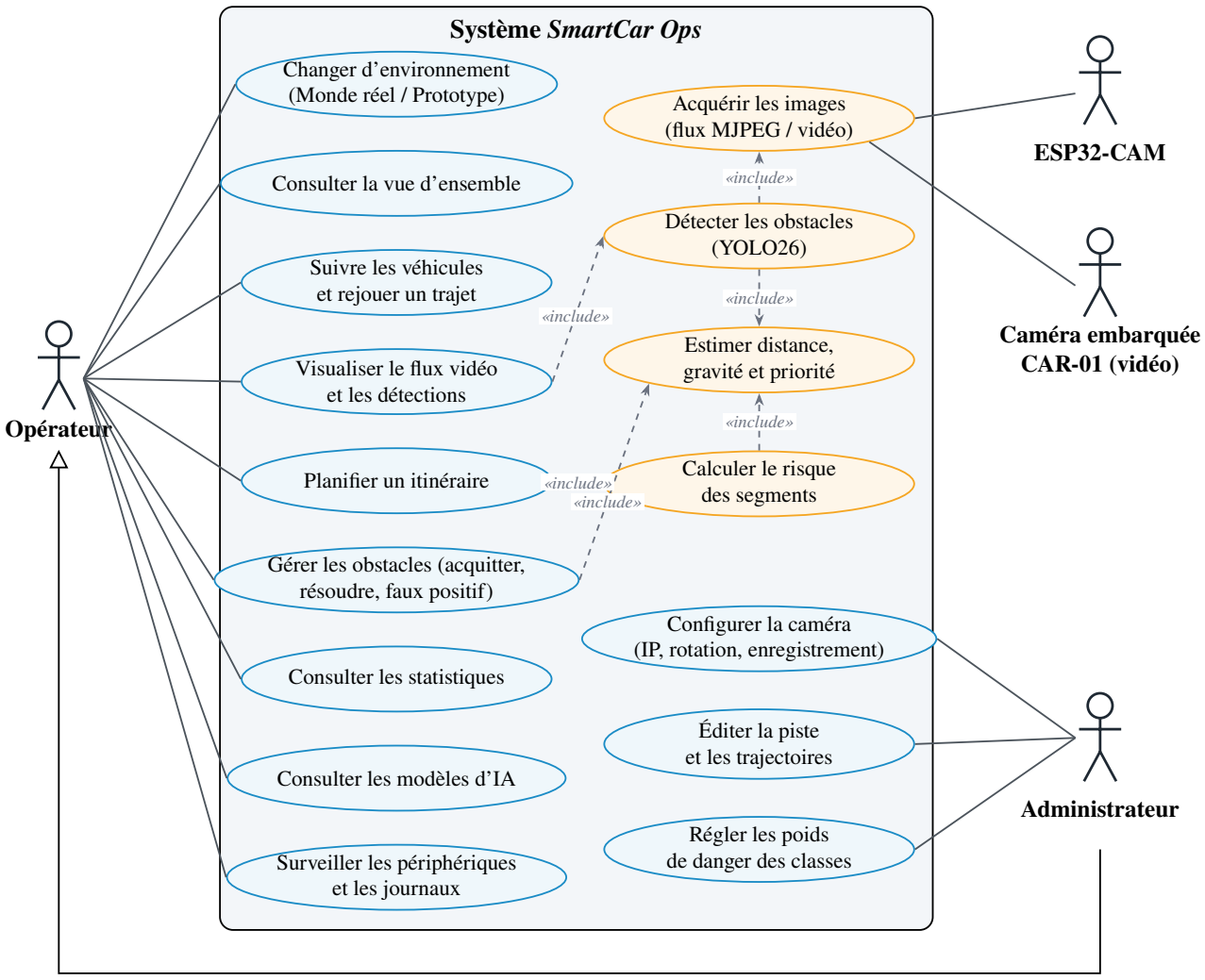


FIGURE 1.3. Diagramme UML de cas d'utilisation de *SmartCar Ops*

1.8 Architecture générale du système

La figure 1.4 présente l'architecture globale, organisée selon les couches de l'IoT.

— **Perception.** Dans le prototype, l'Arduino UNO lit les deux capteurs infrarouges et le capteur ultrasonique, et pilote les deux moteurs à travers le L298N : c'est la boucle de commande embarquée, autonome. L'ESP32-CAM, alimentée séparément, filme la piste. Dans le monde réel, la perception est assurée par des vidéos de caméras embarquées et par un simulateur de flotte qui produit la télémétrie des véhicules.

— **Réseau.** L'ESP32-CAM diffuse un flux MJPEG en HTTP sur le réseau Wi-Fi local (port 81) et expose une interface de réglage (port 80). Son adresse IP étant attribuée dynamiquement, le serveur la retrouve par balayage du sous-réseau.

— **Traitement.** Un conteneur Docker héberge le serveur : relais du flux vidéo (la caméra n'accepte qu'un seul client), redressement de l'image, détection d'objets par les modèles YOLO26 exportés en ONNX, lecture des marqueurs ArUco, estimation de la distance, de la gravité et de la priorité, fusion des détections répétées, calcul du risque des segments et planification d'itinéraires. Les données sont stockées dans une base SQLite. Un second conteneur, utilisé hors ligne, sert à

l'entraînement des modèles sur GPU (JupyterLab et Ultralytics).

— **Application.** L'application Dash présente les huit pages de supervision dans le navigateur de l'opérateur et se rafraîchit chaque seconde.

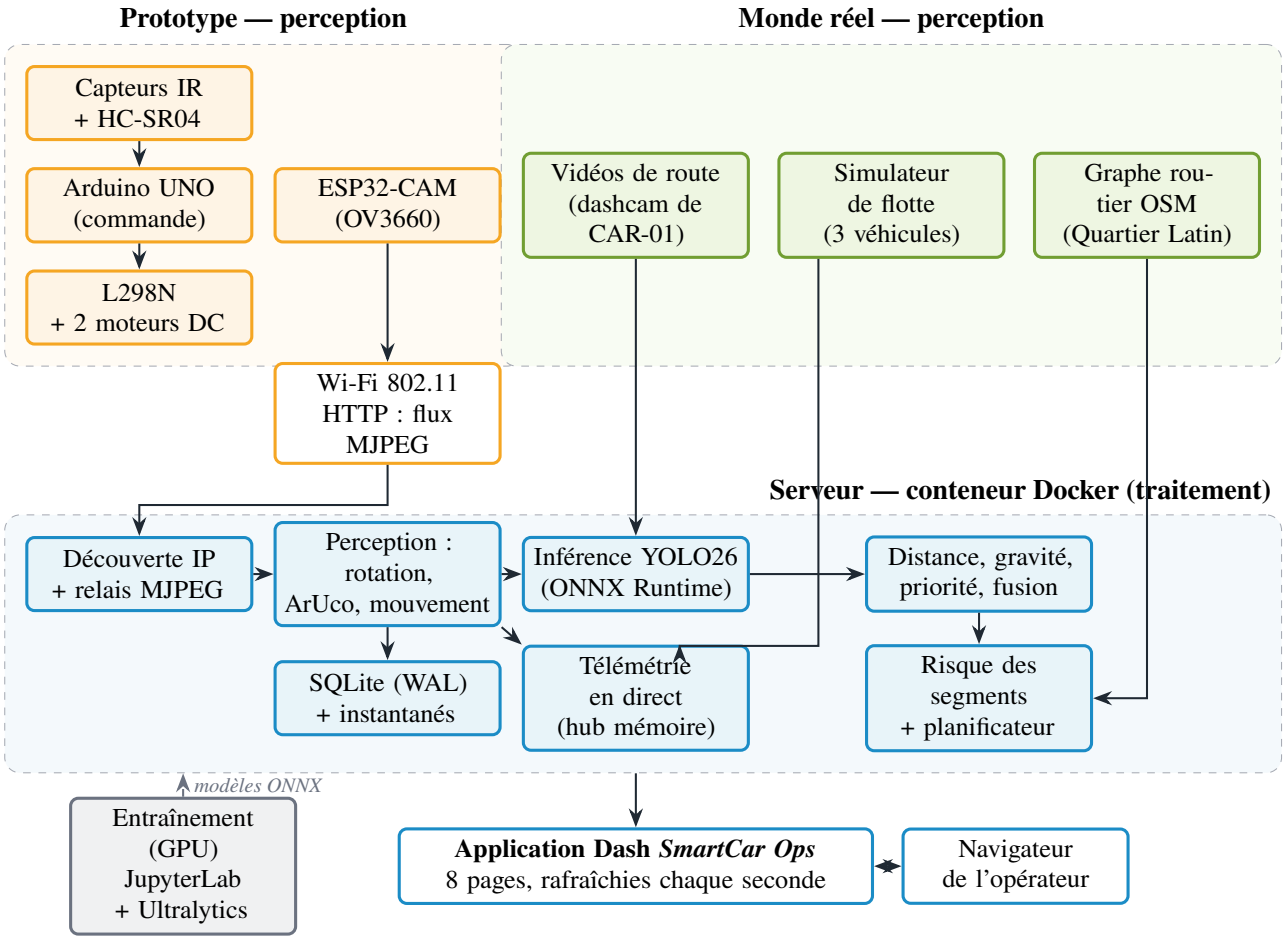


Figure 1.4. Architecture globale du système

Conclusion du chapitre

Ce chapitre a présenté le contexte de l'IoT et de son association avec l'intelligence artificielle, puis le projet lui-même : un prototype, des modèles de détection et une application de supervision couvrant deux environnements. Le cahier des charges et l'architecture générale fixent le cadre des chapitres suivants. Le chapitre 2 détaille la réalisation matérielle du prototype.

Chapitre 2 : Conception et réalisation matérielle

2.1 Introduction

Ce chapitre décrit la partie matérielle du projet : la voiture miniature qui constitue l'environnement *prototype*. Nous présentons d'abord l'architecture matérielle d'ensemble, puis chacun des composants (rôle, caractéristiques et broches utilisées), le système d'alimentation, le schéma de câblage et enfin la réalisation physique du prototype et de sa piste.

2.2 Architecture matérielle du système

Le prototype est organisé autour de deux sous-systèmes indépendants (figure 2.1) :

— **le sous-système de commande**, construit autour de l'Arduino UNO, qui lit les capteurs infrarouges et ultrasonique et commande les deux moteurs à travers le pont en H L298N. Il assure de manière autonome le suivi de la piste et l'arrêt devant un obstacle ;

— **le sous-système de vision**, formé de l'ESP32-CAM montée sur sa carte ESP32-CAM-MB et alimentée par une batterie externe. Il filme la route et diffuse les images en Wi-Fi vers le serveur *SmartCar Ops*.

Les deux sous-systèmes ne sont pas reliés entre eux : il n'existe pas de liaison série entre l'Arduino et l'ESP32-CAM. Toutes les informations du prototype affichées par l'application proviennent donc de l'image.

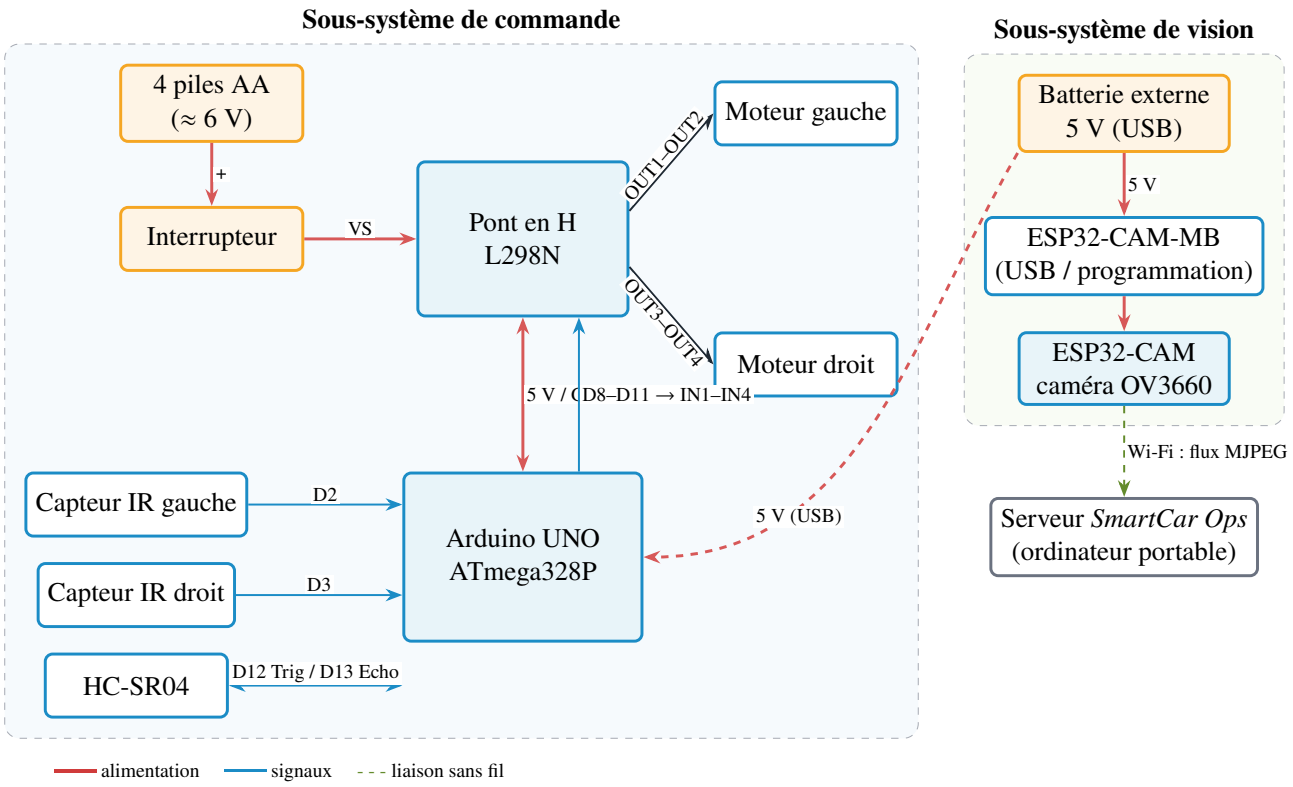


Figure 2.1. Architecture matérielle du prototype

Le tableau 2.1 récapitule les composants utilisés.

Table 2.1. Composants du prototype

Composant	Qté	Rôle
Arduino UNO (ATmega328P)	1	Unité de commande : lecture des capteurs, commande des moteurs
Module L298N (double pont en H)	1	Interface de puissance entre l'Arduino et les moteurs
Moteur à courant continu avec réducteur (« TT »)	2	Propulsion et direction différentielle
Capteur infrarouge à réflexion	2	Détection du ruban noir de la piste (gauche / droite)
Capteur ultrasonique HC-SR04	1	Mesure de la distance à l'obstacle frontal
ESP32-CAM + carte ESP32-CAM-MB	1	Acquisition et diffusion Wi-Fi des images
4 piles alcalines AA 1,5 V + interrupteur	1	Alimentation des moteurs
Batterie externe USB 5 V	1	Alimentation de la caméra et de l'électronique de commande
Châssis, roues, roue folle, fils Dupont	—	Structure mécanique et liaisons

2.3 Arduino UNO

L'Arduino UNO est une carte de développement construite autour du microcontrôleur 8 bits ATmega328P de Microchip [5]. Elle est simple à programmer (langage Arduino, dérivé du C++), robuste et bien documentée, ce qui en fait un choix naturel pour la boucle de commande du prototype.

Ses principales caractéristiques sont données dans le tableau 2.2, et la figure 2.2 identifie ses broches principales.

Table 2.2. Caractéristiques principales de l'Arduino UNO

Microcontrôleur	ATmega328P, 8 bits, 16 MHz
Mémoires	32 Ko de Flash, 2 Ko de SRAM, 1 Ko d'EEPROM
Entrées / sorties numériques	14 (dont 6 compatibles MLI/PWM)
Entrées analogiques	6 (A0 à A5), convertisseur 10 bits
Tension de fonctionnement	5 V (alimentation par USB ou prise jack 7–12 V)
Courant maximal par broche	20 mA recommandés (40 mA absolus)

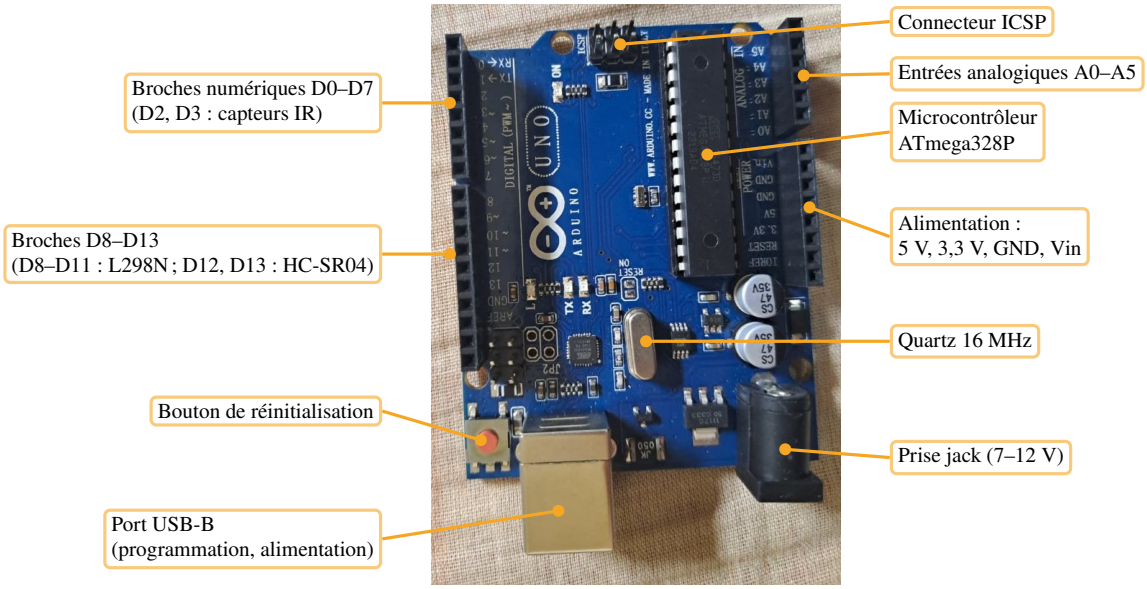


Figure 2.2. Carte Arduino UNO du prototype et ses broches principales

Dans notre montage, l'Arduino utilise huit broches numériques (tableau 2.3). Les broches D2 et D3 reçoivent les sorties tout-ou-rien des capteurs infrarouges ; D8 à D11 imposent le sens de rotation des moteurs au L298N ; D12 déclenche la mesure du HC-SR04 et D13 en reçoit l'écho.

Table 2.3. Affectation des broches de l'Arduino UNO

Broche	**Connectée à**	**Rôle**
D2	Capteur IR gauche (OUT)	Entrée : ruban détecté à gauche
D3	Capteur IR droit (OUT)	Entrée : ruban détecté à droite
D8, D9	L298N IN1, IN2	Sorties : sens du moteur gauche
D10, D11	L298N IN3, IN4	Sorties : sens du moteur droit
D12	HC-SR04 Trig	Sortie : impulsion de déclenchement (10 µs)
D13	HC-SR04 Echo	Entrée : durée de l'écho
5 V, GND	L298N, HC-SR04, capteurs IR	Alimentation logique et masse commune

2.4 Module L298N

Un microcontrôleur ne peut fournir que quelques dizaines de milliampères par broche, alors qu'un moteur en consomme plusieurs centaines. Le module L298N sert d'interface de puissance : il contient deux ponts en H qui permettent de commander indépendamment le sens (et, par les entrées ENA/ENB, la vitesse) de deux moteurs à courant continu [6]. Il accepte une tension moteur de 5 à 35 V, un courant de 2 A par voie, et intègre un régulateur 5 V (78M05). Les bornes et entrées du module sont identifiées sur la figure 2.3.

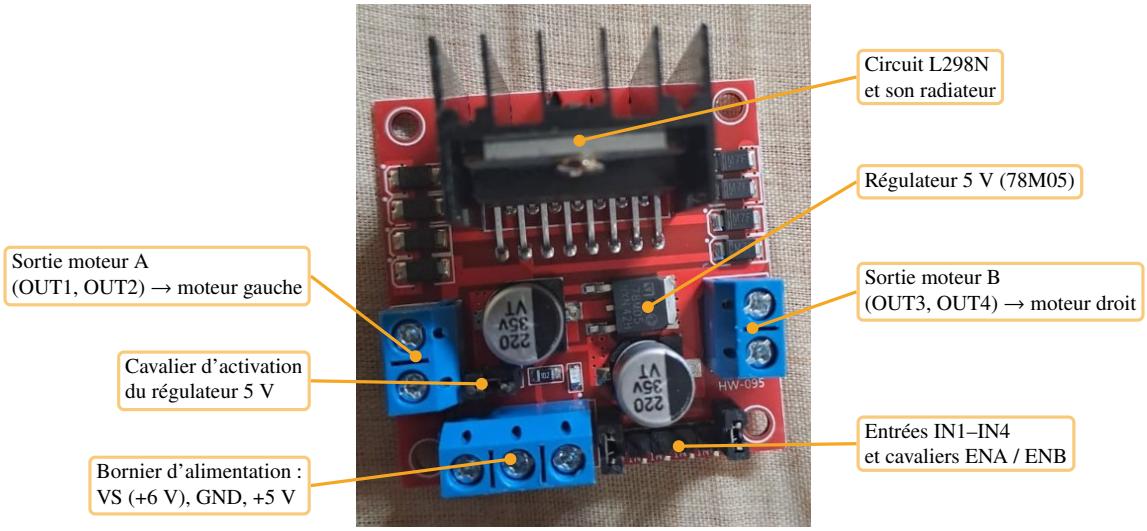


Figure 2.3. Module L298N : bornes d'alimentation, entrées de commande et sorties moteurs

Les cavaliers ENA et ENB étant en place, les deux moteurs tournent à pleine tension et leur sens est fixé par les entrées IN (tableau 2.4). La direction de la voiture est obtenue par *commande différentielle* : pour tourner à gauche, on ralentit ou on arrête le moteur gauche pendant que le moteur droit continue d'avancer.

Table 2.4. Commande d'un moteur par le L298N (moteur gauche : IN1/IN2 ; moteur droit : IN3/IN4)

IN1 (IN3)	IN2 (IN4)	Effet sur le moteur
1	0	Rotation en marche avant
0	1	Rotation en marche arrière
0	0	Roue libre (arrêt)
1	1	Freinage

Il faut noter que le L298N, à transistors bipolaires, provoque une chute de tension de l'ordre de 2 V : avec une alimentation de 6 V, les moteurs reçoivent environ 4 V, ce qui limite la vitesse de la voiture.

2.5 Moteurs à courant continu

La propulsion est assurée par deux moteurs à courant continu munis d'un réducteur en plastique (moteurs dits « TT », figure 2.4). Prévus pour 3 à 6 V, ils offrent, grâce au réducteur (rapport typique

1 :48), un couple suffisant pour entraîner directement les roues de 65 mm. Chaque moteur est fixé sur le châssis par un support et entraîne une roue arrière ; une roue folle à l'avant assure l'équilibre. Les deux moteurs étant commandés séparément, la voiture tourne en faisant varier la vitesse relative des deux roues.



(a) Moteur à réducteur (identique pour les deux côtés)

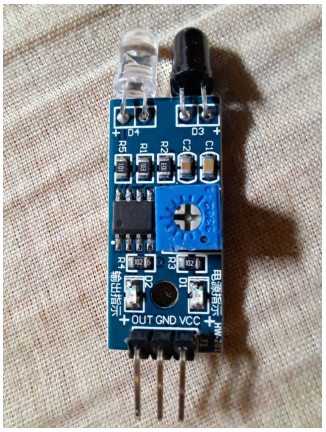


(b) Roues, roue folle et visserie

FIGURE 2.4. Les deux moteurs à courant continu et les éléments de roulement

2.6 Capteurs infrarouges

Deux capteurs infrarouges à réflexion, orientés vers le sol, permettent de suivre la piste (figure 2.5). Chaque module comporte une LED émettrice infrarouge, une photodiode réceptrice, un comparateur LM393 et un potentiomètre de réglage du seuil. Le sol clair réfléchit le rayonnement infrarouge, alors que le ruban noir l'absorbe : la sortie numérique OUT change donc d'état lorsque le capteur passe au-dessus du ruban. Le potentiomètre permet d'ajuster le seuil à la hauteur du capteur et à l'éclairage.



(a) Face avant : LED IR, photodiode, potentiomètre, broches OUT / GND / VCC



(b) Face arrière

FIGURE 2.5. Capteurs infrarouges gauche et droit (deux modules identiques)

Les deux capteurs sont placés de part et d'autre du ruban. Tant que tous deux voient le sol clair, la voiture avance tout droit ; lorsque l'un d'eux détecte le ruban, la ligne s'est décalée de ce côté et la voiture tourne dans cette direction (tableau 2.5).

Table 2.5. Logique de suivi de ligne à deux capteurs

IR gauche	IR droit	Action
sol clair	sol clair	Avancer tout droit
ruban noir	sol clair	Tourner à gauche
sol clair	ruban noir	Tourner à droite
ruban noir	ruban noir	S'arrêter (croisement ou fin de piste)

2.7 Capteur ultrasonique HC-SR04

Le HC-SR04 mesure la distance à l'obstacle situé devant la voiture par temps de vol d'une onde ultrasonore (figure 2.6) [7]. Une impulsion d'au moins 10 µs sur la broche **Trig** déclenche l'émission d'une salve de huit impulsions à 40 kHz ; la broche **Echo** passe alors à l'état haut pendant une durée t égale au temps d'aller-retour de l'onde. La distance s'en déduit par :

$$d = \frac{v_{son}\, t}{2} \approx \frac{343 \text{ m/s} \times t}{2} \quad \Longleftrightarrow \quad d \text{ [cm]} \approx \frac{t \text{ [µs]}}{58} \tag{2.1}$$

Le capteur mesure de 2 cm à 4 m environ, avec un cône de détection d'une quinzaine de degrés. Il est alimenté en 5 V et ses quatre broches sont : **VCC**, **Trig**, **Echo** et **GND**.

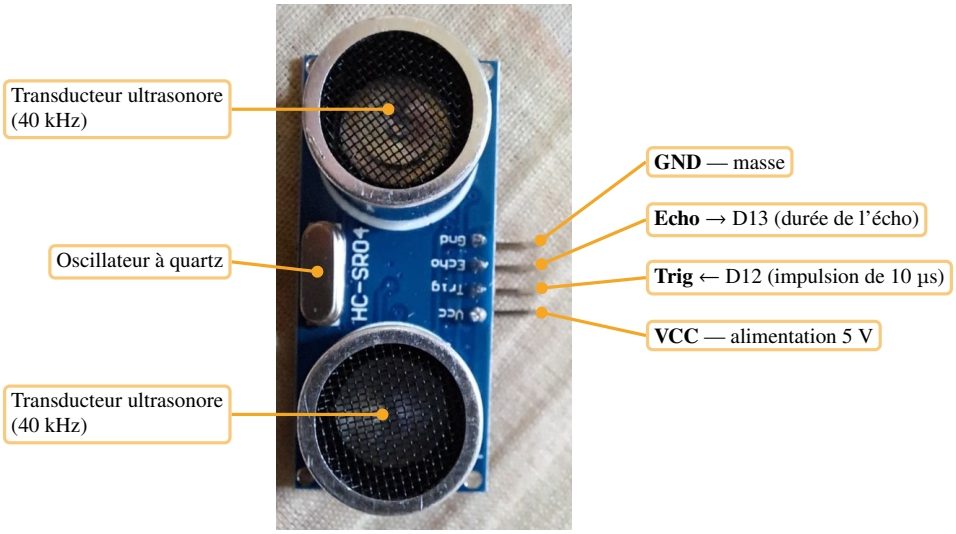


Figure 2.6. Capteur ultrasonique HC-SR04 et ses quatre broches

2.8 ESP32-CAM

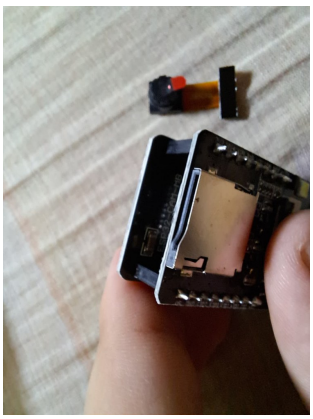
L'ESP32-CAM (modèle AI-Thinker) est un module très compact qui associe un microcontrôleur ESP32 à double cœur (240 MHz) doté du Wi-Fi et du Bluetooth, 4 Mo de mémoire PSRAM, un connecteur de caméra, un lecteur de carte microSD et une LED de flash [8]. Le module de notre prototype porte un capteur OV3660 ; il ne possède pas de port USB et se programme à travers sa carte support **ESP32-CAM-MB**, qui lui fournit une interface USB-série (CH340), des boutons de démarrage et de réinitialisation, et l'alimentation 5 V (figure 2.7).



(a) Module ESP32-CAM et sa caméra



(b) Carte support ESP32-CAM-MB



(c) Assemblage des deux cartes

FIGURE 2.7. Module ESP32-CAM et son support ESP32-CAM-MB

La caméra exécute le programme d'exemple *CameraWebServer* d'Espressif. Celui-ci se connecte au réseau Wi-Fi local et expose deux services HTTP : un flux vidéo MJPEG sur le port 81 (`/stream`) et une interface de réglage sur le port 80 (`/status`, `/control`) pour la résolution, la qualité JPEG, le flash, le retournement ou le miroir de l'image. Nous utilisons la résolution VGA (640×480).

Deux particularités du montage ont eu des conséquences sur le logiciel :

— **L'adresse IP est attribuée dynamiquement** par le point d'accès. Le serveur retrouve donc la caméra automatiquement (dernière adresse connue, puis balayage du sous-réseau), comme décrit au chapitre 4.

— **La caméra est montée sur la tranche**, à l'avant droit de la voiture : l'image arrive tournée de 90°. Le capteur ne sait que retourner ou inverser l'image ; la rotation est donc réalisée par le serveur avant l'affichage et la détection.

2.9 Système d'alimentation

L'alimentation du prototype est volontairement simple et repose sur deux sources séparées, sans circuit de régulation supplémentaire (figure 2.8) :

— **quatre piles alcalines AA** de 1,5 V (Duracell), soit environ 6 V, reliées à l'entrée VS du L298N à travers un interrupteur marche/arrêt : elles alimentent uniquement les moteurs ;

— **une batterie externe USB 5 V**, qui alimente la caméra ESP32-CAM par sa carte MB ainsi que l'électronique de commande. Le rail logique 5 V et la masse sont communs à l'Arduino, au L298N, au HC-SR04 et aux deux capteurs infrarouges.



(a) Support de 4 piles AA



(b) Interrupteur marche / arrêt

Figure 2.8. Éléments du circuit d'alimentation des moteurs

Cette séparation est volontaire. Au démarrage et lors des changements de sens, les moteurs appellent des pointes de courant qui font chuter la tension ; partagée avec la caméra, cette chute provoquerait des redémarrages de l'ESP32 (*brownout*), particulièrement sensible pendant les émissions Wi-Fi. Le tableau 2.6 donne des ordres de grandeur des consommations.

Table 2.6. Ordres de grandeur des consommations (valeurs typiques des fiches techniques)

Élément	Source	Consommation typique
2 moteurs TT	piles AA (via L298N)	2×100 à 250 mA en roulage, davantage au démarrage
Arduino UNO	batterie USB	≈ 50 mA
HC-SR04	rail 5 V (Arduino)	≈ 15 mA
2 capteurs IR	rail 5 V (Arduino)	$2 \times \approx 20$ mA
ESP32-CAM (diffusion Wi-Fi)	batterie USB	≈ 200 à 300 mA, plus avec le flash

Cette solution a l'avantage de la simplicité : les piles se remplacent facilement et la batterie externe se recharge par USB. Compte tenu de la chute de tension du L298N, une batterie lithium de 7,4 V permettrait toutefois aux moteurs de recevoir une tension plus proche de leur valeur nominale et à la voiture d'aller plus vite.

2.10 Schéma de câblage

Le schéma de câblage complet est donné par la figure 2.9, et le tableau 2.7 en reprend les liaisons.

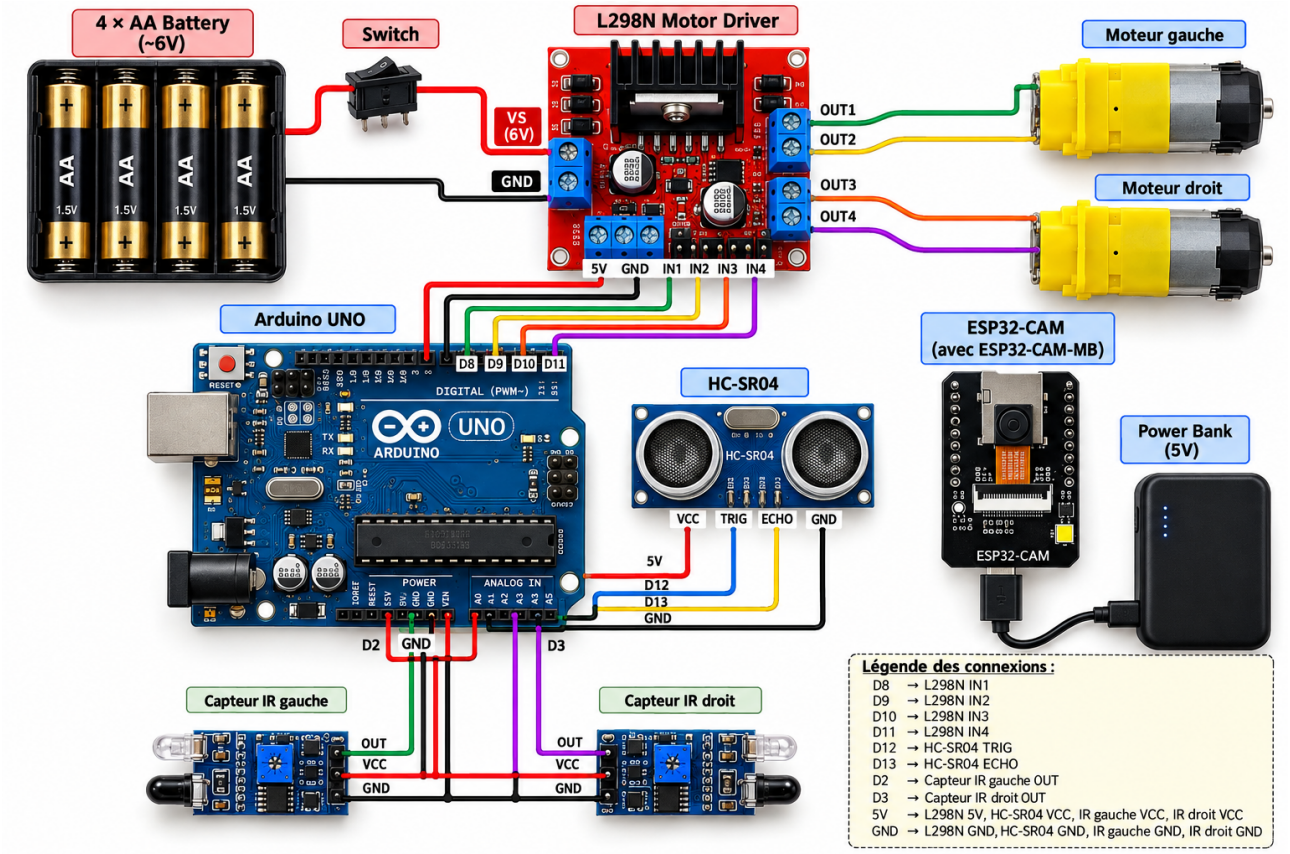


Figure 2.9. Schéma de câblage du prototype

Table 2.7. Liaisons du schéma de câblage

De	Vers
Piles AA (+) → interrupteur	L298N VS (6 V)
Piles AA (−)	L298N GND
L298N OUT1, OUT2 / OUT3, OUT4	Moteur gauche / moteur droit
Arduino D8, D9, D10, D11	L298N IN1, IN2, IN3, IN4
Arduino D12 / D13	HC-SR04 Trig / Echo
Arduino D2 / D3	Sortie OUT du capteur IR gauche / droit
Arduino 5 V	L298N 5 V, HC-SR04 VCC, VCC des capteurs IR
Arduino GND	L298N GND, HC-SR04 GND, GND des capteurs IR
Batterie externe (USB 5 V)	ESP32-CAM-MB → ESP32-CAM ; Arduino UNO (USB)

2.11 Réalisation physique du prototype

Le prototype est monté sur un châssis en acrylique à deux roues motrices et une roue folle (figure 2.10). Les composants sont disposés pour garder un centre de gravité bas et laisser à la caméra un champ de vision dégagé (figure 2.11) :

— à l'avant, le HC-SR04 est fixé au centre du pare-chocs, encadré par les deux capteurs infrarouges orientés vers le sol, à quelques millimètres de la piste ;

— l'ESP32-CAM, montée sur la tranche avec sa carte MB, occupe l'angle avant ; sa caméra regarde vers l'avant, à hauteur de la piste, ce qui donne le point de vue d'un conducteur ;

— l'Arduino UNO est placé au centre du châssis, l'interrupteur juste derrière ;

— à l'arrière, la batterie externe porte le module L298N, au plus près des moteurs.

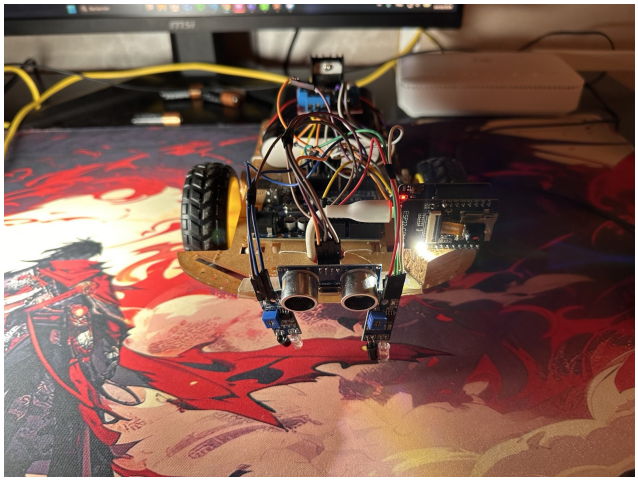


(a) Châssis en acrylique

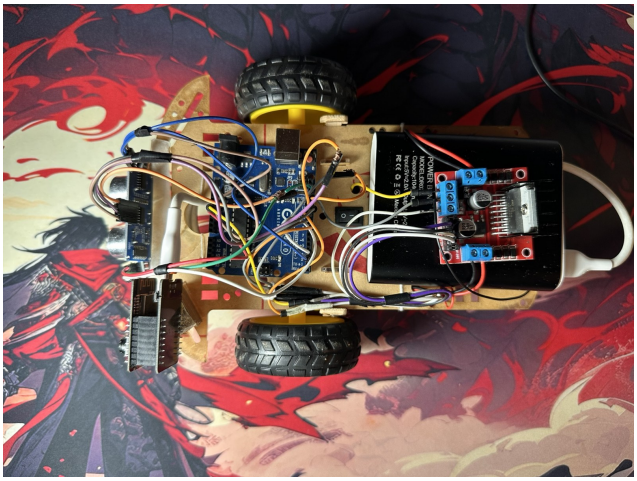


(b) Fils de liaison Dupont

FIGURE 2.10. Éléments mécaniques et de liaison



(a) Vue avant : HC-SR04, capteurs IR et ESP32-CAM



(b) Vue de dessus : Arduino, L298N et batterie externe

FIGURE 2.11. Prototype assemblé

Logique de commande embarquée

Le programme de l'Arduino combine le suivi de ligne et la sécurité frontale. À chaque itération, il mesure la distance frontale ; si un obstacle est trop proche, il arrête les moteurs, sinon il applique la logique de suivi de ligne du tableau 2.5. L'algorithme 1 en résume le principe.

Algorithme 1 Principe de la boucle de commande de l'Arduino

```
répéter
    d ← distance mesurée par le HC-SR04
    si d < d_sécurité alors
        arrêter les deux moteurs                          ▷ obstacle devant la voiture
    sinon
        g ← état du capteur IR gauche,  r ← état du capteur IR droit
        si g = sol clair et r = sol clair alors
            avancer
        sinon si g = ruban alors
            tourner à gauche                              ▷ moteur gauche ralenti
        sinon si r = ruban alors
            tourner à droite                              ▷ moteur droit ralenti
        sinon
            s'arrêter
        fin si
    fin si
fin répéter
```

Piste d'essai

La piste est une boucle fermée tracée au ruban adhésif noir sur un carrelage clair, aux angles arrondis (figure 2.12). Ses dimensions ne sont pas figées : la piste est retracée à chaque séance d'essai selon l'espace disponible, et sa géométrie peut être redéfinie dans l'éditeur de piste de l'application. Deux obstacles y sont placés : une **colonne** rayée à sommet noir et une **boîte** de couleur crème. Ce sont les deux classes que le modèle du prototype doit reconnaître (chapitre 3). Les six points de passage A à F de la piste, définis dans l'application, peuvent recevoir des marqueurs ArUco imprimés que la caméra lit pour localiser la voiture.



(a) Vue de dessus de la piste et des deux obstacles



(b) Obstacle « colonne »

FIGURE 2.12. Piste d'essai et obstacles du prototype

Conclusion du chapitre

Ce chapitre a présenté la réalisation matérielle du prototype : une architecture en deux sous-systèmes indépendants, l'un pour la commande (Arduino, L298N, moteurs, capteurs infrarouges et ultrasonique), l'autre pour la vision (ESP32-CAM). Le prototype suit sa piste de façon autonome et fournit, par sa caméra, les images que les modèles d'intelligence artificielle du chapitre suivant vont analyser.

Chapitre 3 : Conception et développement du modèle d'intelligence artificielle

Introduction du chapitre

Ce chapitre présente la partie « intelligence » du système : les modèles de vision par ordinateur qui détectent les obstacles dans les images. Après un rappel des principes de la vision par ordinateur et de la détection d'objets, nous présentons la famille de modèles YOLO26 et l'environnement Ultralytics. Nous décrivons ensuite la préparation des deux jeux de données (routes réelles et piste du prototype), le développement et l'entraînement des modèles, l'exploitation de leurs prédictions (localisation, distance, gravité, priorité) et enfin l'évaluation de leurs performances.

3.1 Introduction à la vision par ordinateur

3.1.1 Principes de la vision par ordinateur

La vision par ordinateur vise à extraire automatiquement de l'information d'images ou de vidéos. Pour une machine, une image couleur est un tableau de nombres de taille $H \times W \times 3$ (hauteur, largeur, trois canaux rouge, vert et bleu). Les méthodes classiques reposaient sur des descripteurs conçus à la main (contours, histogrammes de gradients, couleurs). Depuis 2012, les **réseaux de neurones convolutifs** (CNN) apprennent eux-mêmes ces descripteurs à partir d'exemples annotés : les premières couches détectent des motifs simples (bords, textures), les couches profondes des structures de plus en plus abstraites (roues, visages, objets entiers).

Une chaîne de vision comporte généralement les étapes de la figure 3.1 : acquisition de l'image, prétraitement (redimensionnement, normalisation), inférence du modèle, post-traitement des prédictions et exploitation du résultat par l'application.

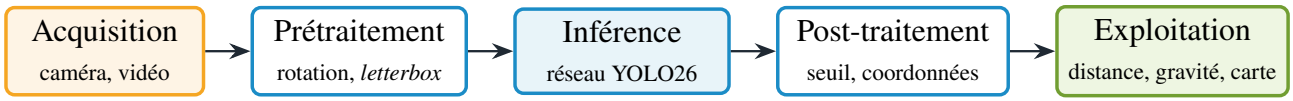


Figure 3.1. Chaîne de traitement d'un système de vision par ordinateur

3.1.2 Détection d'objets et segmentation d'images

On distingue plusieurs tâches de complexité croissante :

— la **classification**, qui attribue une étiquette à l'image entière (« il y a un cône ») ;

— la **détection d'objets**, qui localise chaque objet par une *boîte englobante* (x_1, y_1, x_2, y_2) et lui associe une classe et un score de confiance ;

— la **segmentation**, qui attribue une classe à chaque pixel. La segmentation d'instances distingue en plus les objets individuels et fournit leur contour exact (*masque*).

La détection d'objets est la tâche adaptée à notre besoin : elle indique quels obstacles sont présents, où ils se trouvent dans l'image et avec quelle confiance, tout en restant assez rapide pour le temps réel. Nous avons cependant utilisé la segmentation à une autre étape : pour *annoter* automatiquement les images de notre piste, le modèle de segmentation SAM 2 [9] a produit le masque de chaque obstacle, dont la boîte englobante se déduit directement (figure 3.2).

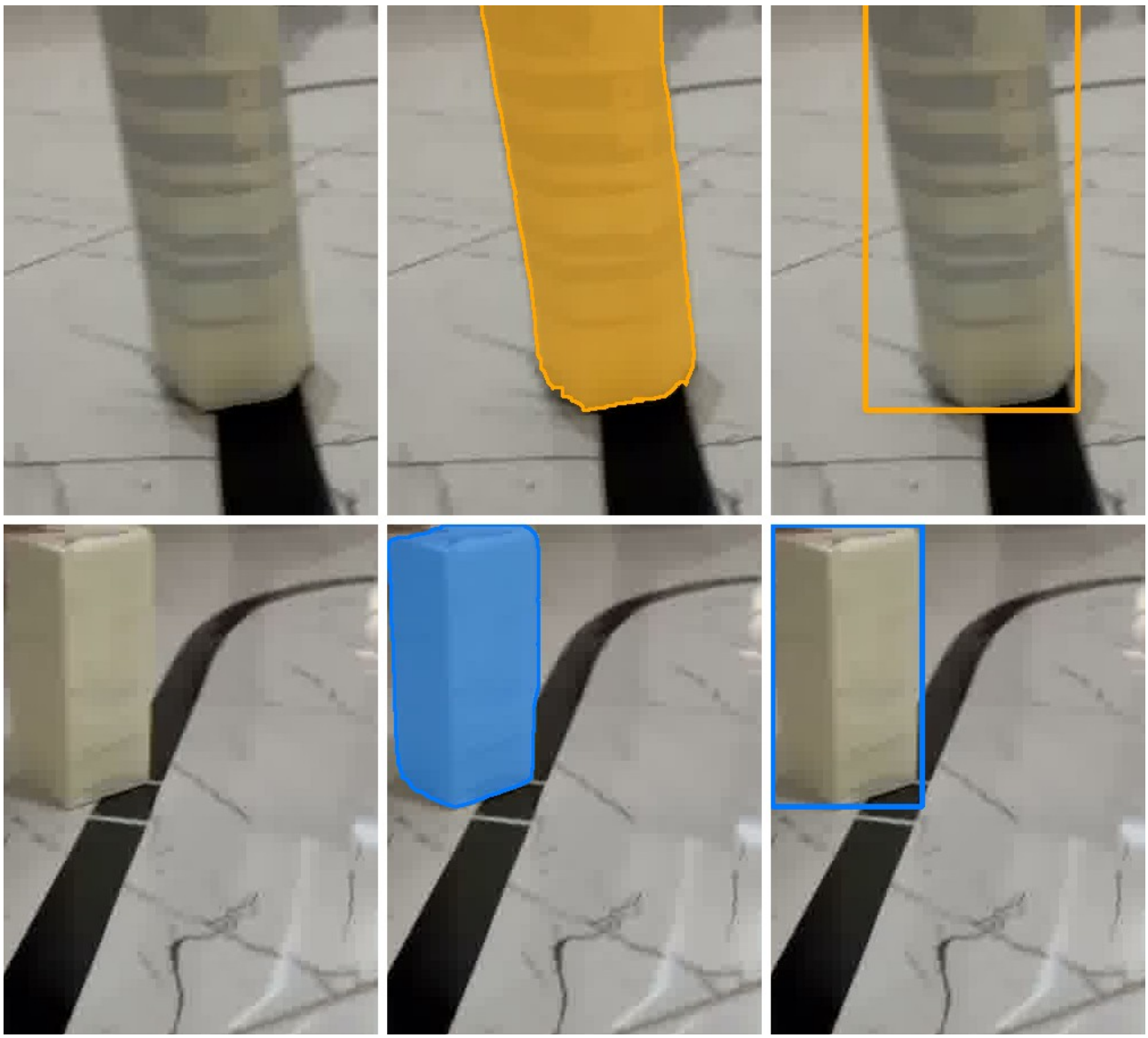


Figure 3.2. Image d'origine, segmentation (masque produit par SAM 2) et détection (boîte englobante déduite du masque) pour les deux obstacles de la piste

3.1.3 Application à la détection des obstacles routiers et des nids-de-poule

Sur la route, deux familles d'obstacles doivent être reconnues. Les **usagers de la route** (piétons, cyclistes, véhicules, animaux) sont des objets bien représentés dans les grands jeux de données publics : un modèle préentraîné les reconnaît déjà. Les **anomalies routières** (nids-de-poule, cônes, barrières, panneaux de chantier) sont plus spécifiques et nécessitent un apprentissage dédié. Les nids-de-poule sont les plus difficiles : ils sont plats, de forme et de taille très variables, souvent remplis d'eau ou peu contrastés avec la chaussée. La figure 3.3 montre les prédictions de nos deux modèles routiers sur des

images de test.



Figure 3.3. Détections des modèles routiers (usagers de la route et anomalies) sur des images de l'ensemble de test, aux seuils de l'application. Les erreurs restent visibles : un nid-de-poule manqué (en haut au centre), un panneau STOP pris pour un panneau de chantier (en bas à droite).

3.2 Présentation des modèles YOLO26

3.2.1 Généralités sur la famille YOLO

Les détecteurs d'objets se répartissent en deux familles. Les détecteurs *à deux étapes* (R-CNN, Faster R-CNN) proposent d'abord des régions candidates, puis les classent : ils sont précis mais lents. Les détecteurs *à une étape* prédisent directement les boîtes et les classes en une seule passe du réseau. YOLO (*You Only Look Once*), introduit par Redmon et al. en 2016 [10], est le représentant le plus connu de cette seconde famille : il découpe l'image en une grille et prédit, pour chaque cellule, des boîtes et des probabilités de classes.

Depuis, la famille a beaucoup évolué (tableau 3.1) : ancres et prédiction multi-échelle (YOLOv3), augmentation *mosaic* (YOLOv4 [11]), têtes sans ancres (YOLOv8), entraînement sans NMS (YOLOv10 [12]) et, avec YOLO26, une architecture pensée pour le déploiement sur processeurs et équipements embarqués [13].

Table 3.1. Quelques étapes de l'évolution de la famille YOLO

Version	Année	Apport principal
YOLO (v1)	2016	Détection en une seule passe sur une grille
YOLOv3	2018	Prédictions à trois échelles, ancres
YOLOv4 / v5	2020	Augmentation *mosaic*, écosystème PyTorch (Ultralytics)
YOLOv8	2023	Tête sans ancres, API unifiée (détection, segmentation, pose)
YOLOv10	2024	Affectation un-à-un : inférence sans NMS
YOLO11	2024	Blocs C3k2 et attention C2PSA
YOLO26	2025	Inférence de bout en bout sans NMS, suppression de la DFL, optimisé CPU / embarqué

3.2.2 Présentation de YOLO26 et de son architecture

YOLO26 est la génération de détecteurs publiée par Ultralytics en 2025 [13]. Elle est déclinée en cinq tailles : *nano* (n), *small* (s), *medium* (m), *large* (l) et *extra-large* (x). Comme ses prédécesseurs, le réseau comporte trois parties (figure 3.4) :

— un **backbone** (extracteur de caractéristiques) convolutif qui réduit progressivement la résolution et produit des cartes de caractéristiques à plusieurs échelles (P3, P4, P5, soit 1/8, 1/16 et 1/32 de l'image) ;

— un **neck** de type FPN/PAN [14, 15] qui combine ces échelles, pour détecter à la fois les petits objets lointains et les grands objets proches ;

— une **tête de détection** qui prédit, pour chaque position, la boîte et les scores de classes.

Ses principales nouveautés visent le déploiement : la tête est **de bout en bout** (affectation un-à-un) et produit directement au plus 300 détections finales, **sans post-traitement NMS** ; la *Distribution Focal Loss* (DFL) est supprimée, ce qui simplifie l'export ; l'entraînement utilise une pondération progressive des pertes (ProgLoss) et une affectation des étiquettes favorable aux petits objets (STAL). Exporté en ONNX, le modèle produit un unique tenseur de sortie de taille $1 \times 300 \times 6$: pour chaque détection, (x_1, y_1, x_2, y_2), le score et la classe. Notre modèle du prototype (YOLO26n, deux classes) compte 2,39 millions de paramètres pour 5,4 GFLOPs.

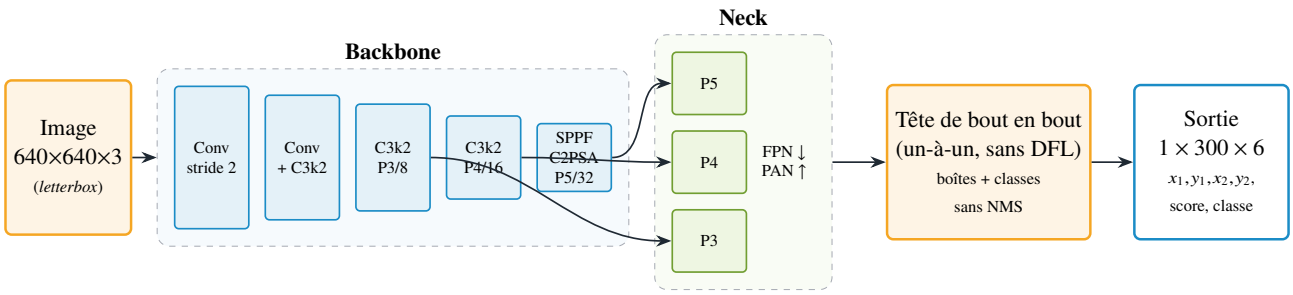


Figure 3.4. Architecture simplifiée d'un détecteur YOLO26 : backbone, neck multi-échelle et tête de bout en bout sans NMS

Le choix des variantes résulte d'un compromis entre précision et vitesse, l'inférence se faisant sur le processeur de l'ordinateur : la variante **nano** pour les usagers de la route et pour le prototype, la variante **small**, un peu plus lourde, pour les anomalies routières, plus difficiles à reconnaître.

3.2.3 Utilisation des poids préentraînés sur COCO

Entraîner un détecteur à partir de zéro demande des centaines de milliers d'images annotées. Nous utilisons l'**apprentissage par transfert** : les modèles partent des poids publiés par Ultralytics, préentraînés sur COCO [16] (118 287 images d'entraînement, 80 classes d'objets courants). Le backbone sait déjà extraire des caractéristiques visuelles générales ; l'affinage (*fine-tuning*) adapte surtout la tête aux nouvelles classes (figure 3.5).

Pour les usagers de la route, aucun réentraînement n'est nécessaire : les classes COCO *person*, *bicycle*, *car*, *bus*, *truck*, *dog*, etc. sont directement regroupées en quatre catégories utiles au véhicule (tableau 3.3).

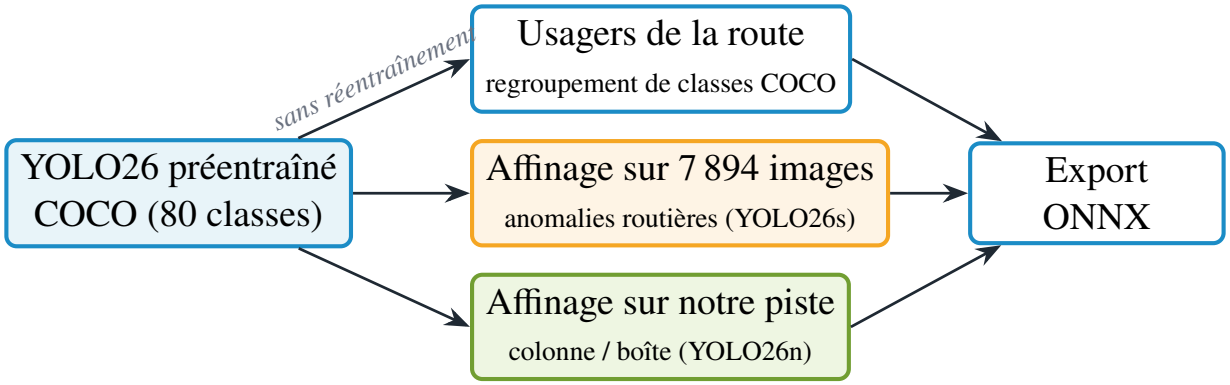


FIGURE 3.5. Apprentissage par transfert à partir des poids COCO

3.2.4 Environnement Ultralytics

La bibliothèque Python **Ultralytics** [13] fournit les modèles YOLO26 et toute la chaîne de travail : chargement des jeux de données décrits par un fichier `data.yaml`, entraînement avec augmentation de données, validation (précision, rappel, mAP, courbes et matrice de confusion), prédiction et export vers de nombreux formats (ONNX, TensorRT, TFLite. . .). L'entraînement a été réalisé dans un **conteneur Docker** basé sur l'image officielle `ultralytics/ultralytics` (Ultralytics 8.4.184, PyTorch avec CUDA), complétée par JupyterLab, ce qui garantit un environnement identique d'une machine à l'autre. Les notebooks `train_road_obstacles.ipynb` et `train_toy_obstacles.ipynb` s'exécutent sur ce noyau et utilisent le GPU de l'ordinateur (NVIDIA GeForce GTX 1650 Ti, 4 Go). La figure 3.6 résume le flux de travail.

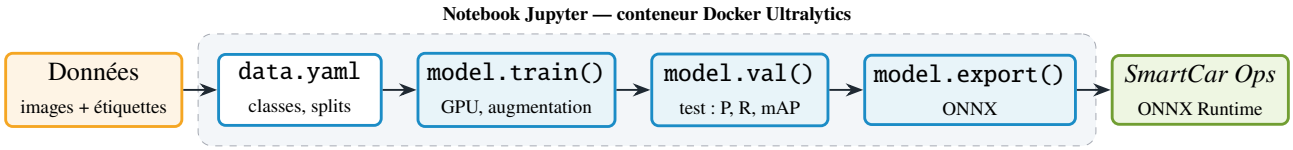


FIGURE 3.6. Flux de travail avec Ultralytics, de la préparation des données à l'intégration

3.2.5 Adaptation du modèle aux besoins du projet

Les besoins du projet ont conduit à trois modèles complémentaires (tableau 3.2). En environnement réel, deux modèles s'exécutent en parallèle sur chaque image et leurs détections sont fusionnées : l'un apporte la connaissance générale de COCO sur les usagers de la route, l'autre la connaissance spécialisée des anomalies. Pour le prototype, un modèle nano dédié reconnaît les deux obstacles de la piste. Les classes de chaque modèle sont traduites vers un vocabulaire commun à l'application (`pedestrian`, `vehicle`, `pothole`, `cone`, `box`. . .), auquel sont associés les poids de danger utilisés pour la gravité.

Table 3.2. Les trois modèles du projet

Modèle	Variante	Entraînement	Classes détectées
Usagers de la route	YOLO26n	préentraîné COCO	piéton, cycliste, véhicule, animal
Anomalies routières	YOLO26s	affiné 8 h, 36 époques	nid-de-poule, cône, barrière, panneau de chantier
Obstacles du prototype	YOLO26n	affiné 30 min, 73 époques	colonne, boîte

3.3 Préparation des données

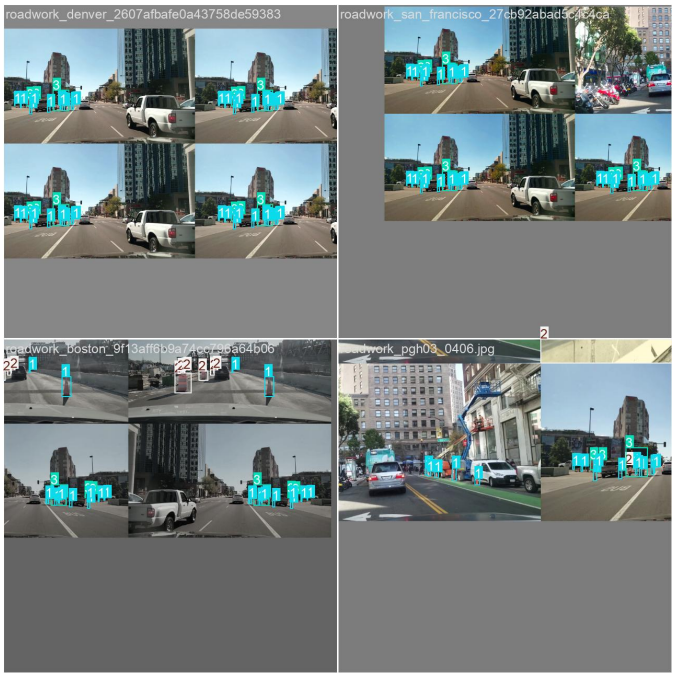
3.3.1 Présentation des données réelles

Le modèle d'anomalies routières a été entraîné sur la réunion de deux jeux de données publics d'images de caméras embarquées :

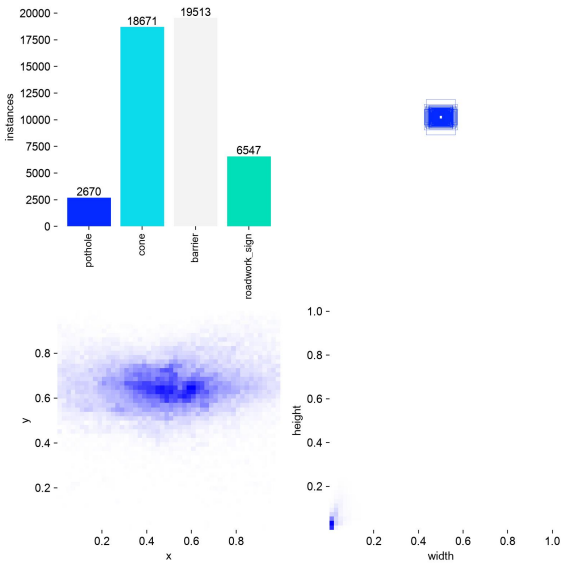
— **Potholes under challenging conditions** [17] (Bučko et al., *Sensors*, 2022, licence CC BY 4.0) : des images de nids-de-poule extraites de vidéos de conduite en Slovaquie, dans des conditions difficiles (pluie, ombres, nuit, chaussée mouillée) ;

— **ROADWork** [18] (Ghosh et al., ICCV 2025, Carnegie Mellon University, licence ODC-By 1.0) : des images de zones de travaux dans plusieurs villes, avec des cônes, des barrières et des panneaux de signalisation temporaire.

Au total, **7 894 images** réelles ont été retenues. Les annotations d'origine ont été converties au format YOLO et leurs nombreuses sous-catégories regroupées en quatre classes. La figure 3.7 montre des images d'entraînement et la répartition des étiquettes.



(a) Lot d'entraînement (mosaïques augmentées)



(b) Statistiques des étiquettes

Figure 3.7. Données réelles : exemples d'images d'entraînement et distribution des annotations

Les vidéos qui alimentent la caméra embarquée de CAR-01 dans l'application proviennent de sources libres (Wikimedia Commons, licences CC BY, CC BY-SA ou domaine public) et des images de test du jeu Potholes : elles ne servent qu'à la démonstration, jamais à l'entraînement.

3.3.2 Présentation des données du prototype

Aucun jeu de données public ne contient notre piste et nos obstacles. Nous avons donc constitué le nôtre : une vidéo filmée à hauteur de voiture en suivant le ruban, dont nous avons extrait **249 images** (832×464 pixels). La vidéo contient des passages devant la colonne, devant la boîte (sous plusieurs faces, dont celle portant le logo) et des portions de piste vide (figure 3.8). Une photo de la piste vue de dessus et une photo de chaque obstacle ont complété ces images pour définir précisément les classes.



Figure 3.8. Données du prototype : images extraites de la vidéo (passage devant la colonne, piste vide, boîte)

3.3.3 Organisation et annotation des images

Les deux jeux de données suivent l'organisation standard d'Ultralytics : un dossier `images/` et un dossier `labels/`, chacun divisé en `train`, `val` et `test`. Chaque image possède un fichier texte contenant une ligne par objet :

```
<classe> <x_centre> <y_centre> <largeur> <hauteur>
```

où les coordonnées sont normalisées entre 0 et 1. Une image sans objet a un fichier vide : elle sert d'exemple négatif (« rien à détecter ici »).

Pour les données du prototype, l'annotation manuelle de chaque image aurait été longue. Nous avons utilisé le **suivi vidéo de SAM 2** (*Segment Anything Model 2*) [9] : pour chaque séquence où un obstacle est visible, un seul clic sur l'obstacle dans une image de référence suffit ; SAM 2 propage ensuite le masque dans toute la séquence, en avant et en arrière (figure 3.9). Un filtre élimine les pixels très sombres pour ne pas inclure le ruban noir sous la colonne. Les boîtes sont déduites des masques et **toutes ont été vérifiées visuellement** sur des planches de contrôle. On obtient 248 images annotées (une image floue a été écartée) contenant 141 boîtes.

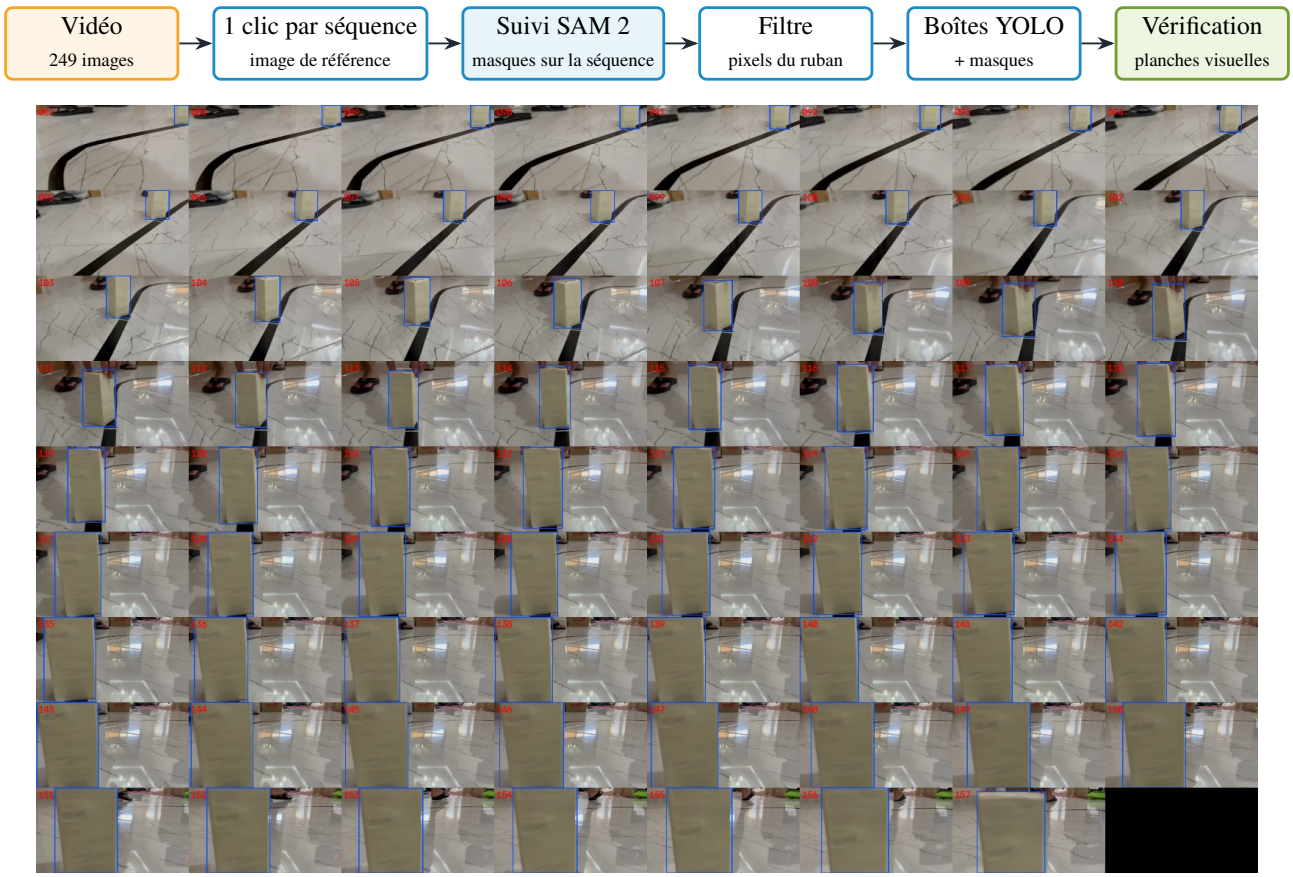


Figure 3.9. Annotation automatique par suivi vidéo avec SAM 2 : chaîne de traitement et planche de contrôle d'une séquence de la boîte

3.3.4 Définition des classes à reconnaître

Les classes ont été définies en fonction de l'usage : chacune doit correspondre à une réaction différente du véhicule ou à un niveau de danger différent (tableaux 3.3 et 3.4).

Table 3.3. Classes du modèle des usagers de la route (regroupement des classes COCO)

Classe de l'application	Classes COCO regroupées
`pedestrian` (piéton)	person
`cyclist` (cycliste)	bicycle
`vehicle` (véhicule)	car, motorcycle, bus, truck, train
`animal`	dog, cat, horse, cow, sheep, bird

Table 3.4. Classes des modèles entraînés

Modèle	Classe	Contenu
Anomalies	`pothole`	nids-de-poule
	`cone`	cônes et balises tubulaires
	`barrier`	fûts, barricades, barrières, clôtures, panneaux verticaux
	`roadwork_sign`	panneaux de chantier, flèches lumineuses, panneaux à messages
Prototype	`column`	colonne rayée à sommet noir
	`box`	boîte de couleur crème (logo rouge sur une face)

3.3.5 Prétraitement et augmentation des données

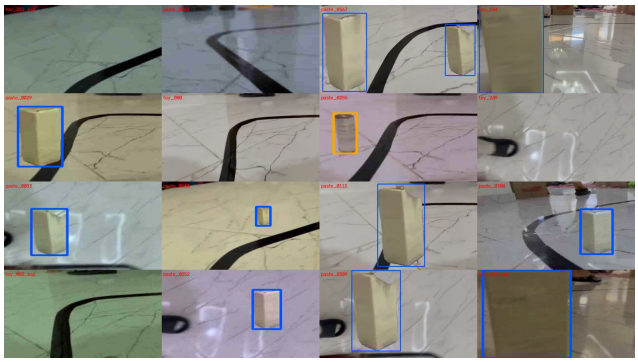
Toutes les images sont redimensionnées en conservant leurs proportions et complétées par des bandes (*letterbox*) jusqu'à la taille d'entrée du réseau : 640×640 pour le modèle routier, 320×320 pour le prototype, résolution proche de celle de l'ESP32-CAM.

Pendant l'entraînement, Ultralytics applique des augmentations aléatoires qui multiplient la variété des exemples : assemblage de quatre images en mosaïque (désactivé pendant les 10 dernières époques), variations de teinte, de saturation et de luminosité, translations, changements d'échelle (±50 %) et symétries horizontales.

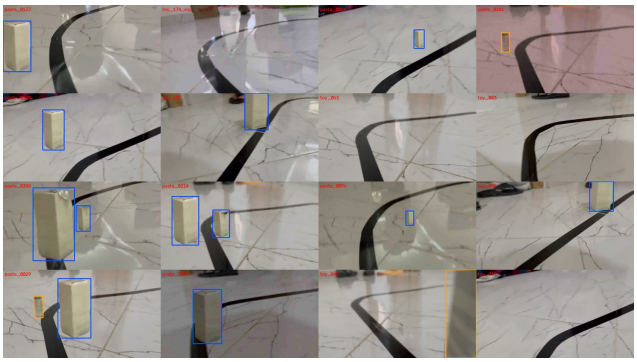
Pour le prototype, deux augmentations spécifiques ont été développées :

— **l'aspect ESP32-CAM** : la vidéo a été filmée avec un téléphone en 16 :9, alors que l'ESP32-CAM fournit des images 4 :3 de moindre qualité. 60 % des images d'entraînement sont donc converties en images « style ESP32-CAM » : recadrage aléatoire au format 4 :3, réduction à 320×240, flou, dominante de couleur, bruit et forte compression JPEG ; les boîtes sont recalculées et un obstacle coupé à plus de 60 % par le recadrage est écarté (figure 3.10a) ;

— **le copier-coller** [19] : les obstacles, découpés grâce à leur masque SAM 2, sont collés sur des images de piste vide, à de nouvelles positions et distances (plus grands lorsqu'ils sont proches de la caméra), avec ajustement de la luminosité. 400 images supplémentaires sont ainsi créées, avec des boîtes exactes (figure 3.10b).



(a) Images converties au style ESP32-CAM



(b) Images d'entraînement, dont copier-coller

Figure 3.10. Augmentations spécifiques au prototype

3.3.6 Division des données en ensembles d'entraînement, de validation et de test

Les données sont réparties en trois ensembles : l'**entraînement** sert à ajuster les poids, la **validation** à suivre l'apprentissage et à choisir les réglages, et le **test**, jamais utilisé avant l'évaluation finale, à mesurer les performances.

Une précaution est essentielle avec des images extraites de vidéos : deux images voisines sont presque identiques. Une répartition aléatoire placerait des quasi-doublons à la fois en entraînement et en test et gonflerait artificiellement les résultats. Nous avons donc découpé **dans le temps** :

— pour les nids-de-poule, chaque trajet est divisé en 80 / 10 / 10 % avec un intervalle de séparation ; pour ROADWork, la répartition géographique officielle est conservée (validation et test proviennent de lieux absents de l'entraînement) ;

— pour le prototype, chaque séquence de la vidéo est découpée en blocs temporels : un bloc pour la validation, un autre pour le test, le reste pour l'entraînement, avec des images d'intervalle exclues (figure 3.11).

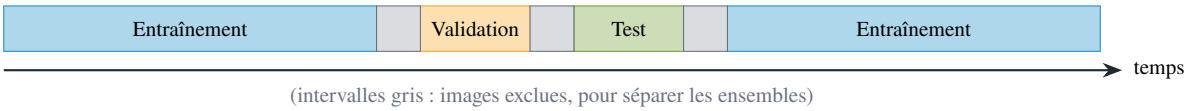


Figure 3.11. Découpage temporel d'une séquence vidéo du prototype

Table 3.5. Répartition des données

Ensemble	Entraînement	Validation	Test
Anomalies routières (images réelles)	6 324	755	815
objets annotés dans le test			5 105
Prototype — images réelles de la vidéo	169	26	26
+ images style ESP32-CAM	96	—	26
+ images par copier-coller	400	—	—

Le test du prototype comprend 26 images réelles (15 obstacles, 11 images de piste vide) et leurs 26 équivalents convertis au style ESP32-CAM 320×240, qui constituent un second ensemble de test plus proche des conditions réelles.

3.4 Développement des deux modèles

3.4.1 Modèle destiné aux données réelles

Le modèle d'anomalies routières est un YOLO26s affiné à partir des poids COCO sur les 6 324 images d'entraînement. Il est associé au modèle COCO des usagers de la route : les deux sont décrits dans un *profil* (`models/real_road.json`) qui indique, pour chacun, le fichier ONNX, la taille d'entrée, le seuil de confiance et la correspondance des classes. Les deux modèles s'exécutent en parallèle sur chaque image de la caméra embarquée.

3.4.2 Modèle destiné au prototype

Le modèle du prototype est un YOLO26n affiné sur les 665 images d'entraînement (169 réelles, 96 au style ESP32-CAM et 400 par copier-coller). La variante nano et l'entrée 320×320 permettent une analyse en temps réel du flux de la caméra sur le processeur. Son profil (`models/proto_toy.json`) est chargé automatiquement lorsque l'application est en mode Prototype.

3.4.3 Configuration des modèles et des poids préentraînés

Le tableau 3.6 résume la configuration d'entraînement. La carte graphique (GTX 1650 Ti, 4 Go) a imposé deux adaptations : la précision mixte (AMP) étant instable sur ce modèle de GPU, Ultralytics l'a désactivée et l'entraînement s'est fait en 32 bits ; la taille de lot a été réduite à 4 images pour le modèle *small* afin de tenir dans la mémoire. Pour le modèle routier, la durée a été fixée par un budget de temps de 8 heures (paramètre `time`) plutôt que par un nombre d'époques.

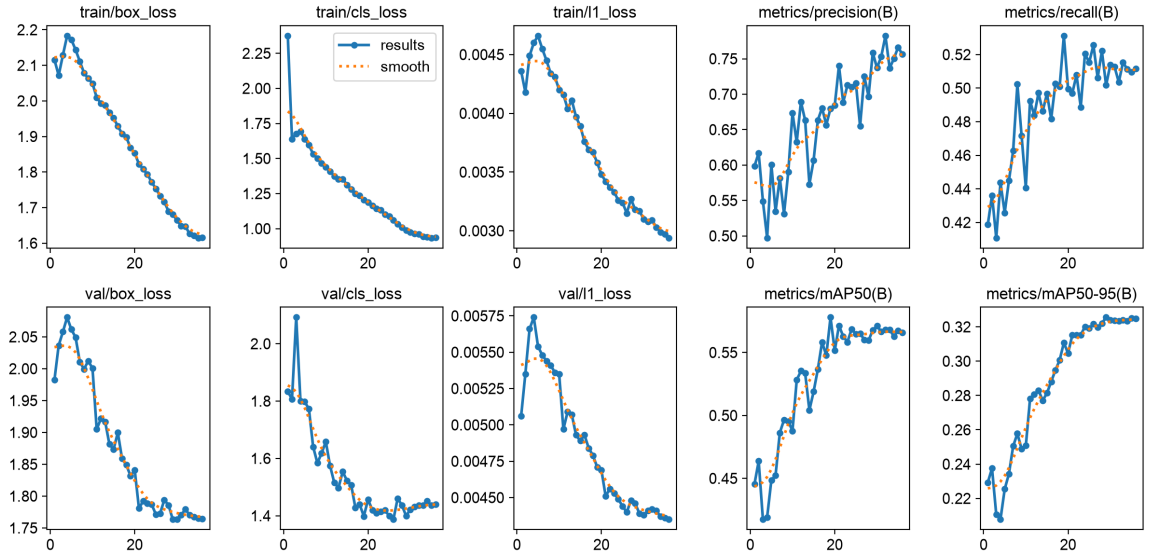
Table 3.6. Configuration d'entraînement des deux modèles

Paramètre	**Anomalies routières**	**Prototype**
Modèle de départ	`yolo26s.pt` (COCO)	`yolo26n.pt` (COCO)
Taille d'entrée	640×640	320×320
Taille de lot	4	16
Durée	budget de 8 h (36 époques)	80 époques max. (arrêt à 73)
Arrêt anticipé (*patience*)	100 époques	30 époques
Taux d'apprentissage	0,01, décroissance en cosinus	0,01, décroissance en cosinus
Mosaïque	oui, désactivée les 10 dernières époques	idem
Précision de calcul	FP32 (AMP désactivée)	FP32
Matériel	GTX 1650 Ti (4 Go)	GTX 1650 Ti (4 Go)
Durée réelle	8,0 h	30 min

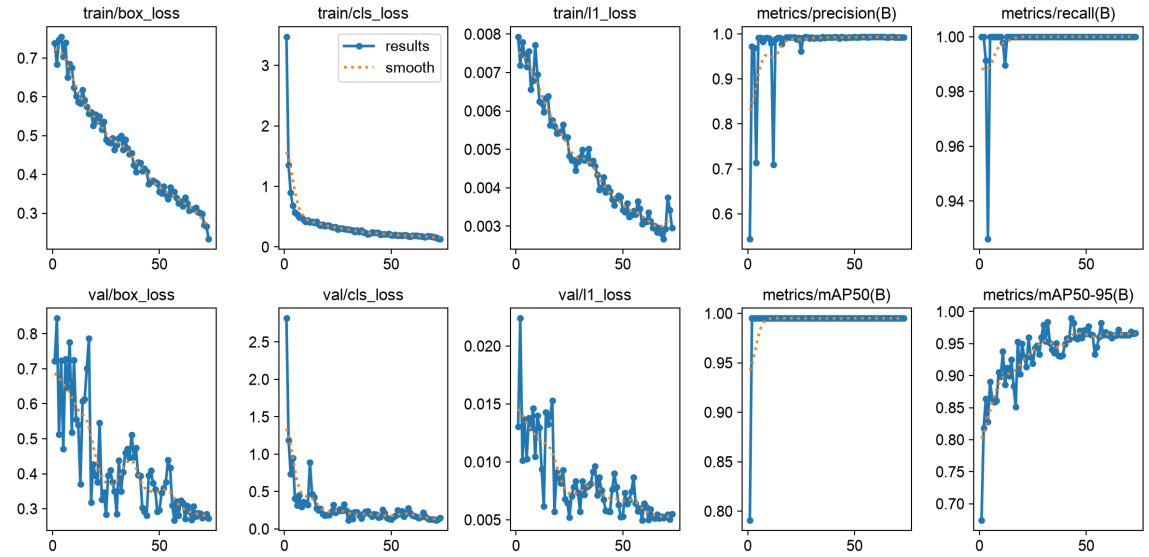
3.4.4 Procédure d'entraînement et de validation

À chaque époque, le modèle parcourt l'ensemble d'entraînement par lots ; pour chaque lot, il calcule une fonction de perte qui combine une perte de localisation (écart entre boîtes prédites et réelles, à base d'IoU) et une perte de classification, puis met à jour ses poids par descente de gradient. À la fin de chaque époque, il est évalué sur l'ensemble de validation et les meilleurs poids sont conservés (`best.pt`). Les notebooks affichent la progression en temps réel ; un notebook de suivi (`monitor_training.ipynb`) permet de consulter les courbes pendant un entraînement de longue durée.

La figure 3.12 présente les courbes d'apprentissage des deux modèles. Pour le modèle routier, les pertes diminuent régulièrement et le mAP de validation progresse encore à la fin du budget de 8 heures, ce qui indique qu'un entraînement plus long améliorerait encore le modèle. Pour le prototype, les métriques de validation atteignent rapidement leur maximum et l'arrêt anticipé intervient à la 73e époque.



(a) Modèle d'anomalies routières (YOLO26s, 36 époques)



(b) Modèle du prototype (YOLO26n, 73 époques)

Figure 3.12. Courbes d'apprentissage : pertes d'entraînement et de validation, précision, rappel et mAP par époque

3.4.5 Sauvegarde et chargement des modèles

À la fin de chaque notebook, les meilleurs poids sont exportés au format **ONNX** (`opset` 17, graphe simplifié) et copiés dans le dossier `models/` de l'application, avec les poids PyTorch (`.pt`) pour un éventuel réentraînement. Le notebook écrit aussi le profil du modèle et met à jour le registre `models/registry.json`, qui contient les métriques d'évaluation affichées dans la page *AI models*. L'exemple suivant montre le profil du modèle du prototype :

```
{ "name": "Toy track obstacles",
  "models": [ { "id": "toy-obstacles", "path": "models/toy_obstacles.onnx",
                "imgsz": 320, "conf": 0.4,
                "classes": { "column": "column", "box": "box" } } ] }
```

Au démarrage, l'application lit le profil de chaque environnement et charge les modèles avec ONNX Runtime ; un modèle absent est simplement signalé (« pas encore entraîné ») sans empêcher l'application de fonctionner. Le format ONNX rend l'application indépendante de PyTorch : elle n'a besoin que d'ONNX Runtime, beaucoup plus léger.

3.5 Détection et analyse des obstacles

3.5.1 Détection des objets et des anomalies routières

Pour chaque image, le modèle renvoie une liste de détections : la classe de l'objet, un score de confiance et une boîte englobante. En environnement réel, les deux modèles routiers analysent la même image et leurs détections sont réunies.

3.5.2 Localisation des objets dans les images

La boîte indique où se trouve l'objet dans l'image. L'application en déduit deux informations simples :

— **la distance** : plus un objet est proche, plus sa boîte est grande. Connaissant la taille habituelle de l'objet (environ 1,7 m pour un piéton, 0,7 m pour un cône), on estime sa distance à partir de la hauteur de sa boîte ; pour un nid-de-poule, objet plat, on utilise sa position dans l'image (plus il est bas, plus il est proche) ;

— **la position dans la voie** : l'obstacle est « sur la trajectoire » si sa boîte se trouve au centre de l'image, devant le véhicule, et « sur le côté » sinon.

3.5.3 Exploitation des scores de confiance

Le score de confiance, entre 0 et 1, indique la certitude du modèle. Les détections trop incertaines sont ignorées : le seuil est de 0,40 pour les usagers de la route et pour le prototype, et de 0,35 pour les anomalies routières afin de manquer moins de nids-de-poule. La confiance intervient aussi dans la gravité : une détection incertaine compte moins qu'une détection certaine.

3.5.4 Identification des nids-de-poule et des obstacles

Les **anomalies fixes** (nid-de-poule, cône, barrière, panneau) sont enregistrées avec leur position sur la carte et une photo. Les **usagers de la route** (piéton, cycliste, véhicule, animal) bougent : ils ne sont enregistrés que s'ils sont devant le véhicule et proches. Un même obstacle vu plusieurs fois n'est enregistré qu'une seule fois, avec un compteur d'observations. Enfin, un obstacle proche sur la trajectoire fait ralentir ou s'arrêter le véhicule.

3.5.5 Estimation de la gravité des anomalies

Chaque obstacle reçoit un score de gravité de 0 à 100, qui combine quatre éléments :

$$\text{score} = 100 \times \underbrace{w}_{\text{danger de la classe}} \times \underbrace{(0{,}5 + 0{,}5\,p)}_{\text{proximité}} \times \underbrace{c}_{\text{confiance}} \times \underbrace{t}_{\text{trajectoire}} \tag{3.1}$$

où w est le poids de danger de la classe (1 pour un piéton, 0,6 pour un nid-de-poule, 0,4 pour un cône. . .), p la proximité entre 0 (loin) et 1 (très proche), c la confiance du modèle et t vaut 1 si l'obstacle est sur la trajectoire et 0,5 s'il est sur le côté. Les poids de danger sont réglables dans l'application.

3.5.6 Attribution d'un niveau de priorité

Le score donne l'un des quatre niveaux de priorité du tableau 3.7. Les obstacles sont ensuite triés du plus grave au moins grave, pour que l'opérateur traite d'abord les plus dangereux.

Table 3.7. Niveaux de priorité

Niveau	Score	Signification
■ Critique	≥ 60	danger immédiat, à traiter en priorité
■ Élevé	40 – 60	danger important
■ Moyen	20 – 40	à surveiller
■ Faible	< 20	information

Par exemple, un piéton proche ($p = 0{,}6$), détecté à 90 % sur la trajectoire, obtient $100 \times 1 \times 0{,}8 \times 0{,}9 \times 1 = 72$: il est **critique**. Un cône sur le côté, dans les mêmes conditions, obtient $100 \times 0{,}4 \times 0{,}8 \times 0{,}9 \times 0{,}5 = 14{,}4$: il est **faible**.

3.6 Évaluation des performances

3.6.1 Méthodologie d'évaluation

Toutes les métriques finales sont mesurées sur les **ensembles de test**, jamais utilisés pendant l'entraînement ni pour choisir les réglages. Pour le modèle routier, le réglage d'inférence (taille d'entrée, augmentation au moment du test) a été choisi sur l'ensemble de validation, puis appliqué une seule fois au test. Nous indiquons toujours le réglage correspondant à chaque résultat. Ultralytics donne la précision et le rappel au seuil de confiance qui maximise le score F1 moyen ; les matrices de confusion sont calculées au seuil réellement utilisé dans l'application.

3.6.2 Précision, rappel et score F1

Une détection est un **vrai positif** (TP) si elle correspond à un objet réel de la même classe, un **faux positif** (FP) sinon ; un objet réel non détecté est un **faux négatif** (FN). On définit :

$$\text{Précision} = \frac{TP}{TP + FP}, \qquad \text{Rappel} = \frac{TP}{TP + FN}, \qquad F_1 = 2 \cdot \frac{\text{Précision} \times \text{Rappel}}{\text{Précision} + \text{Rappel}} \tag{3.2}$$

La précision mesure la fiabilité des alertes (peu de fausses alarmes), le rappel la capacité à ne pas manquer d'obstacles. Pour un véhicule, un rappel élevé est souvent prioritaire, mais trop de fausses alarmes rendraient le système inutilisable : le score F1 résume ce compromis.

3.6.3 Intersection sur l'union (IoU) et mAP

Pour décider si une boîte prédite correspond à un objet réel, on mesure leur recouvrement par l'**intersection sur l'union** (figure 3.13) :

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{3.3}$$

Une prédiction est correcte si son IoU avec un objet réel dépasse un seuil (0,5 en général).

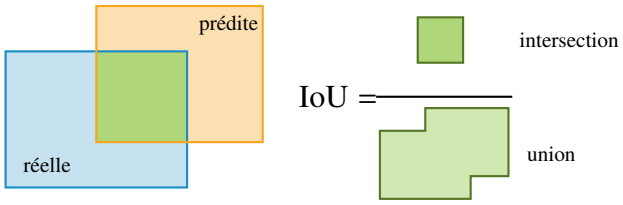


Figure 3.13. Intersection sur l'union entre une boîte réelle et une boîte prédite

En faisant varier le seuil de confiance, on obtient une courbe précision-rappel ; l'aire sous cette courbe est la **précision moyenne** (AP) d'une classe. Le **mAP** est la moyenne des AP sur toutes les classes [20, 21] :

— **mAP@50** : IoU minimale de 0,5 ;

— **mAP@50-95** : moyenne des mAP pour des seuils d'IoU de 0,50 à 0,95 par pas de 0,05, plus exigeante sur la précision de localisation.

3.6.4 Temps d'inférence et vitesse de traitement

La vitesse est mesurée à deux niveaux (tableau 3.8). Sur le GPU d'entraînement, Ultralytics mesure le temps d'inférence pur par image. Dans l'application, les modèles ONNX s'exécutent sur le processeur de l'ordinateur ; le temps dépend alors de la charge de la machine, car l'application, les simulateurs et le flux vidéo partagent le même processeur.

Table 3.8. Temps d'inférence mesurés

Modèle et conditions	Temps par image	Cadence
Anomalies routières, GPU, 640 px	10 ms	—
Anomalies routières, GPU, 960 px + TTA	50 ms	—
Prototype, GPU, 320 px	22 ms (30 ms en test ESP32)	—
Application, CPU : usagers + anomalies en parallèle (machine peu chargée)	≈ 190 ms	≈ 5 images/s
Application, CPU : usagers + anomalies (machine chargée)	≈ 730 ms	≈ 1,4 image/s
Application, CPU : modèle du prototype (image 480×640)	≈ 70 ms	≈ 14 images/s

3.6.5 Résultats obtenus sur les données réelles

Le tableau 3.9 donne les résultats du détecteur d'anomalies routières sur les 815 images de test (5 105 objets), au réglage temps réel utilisé dans l'application (640 px) et au meilleur réglage hors ligne (960 px avec augmentation au moment du test, TTA).

Table 3.9. Résultats du détecteur d'anomalies routières sur l'ensemble de test

		640 px (temps réel)				960 px + TTA			
Classe	**Objets**	P	R	mAP50	mAP50-95	P	R	mAP50	mAP50-95
nid-de-poule	299	0,601	0,254	0,312	0,108	0,626	0,268	0,330	0,119
cône	2 358	0,712	0,715	0,694	0,416	0,681	0,763	0,744	0,464
barrière	1 977	0,692	0,539	0,578	0,317	0,688	0,594	0,634	0,369
panneau	471	0,638	0,495	0,520	0,367	0,630	0,567	0,588	0,430
toutes	**5 105**	**0,661**	**0,501**	**0,526**	**0,302**	**0,656**	**0,548**	**0,574**	**0,346**

Les cônes et les barrières, nombreux et bien contrastés, sont les mieux reconnus. Les nids-de-poule restent difficiles (AP@50 de 0,31) : ils sont peu nombreux dans l'ensemble (299 objets de test) et visuellement très variables. Le réglage 960 px + TTA améliore le mAP@50 de 0,526 à 0,574, au prix d'un temps d'inférence cinq fois plus long ; il n'est donc pas utilisé en temps réel.

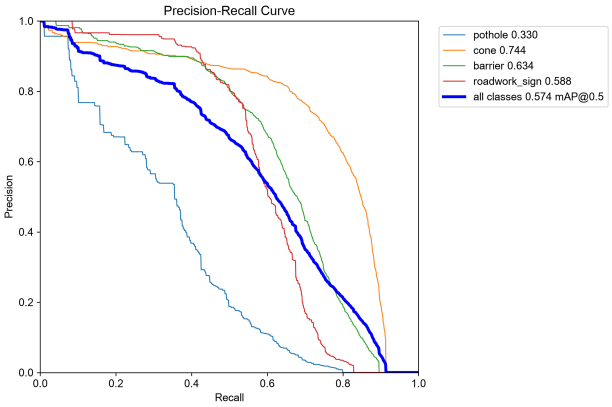
Deux analyses complémentaires éclairent ces chiffres. D'une part, si l'on ne demande au modèle que de *trouver* l'obstacle sans en nommer le type exact (évaluation agnostique de la classe, un cône pris pour une barrière restant une détection correcte), le mAP@50 atteint 0,630 en temps réel et 0,677 au meilleur réglage (tableau 3.10). D'autre part, le seuil de confiance règle le compromis entre fausses alarmes et obstacles manqués : à un seuil de 0,70, la précision moyenne atteint 0,916, mais le rappel tombe à 0,300.

Table 3.10. Évaluation agnostique de la classe et points de fonctionnement (ensemble de test)

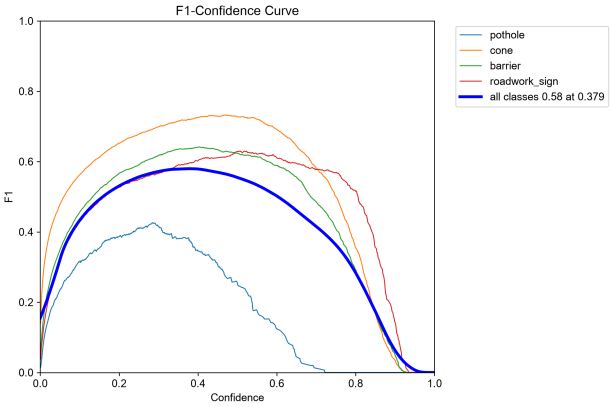
Évaluation agnostique	P	R	F1	mAP50	mAP50-95
640 px (temps réel)	0,726	0,597	0,655	0,630	0,364
960 px + TTA	0,711	0,646	0,677	0,677	0,412

Seuil de confiance	**P (par classe)**	**R (par classe)**	**P (agnostique)**	**R (agnostique)**
0,25	0,520	0,612	0,563	0,720
0,40	0,673	0,534	0,707	0,647
0,50	0,763	0,474	0,781	0,579
0,70	0,916	0,300	0,893	0,370

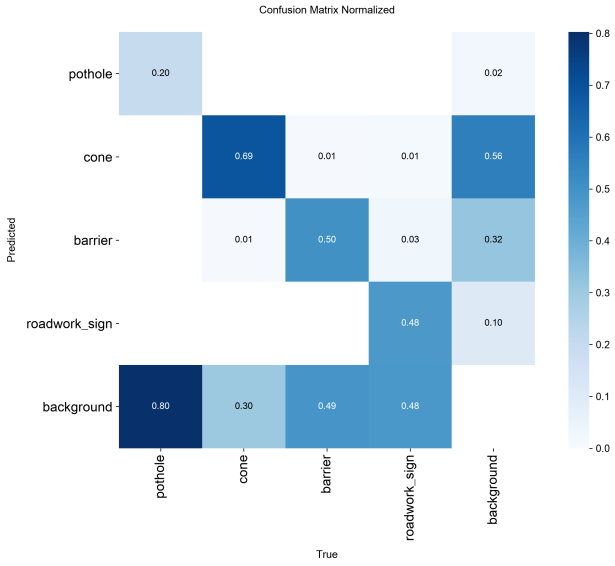
La figure 3.14 présente les courbes précision-rappel et F1 ainsi que la matrice de confusion au seuil utilisé dans l'application (0,35). La matrice montre que les confusions entre classes sont rares (au plus 3 %) : les erreurs sont surtout des objets manqués (ligne *background*) et des fausses détections sur le fond, en particulier des cônes.



(a) Courbes précision-rappel (960 px + TTA)



(b) Score F1 en fonction du seuil



(c) Matrice de confusion normalisée (640 px, seuil 0,35)

Figure 3.14. Résultats du détecteur d'anomalies routières sur l'ensemble de test

3.6.6 Résultats obtenus sur les données du prototype

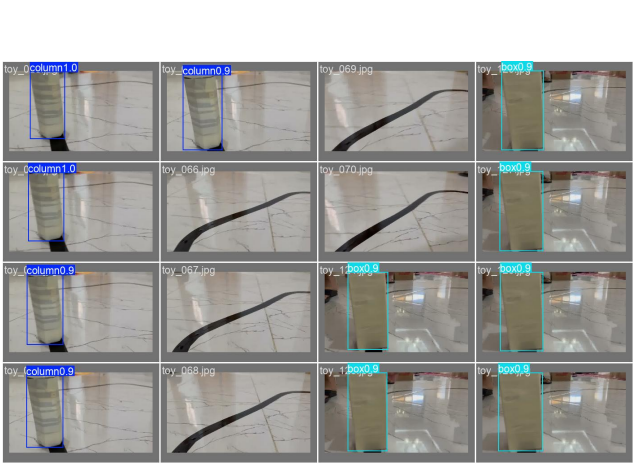
Le tableau 3.11 donne les résultats du modèle du prototype sur ses deux ensembles de test. Au seuil de 0,40 utilisé dans l'application, les 15 obstacles des images de test sont tous détectés avec la bonne classe, sans aucune fausse détection, aussi bien sur les images d'origine que sur leurs versions au style ESP32-CAM (tableau 3.12). Le mAP@50-95 plus faible sur les images ESP32-CAM (0,921 contre 0,976) traduit des boîtes un peu moins précises sur des images petites et dégradées.

Table 3.11. Résultats du modèle du prototype sur l'ensemble de test (26 images, 15 obstacles)

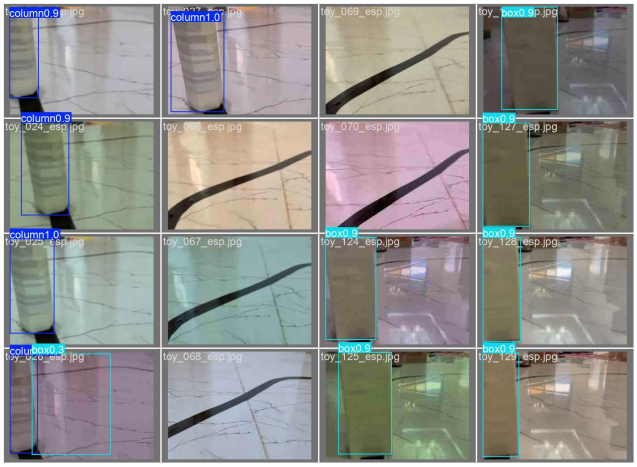
Ensemble de test	P	R	F1	mAP50	mAP50-95
Images d'origine (832 × 464)	0,989	1,000	0,994	0,995	0,976
Mêmes images au style ESP32-CAM (320 × 240)	0,991	1,000	0,995	0,995	0,921

Table 3.12. Matrice de confusion du modèle du prototype (test, seuil 0,40)

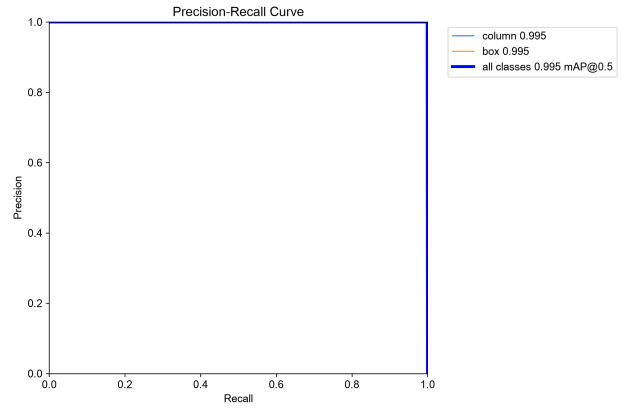
Réel \ prédit	colonne	boîte	fond (manqué)
colonne	6	0	0
boîte	0	9	0
fond (fausse détection)	0	0	—



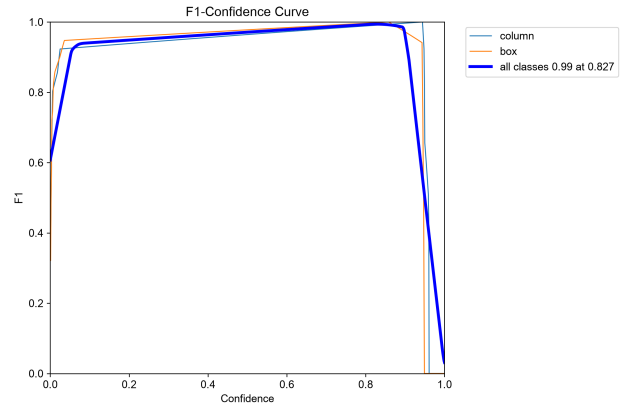
(a) Images de test d'origine



(b) Images de test au style ESP32-CAM



(c) Courbe précision-rappel



(d) Score F1 en fonction du seuil

Figure 3.15. Résultats du modèle du prototype sur l'ensemble de test

Ces résultats doivent être nuancés. L'ensemble de test est petit et provient de la même vidéo que l'entraînement (même éclairage, même sol), même si le découpage temporel évite les quasi-doublons. Le test sur la vraie caméra de la voiture, réalisé après l'entraînement, confirme que la boîte est reconnue avec une confiance de 95 % (figure 3.16) ; il a aussi révélé deux limites. D'une part, la caméra étant montée sur la tranche, l'image doit être redressée avant l'analyse : sans rotation, le modèle ne détecte rien. D'autre part, des éléments jamais vus pendant l'entraînement, comme un meuble brun en bord d'image ou une main devant l'objectif, ont parfois été pris pour une colonne. L'enregistrement d'images réelles de l'ESP32-CAM, prévu dans l'application, permettra d'ajouter ces cas à l'entraînement.



(a) Image brute (caméra sur la tranche) : aucune détection



(b) Image redressée : boîte détectée à 95 %

FIGURE 3.16. Test du modèle du prototype sur le flux réel de l'ESP32-CAM

3.6.7 Analyse comparative des deux modèles

Le tableau 3.13 compare les deux modèles entraînés. L'écart de performances s'explique d'abord par la difficulté des tâches : le modèle routier doit reconnaître des objets très variés dans des milliers de scènes différentes (villes, météo, éclairage), alors que le modèle du prototype reconnaît deux objets bien définis dans un environnement contrôlé. Les résultats du modèle routier, obtenus sur des lieux absents de l'entraînement, sont donc plus représentatifs d'un usage réel ; ceux du prototype mesurent surtout la bonne adaptation au circuit.

TABLE 3.13. Comparaison des deux modèles entraînés

Critère	**Anomalies routières**	**Prototype**
Variante	YOLO26s (*small*)	YOLO26n (*nano*)
Données	7 894 images réelles, 4 classes	249 images d'une vidéo + augmentations, 2 classes
Entraînement	8 h, 36 époques	30 min, 73 époques
mAP@50 (test)	0,526 (temps réel), 0,574 (meilleur réglage)	0,995
mAP@50-95 (test)	0,302, 0,346	0,976 (0,921 style ESP32)
Vitesse sur CPU	1,4 à 5 images/s (avec le modèle des usagers)	≈ 14 images/s
Point fort	généralisation à des lieux inconnus	très fiable sur son circuit
Limite	nids-de-poule (rappel 0,25)	domaine étroit, petit test

Conclusion du chapitre

Ce chapitre a présenté la conception des modèles de détection : le choix de YOLO26, la constitution et l'annotation des données (dont une annotation automatique par SAM 2 pour le prototype), l'entraînement dans un environnement Docker reproductible, l'exploitation des détections (distance, gravité, priorité) et une évaluation détaillée. Le modèle routier atteint un mAP@50 de 0,53 en temps réel sur des lieux inconnus, et le modèle du prototype reconnaît tous les obstacles de son ensemble de test. Le chapitre suivant présente l'application qui exploite ces modèles.

Chapitre 4 : Conception et développement de l'application SmartCar Ops

Introduction du chapitre

Ce chapitre présente *SmartCar Ops*, l'application web de supervision qui réunit les deux autres briques du projet : elle reçoit les images du prototype et de la caméra embarquée, exécute les modèles de détection, enregistre et priorise les obstacles, et les met à la disposition de l'opérateur. Nous décrivons d'abord ses objectifs, ses choix technologiques et son architecture, puis la gestion des données, avant de présenter chacune des huit pages. Nous terminons par une synthèse des interactions entre les modules.

4.1 Présentation générale de l'application

4.1.1 Objectifs et fonctionnalités principales

SmartCar Ops a pour objectif de donner à un opérateur une vue complète et à jour de la situation : où sont les véhicules, ce qu'ils voient, quels obstacles ont été détectés, lesquels sont les plus dangereux et quel itinéraire les évite. Ses fonctionnalités principales sont :

— la supervision en temps réel de la flotte du monde réel et de la voiture du prototype ;

— l'affichage des flux vidéo avec les détections des modèles d'IA superposées ;

— l'enregistrement, la fusion, l'évaluation de la gravité et la priorisation des obstacles ;

— la planification d'itinéraires qui pondère la distance et le risque ;

— des statistiques, la présentation des performances des modèles et la surveillance des équipements ;

— le basculement instantané entre les environnements *Monde réel* et *Prototype*.

4.1.2 Choix de Dash et de Plotly

L'application est développée en Python avec **Dash** [22], un framework de Plotly qui permet de construire des applications web analytiques sans écrire de JavaScript. L'interface est décrite par des composants Python (convertis en composants React dans le navigateur) et son comportement par des *callbacks* : des fonctions Python déclenchées lorsqu'une entrée change (clic, sélection, minuterie) et qui mettent à jour des sorties.

Ce choix présente plusieurs avantages pour notre projet :

— **un seul langage** pour l'IA, le traitement d'images (OpenCV [23], ONNX Runtime [24]) et l'interface ;

— des **graphiques interactifs** de qualité grâce à Plotly [25] : cartes routières (MapLibre), histogrammes, chronologies, cartes de chaleur ;

— le **multi-pages** intégré (`use_pages`) et la bibliothèque *Dash Bootstrap Components* pour la mise en page ;

— un serveur **Flask** sous-jacent, que nous utilisons aussi pour relayer le flux vidéo.

4.1.3 Architecture logicielle de l'application

L'application est organisée en quatre couches (figure 4.1) :

— la couche **présentation** : `app.py` construit la structure commune (barre latérale, barre supérieure, commutateur d'environnement, minuteries), les modules `pages/` définissent les huit pages et leurs callbacks, et `components/` regroupe le thème, les composants d'interface et les fonctions de création de figures ;

— la couche **métier** (`core/`) : télémétrie en mémoire (`hub`), obstacles et gravité, graphe routier et planification, trajets, journal d'événements, réglages et accès à la base de données ;

— la couche **services** (`sources/`) : fils d'exécution d'arrière-plan qui lisent la caméra ESP32-CAM et les vidéos, exécutent les détecteurs et font tourner les simulateurs ;

— la couche **données** : base SQLite, instantanés et réglages dans un volume Docker, modèles ONNX et leurs profils, graphe routier et plan de la piste.

L'ensemble s'exécute dans un conteneur Docker [26] : la commande `docker compose up` suffit pour lancer l'application sur n'importe quelle machine.

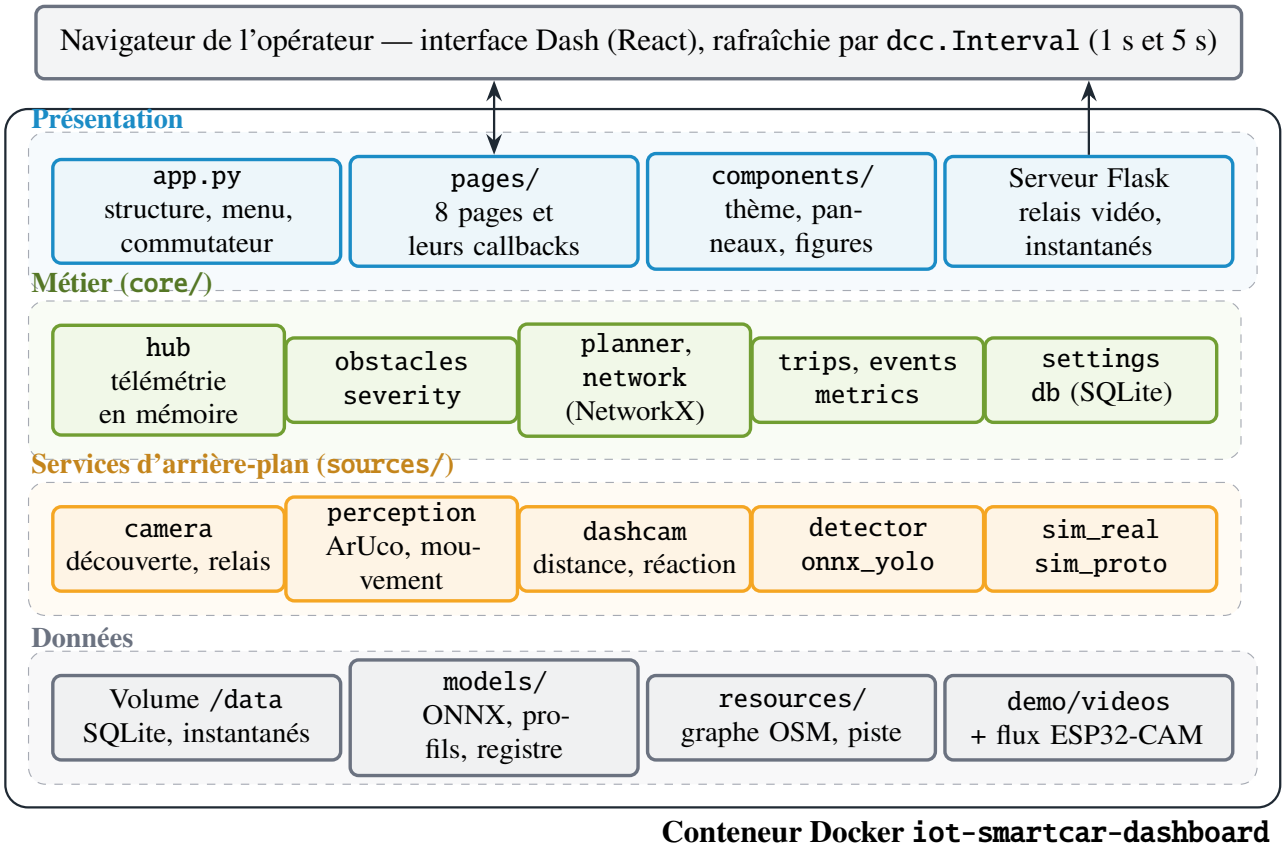


Figure 4.1. Architecture logicielle de *SmartCar Ops*

4.1.4 Organisation de l'interface et navigation

L'interface suit une organisation classique de tableau de bord. À gauche, une barre latérale flottante regroupe les huit pages par thème (figure 4.2) : **Live** (Tracking, Camera), **Planning** (Route planner), **Safety** (Obstacles), **Analytics** (Live stats, AI models) et **System** (Devices & logs), la page **Overview** étant placée en tête. Une horloge et un voyant d'activité figurent en bas de la barre. En haut, une barre affiche le titre et la description de la page courante, l'état de la source de données et le commutateur d'environnement.

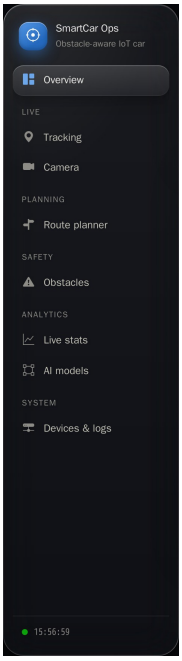


Figure 4.2. Menu latéral complet de l'application

4.1.5 Gestion des deux environnements : Real World et Prototype

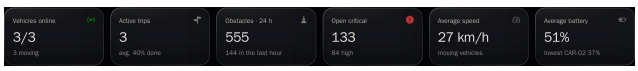
Le commutateur de la barre supérieure fait basculer toute l'application entre les deux environnements (figure 4.3). Sa valeur est mémorisée dans le navigateur et transmise à chaque callback : chaque page interroge alors les données de l'environnement choisi et adapte son contenu. Les unités passent des mètres aux centimètres, la carte routière de Paris laisse place au plan de la piste, et les indicateurs propres à la flotte (batterie, nombre de véhicules) sont remplacés par ceux du prototype (état de la voiture, progression du parcours, cadence de la caméra). L'indicateur de source précise l'origine des données : *Simulated data* pour le monde réel, *Live · ESP32-CAM* lorsque la caméra du prototype est connectée, ou *Demo* lorsqu'elle ne l'est pas.



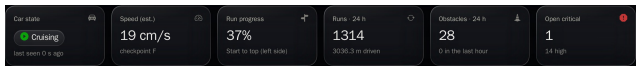
(a) Environnement *Real-world*



(b) Environnement *Prototype*



(c) Indicateurs de la flotte (monde réel)



(d) Indicateurs de la voiture (prototype)

FIGURE 4.3. Changement d'environnement : barre supérieure et indicateurs de la page Overview

4.2 Architecture et gestion des données

4.2.1 Architecture générale des échanges de données

La figure 4.4 décrit la circulation des données. Les *sources* (caméra ESP32-CAM, vidéos de la caméra embarquée, simulateurs) sont lues par des services qui s'exécutent en arrière-plan, indépendamment des pages. Les images passent par les détecteurs ; les résultats sont enregistrés dans deux espaces de stockage : un **hub en mémoire** pour les données très volatiles (position et état courants des véhicules, dernières détections) et la **base SQLite** pour les données à conserver (obstacles, trajets, points de trajectoire, événements). Les pages lisent ces données à chaque rafraîchissement ; le flux vidéo annoté est servi directement par le serveur Flask.

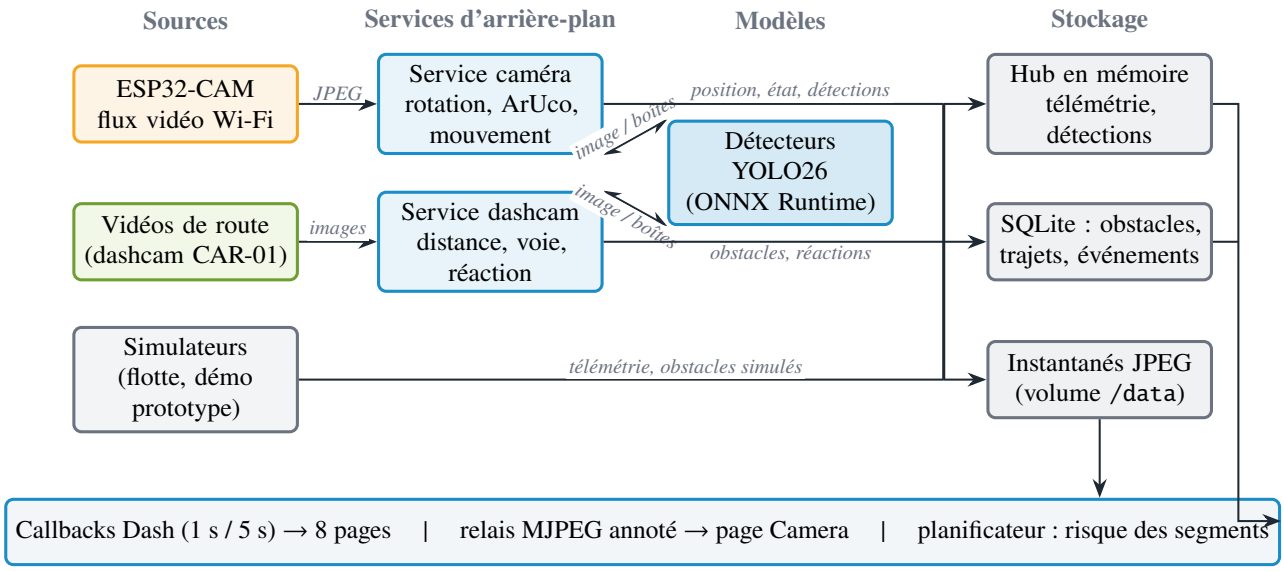


FIGURE 4.4. Flux des données entre les composants de *SmartCar Ops*

4.2.2 Intégration des modèles YOLO26

Les modèles sont chargés une seule fois, au démarrage des services, à partir de leurs profils (chapitre 3) : le service de la caméra embarquée charge les deux modèles routiers, le service de

l'ESP32-CAM le modèle du prototype. L'inférence s'exécute dans un fil dédié qui analyse toujours l'image la plus récente, sans bloquer ni la lecture vidéo ni l'interface. Les pages n'appellent jamais directement les modèles : elles lisent leurs résultats dans le hub et la base de données. La page *AI models* affiche les modèles effectivement chargés et leur temps d'exécution.

4.2.3 Acquisition et traitement des données du prototype

L'Arduino ne disposant d'aucune liaison sans fil, toutes les données du prototype sont extraites de l'image de l'ESP32-CAM :

— **Découverte de la caméra** : son adresse IP étant dynamique, le service essaie successivement l'adresse saisie dans l'application, celle de la configuration, la dernière adresse qui a fonctionné, puis balaye les sous-réseaux configurés en testant en parallèle les ports 80 et 81 et la réponse `/status`.

— **Relais** : la caméra ne servant qu'un client à la fois, le serveur est son unique client et redistribue les images à tous les onglets ouverts par la route `/camera/stream.mjpg`.

— **Redressement** : l'image est tournée selon le montage de la caméra (90° pour notre prototype).

— **Perception** : les marqueurs ArUco [27] imprimés aux points de passage A à F indiquent à quel point de la piste se trouve la voiture ; entre deux marqueurs, la position est estimée à partir de la vitesse mesurée. La différence entre images successives indique si la voiture roule ou est arrêtée. Le détecteur fournit les obstacles.

— **Trajets** : le passage au premier marqueur de la trajectoire active ouvre un trajet, le passage au dernier le clôture avec la distance parcourue.

Lorsque la caméra est hors ligne, un simulateur de démonstration fait rouler une voiture virtuelle sur la piste, afin que les pages restent utilisables.

4.2.4 Gestion des données issues du monde réel

L'environnement monde réel s'appuie sur le graphe routier du Quartier Latin à Paris, extrait d'OpenStreetMap [28] (309 intersections, 406 tronçons). Un simulateur fait circuler trois véhicules sur ce graphe : il choisit des trajets, fait avancer les véhicules à des vitesses réalistes, gère leur batterie et produit des rencontres d'obstacles plus fréquentes sur les tronçons à risque. Le véhicule CAR-01 fait exception : ses obstacles proviennent uniquement de sa caméra embarquée, alimentée par de vraies vidéos de route (ou un flux réseau, par exemple un téléphone fixé sur un tableau de bord) et analysée par les modèles. Chaque obstacle statique détecté est placé sur la carte devant CAR-01, à la distance estimée, et peut le faire ralentir ou s'arrêter.

4.2.5 Organisation et stockage des informations

Les données persistantes sont stockées dans une base **SQLite** [29] en mode WAL (journalisation anticipée, qui permet des lectures pendant les écritures), placée dans un volume Docker. Elle comporte quatre tables :

— `obstacles` : environnement, classe, confiance, proximité, distance, présence sur la trajectoire, position, tronçon, score de gravité, niveau, statut (nouveau, acquitté, résolu, faux positif), nombre d'observations, dates de première et dernière observation, instantané, source ;

— `trips` et `trip_points` : trajets (véhicule, itinéraire, durée, distance, obstacles rencontrés) et points de trajectoire horodatés pour la relecture ;

— `events` : journal des événements (niveau, catégorie, message).

Les réglages (adresse de la caméra, rotation, trajectoire active, poids de danger) sont conservés dans un petit fichier JSON, et les instantanés dans un dossier d'images du même volume.

4.2.6 Actualisation des données et affichage en temps réel

Deux minuteries (`dcc.Interval`) cadencent l'interface : chaque seconde pour les données en direct (positions, états, détections, indicateurs) et toutes les cinq secondes pour les vues plus lourdes (statistiques, listes). Seuls les callbacks de la page affichée sont exécutés. La vidéo, elle, n'utilise pas ce mécanisme : la balise image du navigateur reçoit directement le flux MJPEG relayé, à la cadence de la caméra.

4.3 Page Overview : tableau de bord général

4.3.1 Présentation de la vue d'ensemble

La page *Overview* est la page d'accueil (figure 4.5). Elle répond en un coup d'œil à la question « que se passe-t-il maintenant ? » : une rangée d'indicateurs, une carte en direct, une vignette de la caméra, la liste des obstacles à traiter et les derniers événements.

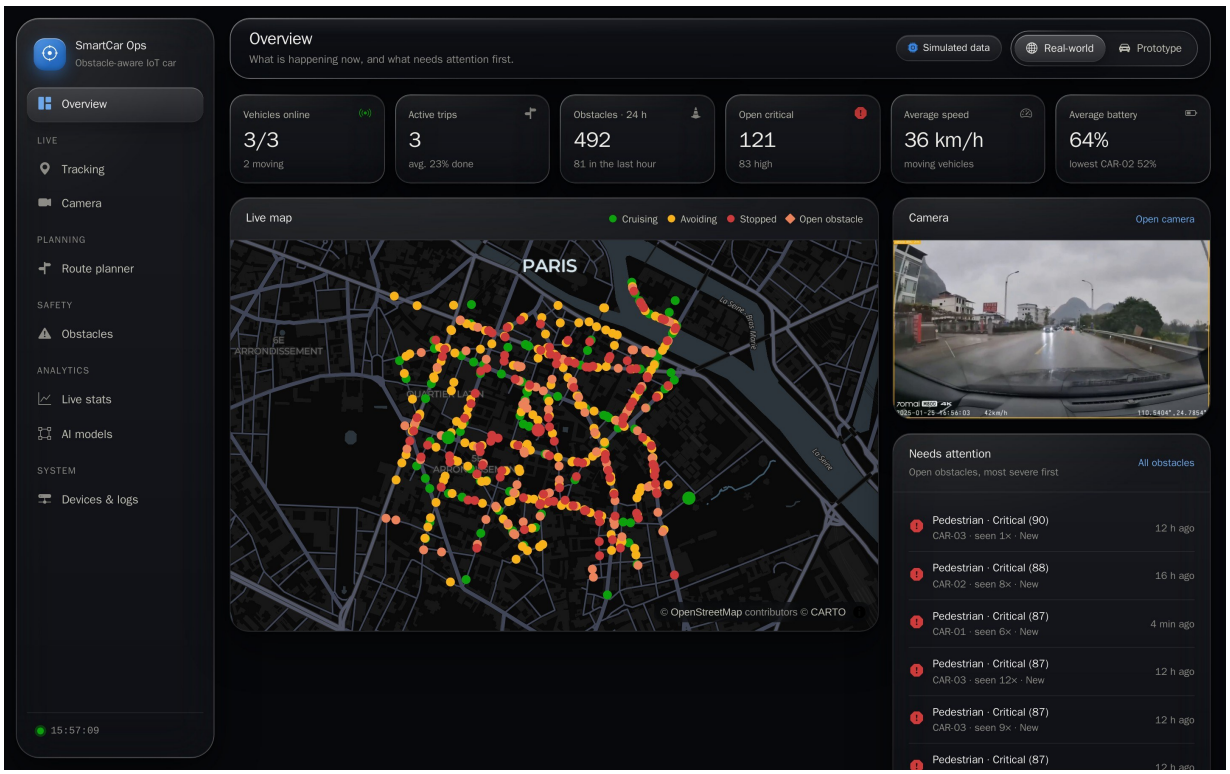


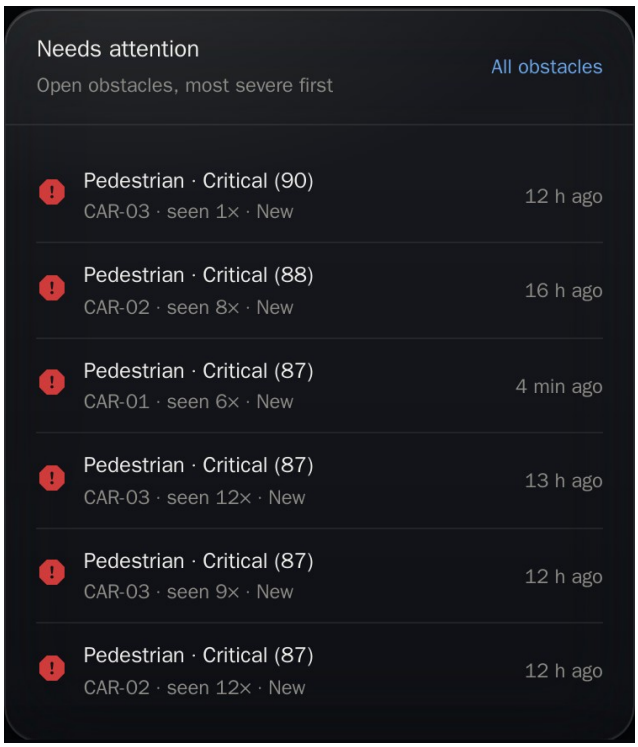
FIGURE 4.5. Tableau de bord général (page Overview, environnement monde réel)

4.3.2 Affichage des indicateurs clés

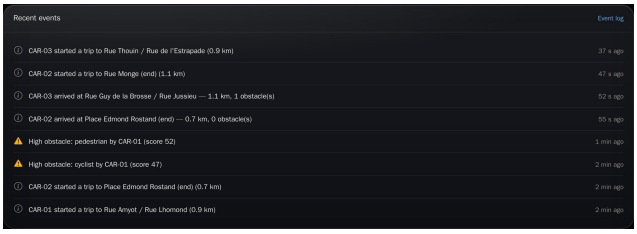
En environnement monde réel, les indicateurs sont : le nombre d'obstacles critiques ouverts (et d'obstacles de niveau élevé), le nombre d'obstacles détectés en 24 heures et dans la dernière heure, les véhicules en ligne et en mouvement, les trajets en cours, la vitesse moyenne et la batterie moyenne de la flotte. En mode prototype, ils deviennent l'état de la voiture, sa vitesse estimée, la progression du parcours et le nombre de parcours des dernières 24 heures (figure 4.3).

4.3.3 Synthèse des véhicules et des obstacles

La carte en direct montre les véhicules, colorés selon leur état (en route, en évitement, arrêté) et les obstacles ouverts. Le panneau *Needs attention* liste les obstacles ouverts du plus grave au moins grave et le panneau *Recent events* les derniers événements : arrivée d'un véhicule, obstacle critique, connexion de la caméra. . . (figure 4.6).



(a) Obstacles à traiter

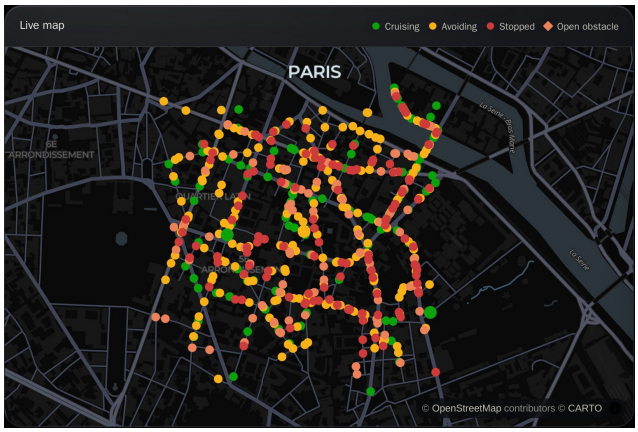


(b) Événements récents

FIGURE 4.6. Synthèse des obstacles et des événements de la page Overview

4.3.4 Comparaison des environnements Real World et Prototype

La structure de la page est identique dans les deux environnements, ce qui facilite la prise en main. Seuls le fond de carte et les indicateurs changent : carte routière et flotte de trois véhicules d'un côté, plan de la piste et voiture unique de l'autre (figure 4.7).



(a) Monde réel : Quartier Latin



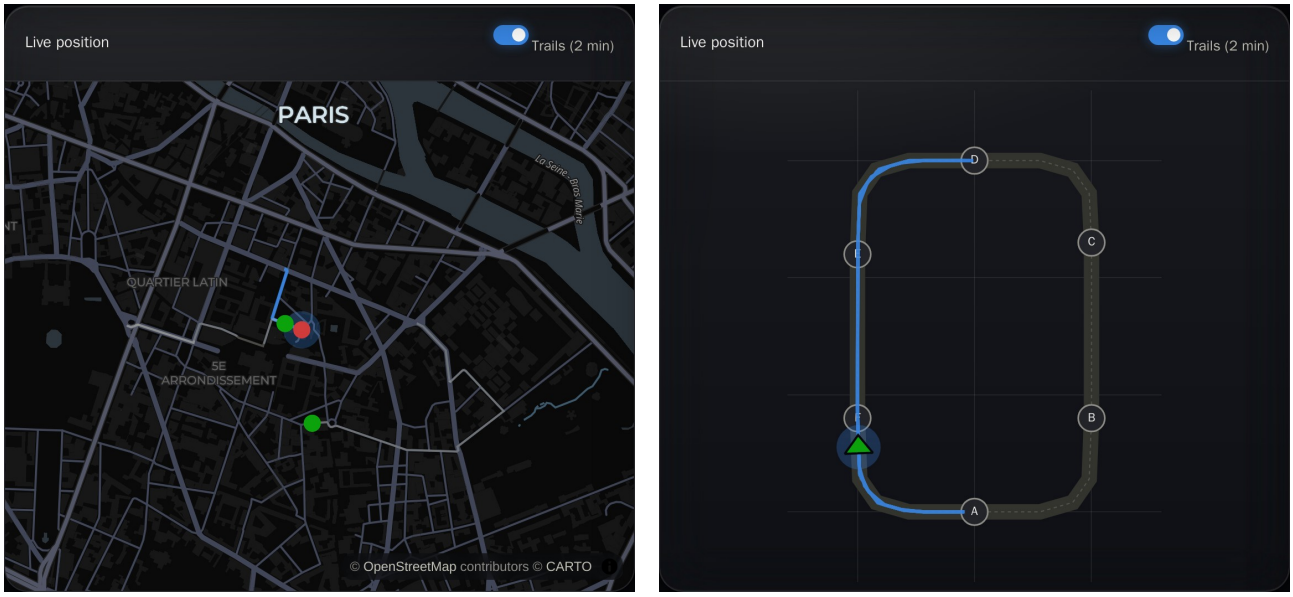
(b) Prototype : plan de la piste

FIGURE 4.7. Carte en direct dans les deux environnements

4.4 Page Tracking : suivi et localisation des véhicules

4.4.1 Visualisation de la position des véhicules

La carte de la page *Tracking* affiche la position de chaque véhicule, rafraîchie chaque seconde, avec une flèche indiquant sa direction (figure 4.8). Le véhicule sélectionné est mis en évidence. Dans le prototype, la position est donnée sur le plan de la piste, avec les points de passage A à F.

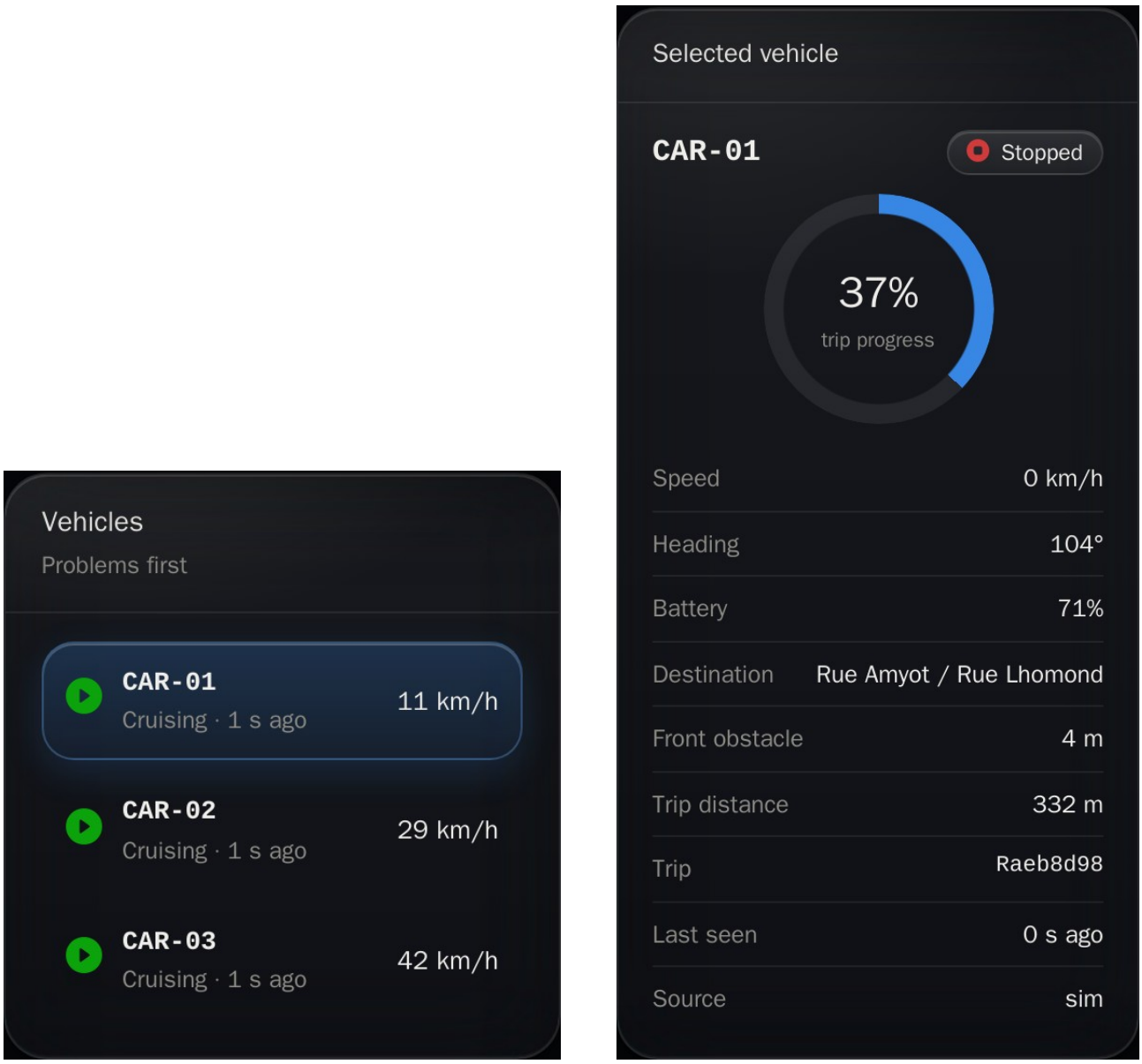


(a) Monde réel **(b)** Prototype

FIGURE 4.8. Carte de suivi avec la position des véhicules et leurs traces

4.4.2 Suivi des déplacements

L'option *Trails (2 min)* trace le chemin parcouru par chaque véhicule pendant les deux dernières minutes. La trace est interrompue à chaque nouveau trajet et lors d'un saut de position (retour de la voiture de démonstration au départ), pour ne pas dessiner de lignes trompeuses. Le panneau *Vehicles* liste les véhicules en plaçant d'abord ceux qui posent problème (arrêtés, ligne perdue, en évitement), et le panneau *Selected vehicle* détaille le véhicule choisi : état, vitesse, batterie, distance à l'obstacle frontal et progression du trajet (figure 4.9).



(a) Liste des véhicules **(b)** Véhicule sélectionné

FIGURE 4.9. Suivi des véhicules

4.4.3 Historique des trajectoires

Chaque trajet est enregistré avec ses points de trajectoire horodatés (position, vitesse, état), échantillonnés chaque seconde. La table *Trip history* présente les trajets récents : heure de départ, véhicule, itinéraire, durée, distance et nombre d'obstacles rencontrés.

4.4.4 Consultation des parcours précédents

La sélection d'un trajet dans la table le rejoue sur une carte dédiée : le tracé complet apparaît, un curseur permet de parcourir le trajet dans le temps et un graphique montre le profil de vitesse (figure 4.10). Le dernier trajet terminé est sélectionné automatiquement à l'ouverture de la page.

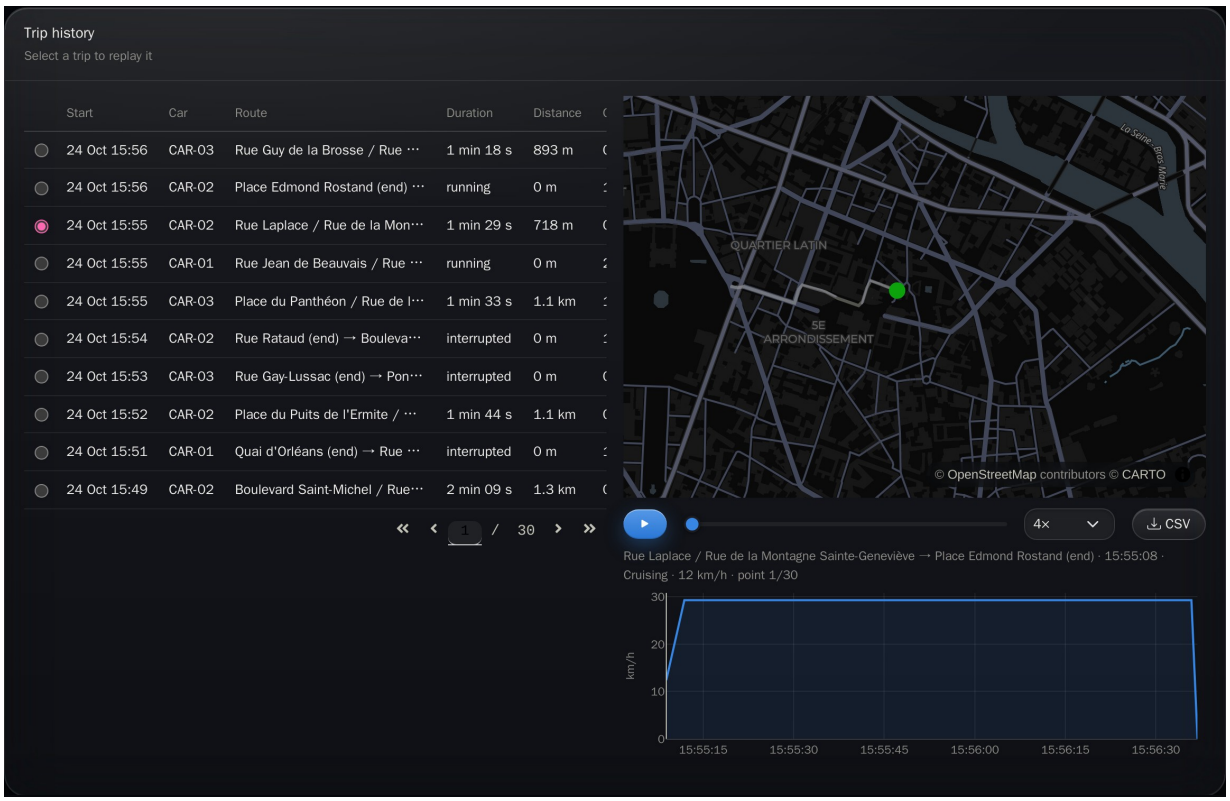


FIGURE 4.10. Historique des trajets et relecture d'un parcours précédent

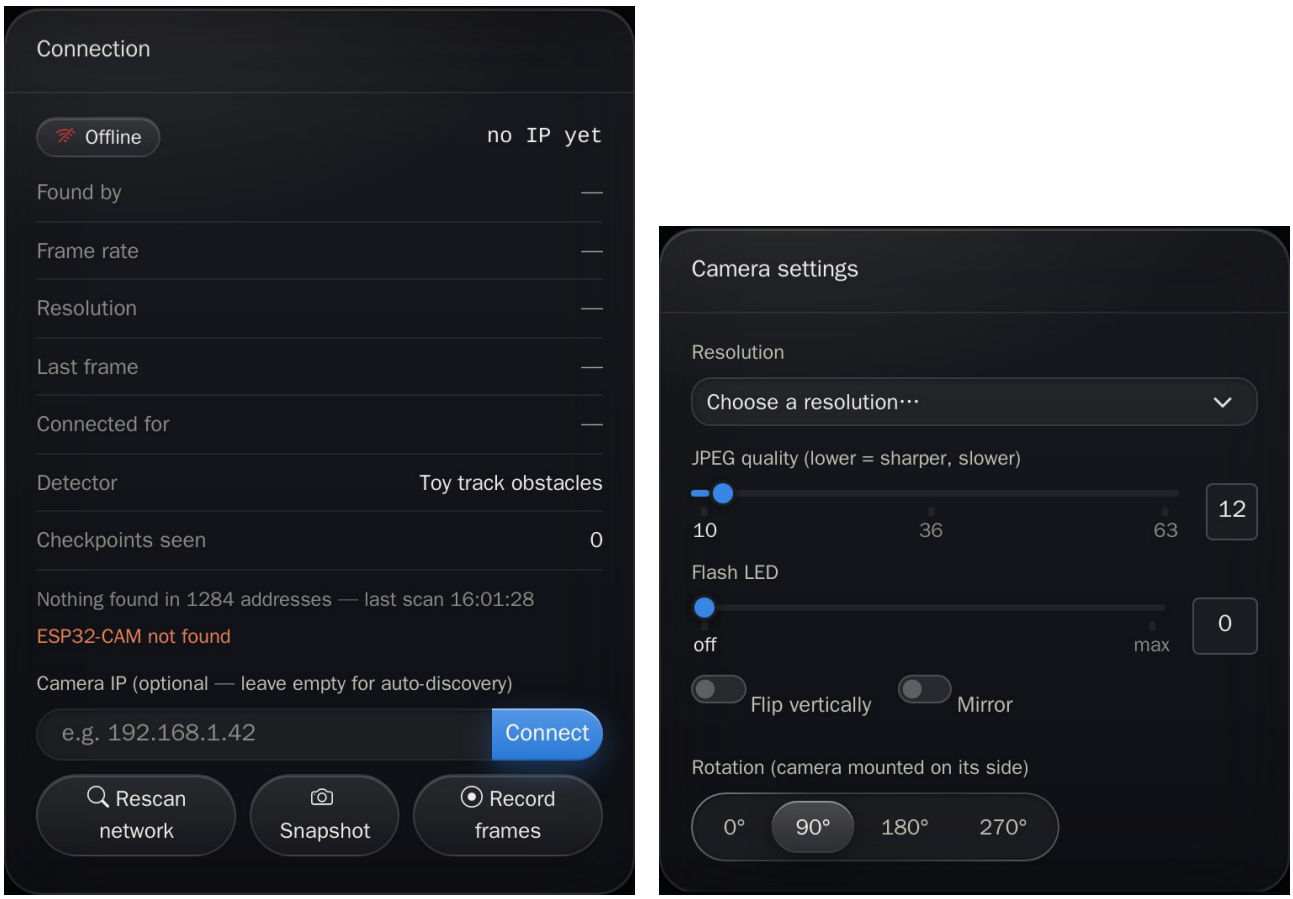
4.4.5 Gestion des données de localisation

Les positions courantes sont conservées dans le hub en mémoire (historique glissant pour les traces), alors que les points de trajectoire des trajets sont écrits dans la base. En monde réel, les coordonnées sont des longitudes et latitudes et les distances sont calculées en mètres ; dans le prototype, ce sont des coordonnées en centimètres sur le plan de la piste. Un module de géométrie commun fournit les calculs de distance, de longueur de tracé et d'interpolation dans les deux systèmes.

4.5 Page Camera : visualisation du flux vidéo

4.5.1 Intégration du flux vidéo de l'ESP32-CAM

En mode prototype, la page affiche le flux de l'ESP32-CAM relayé par le serveur. Le panneau *Connection* indique l'état de la caméra, son adresse IP et la manière dont elle a été trouvée, la cadence, la résolution, le détecteur utilisé et le nombre de points de passage reconnus ; il permet de saisir une adresse, de relancer la recherche sur le réseau, de prendre un instantané et d'**enregistrer des images** pour enrichir le jeu de données du prototype. Le panneau *Camera settings* règle la résolution, la qualité JPEG, le flash, le retournement, le miroir et la **rotation** de l'image (figure 4.11).



(a) Connexion à l'ESP32-CAM **(b)** Réglages de la caméra

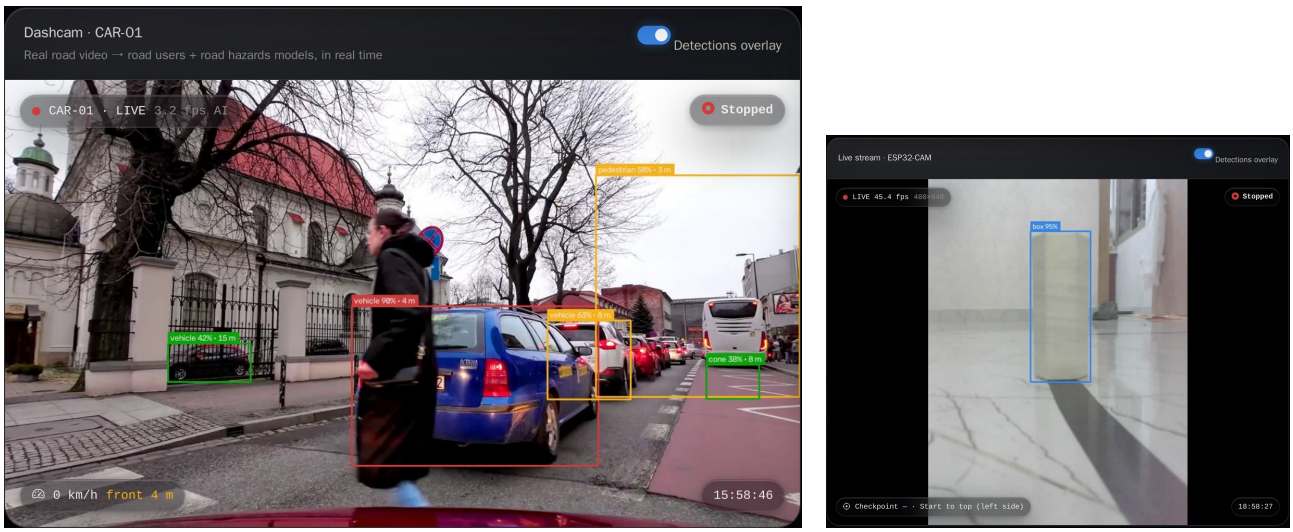
Figure 4.11. Panneaux de connexion et de réglage de l'ESP32-CAM

4.5.2 Affichage des images en direct

L'image est affichée dans un cadre au format de la caméra, surmonté d'un affichage tête haute : indicateur *LIVE*, cadence, résolution, état de la voiture, point de passage et itinéraire, heure. Un interrupteur permet d'afficher ou de masquer les détections. Lorsque la caméra est hors ligne, une image d'attente explique la situation (recherche en cours, dernière adresse connue).

4.5.3 Intégration des résultats de l'intelligence artificielle

Les détections du modèle sont dessinées directement sur l'image relayée : boîte, classe et confiance. En monde réel, la couleur de la boîte correspond au niveau de gravité et l'étiquette indique aussi la distance estimée (figure 4.12).

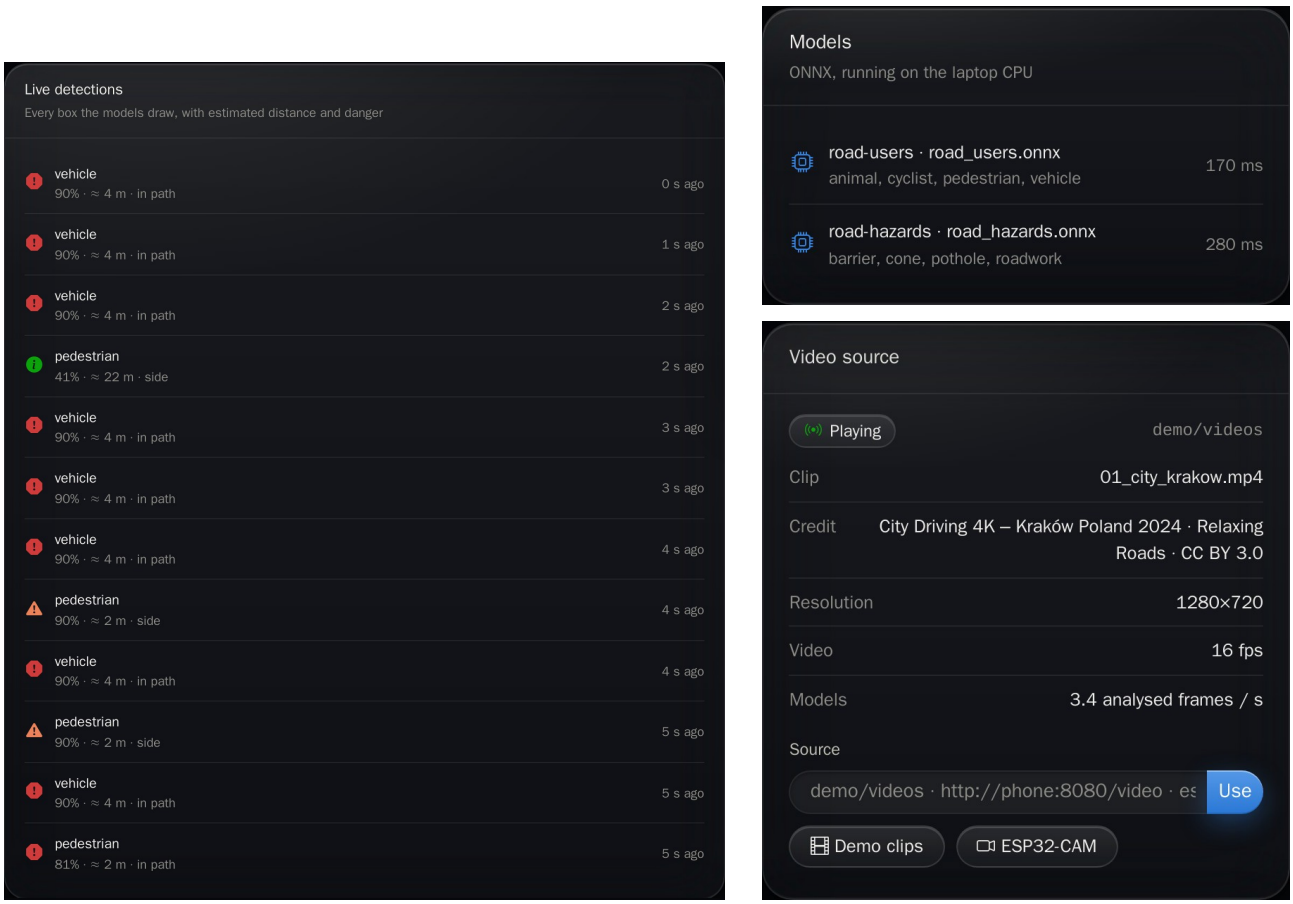


(a) Monde réel : caméra embarquée de CAR-01 **(b)** Prototype : ESP32-CAM

FIGURE 4.12. Flux vidéo avec les prédictions des modèles affichées

4.5.4 Visualisation des détections sur les images

Sous la vidéo, le panneau *Live detections* liste les dernières détections (une entrée par classe et par demi-seconde au plus) avec la confiance, la distance estimée et la position par rapport à la trajectoire ; le panneau *Models* rappelle les modèles chargés et leur temps d'exécution (figure 4.13). En mode prototype, le panneau *Snapshots* rassemble les instantanés enregistrés.



(a) Dernières détections **(b)** Modèles chargés et source vidéo

FIGURE 4.13. Détections, modèles et source de la page Camera (monde réel)

4.5.5 Comparaison entre les vues Real World et Prototype

Les deux vues partagent la même logique (relais, détection, affichage tête haute, liste des détections) mais diffèrent par leur source et leur exploitation. En monde réel, la source est une vidéo de route choisie dans le panneau *Video source* (clips de démonstration, flux réseau ou ESP32-CAM) et les détections pilotent les réactions de CAR-01 et la carte des obstacles. En prototype, la source est l'ESP32-CAM de la voiture ; les détections alimentent la liste des obstacles de la piste et l'état de la voiture.

4.6 Page Route Planner : planification des itinéraires

4.6.1 Présentation de l'interface cartographique

La page *Route planner* associe un formulaire, une carte et une liste d'itinéraires proposés (figure 4.14). La carte affiche le réseau, les obstacles ouverts et, si l'option *Risk layer* est active, les tronçons à risque colorés selon leur niveau.

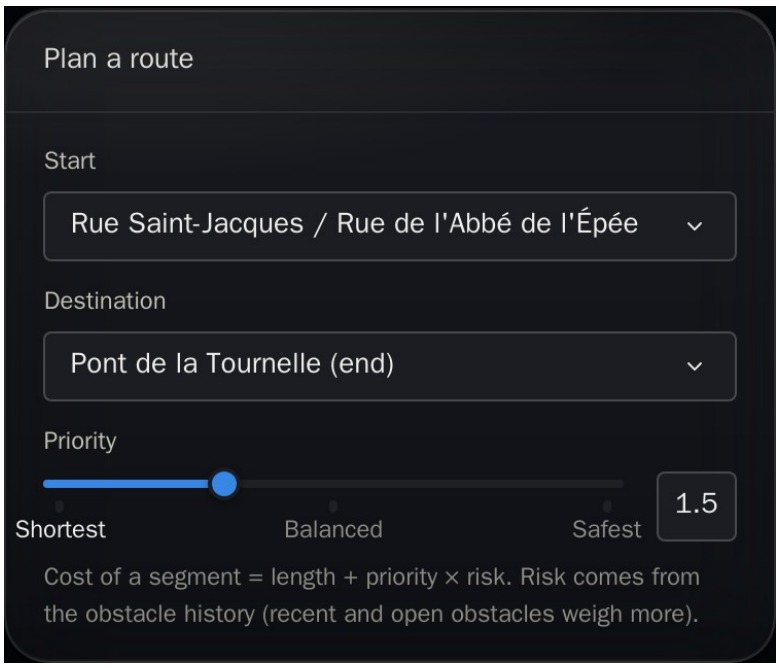


Figure 4.14. Formulaire de planification d'itinéraire

4.6.2 Sélection des points de départ et d'arrivée

Le départ et la destination se choisissent dans des listes d'intersections nommées d'après leurs rues (monde réel) ou parmi les points de passage de la piste (prototype). Des valeurs par défaut pertinentes sont proposées : deux intersections éloignées dans le monde réel, le départ et l'arrivée d'une trajectoire de la piste dans le prototype.

4.6.3 Représentation des obstacles sur la carte

Les obstacles ouverts sont représentés par des marqueurs colorés selon leur niveau de gravité ; les tronçons dont le risque dépasse un seuil sont surlignés en orange (élevé) ou en rouge (critique). Les seuils diffèrent entre les deux environnements, car la piste, très courte, accumule le risque plus vite.

4.6.4 Calcul des itinéraires possibles

Le réseau est représenté par un graphe : les nœuds sont les intersections (ou points de passage), les arêtes les tronçons, avec leur géométrie et leur longueur. Les itinéraires sont calculés par l'algorithme de plus court chemin de Dijkstra [30] de la bibliothèque NetworkX [31], avec un coût qui combine distance et risque.

4.6.5 Prise en compte de la distance, des obstacles et de leur gravité

Chaque tronçon reçoit un **risque** calculé à partir de l'historique des obstacles qui s'y trouvent :

$$\text{risque}(e) = \sum_{o \in e} \frac{\text{score}_o}{100} \times w_{\text{statut}} \times 0{,}5^{\,\text{âge}_o / T_{1/2}} \tag{4.1}$$

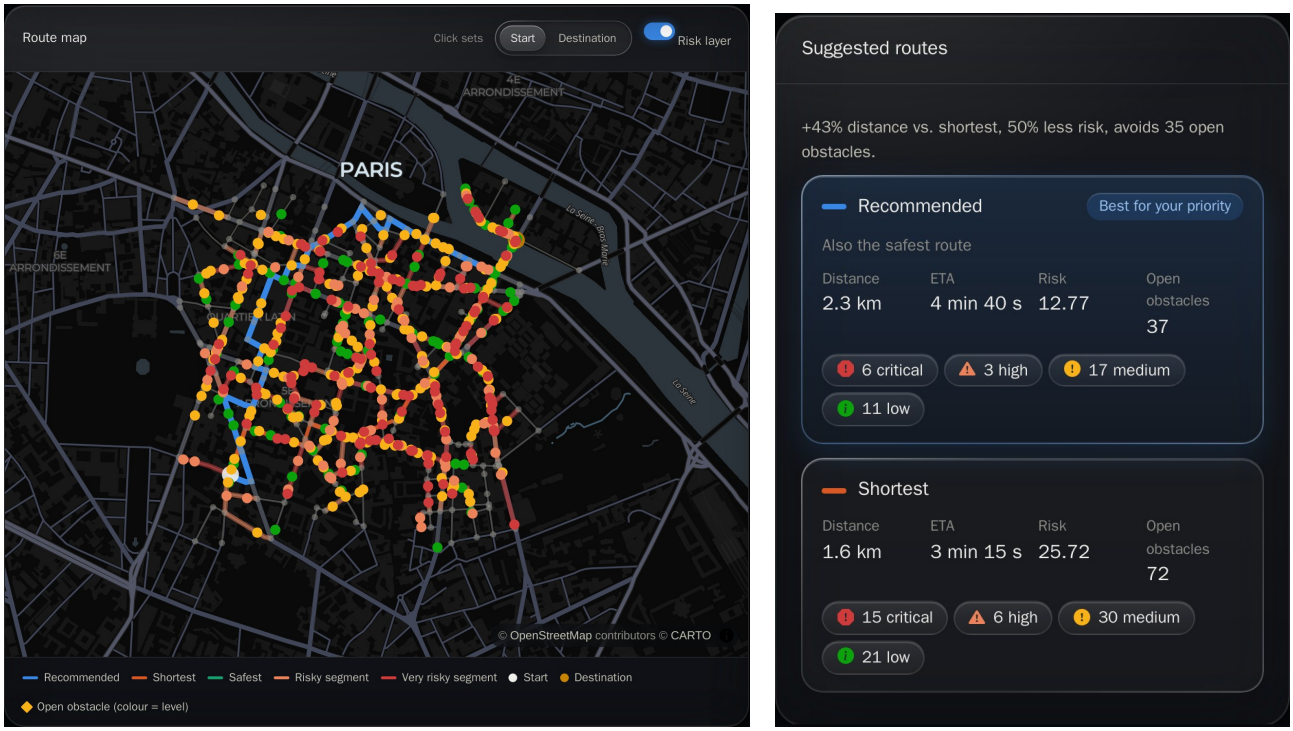
Les obstacles graves comptent davantage, les obstacles anciens s'effacent progressivement (demi-vie $T_{1/2}$ de 7 jours dans le monde réel, 2 jours sur la piste), les obstacles résolus ne comptent plus que pour 30 % et les faux positifs sont ignorés. Le coût d'un tronçon est alors :

$$\text{coût}(e) = \text{longueur}(e) + \alpha \times U \times \text{risque}(e) \tag{4.2}$$

où U exprime la distance qu'« équivaut » une unité de risque (150 m dans le monde réel, 60 cm sur la piste) et α la priorité choisie par l'opérateur avec le curseur : $\alpha = 0$ donne le plus court chemin, une grande valeur l'itinéraire le plus sûr.

4.6.6 Sélection et affichage de l'itinéraire recommandé

Trois itinéraires sont calculés : *Recommended* (priorité choisie), *Shortest* ($\alpha = 0$) et *Safest* ($\alpha = 8$). Lorsque deux d'entre eux coïncident, ils sont fusionnés. L'itinéraire recommandé est tracé sur la carte avec les obstacles (figure 4.15) ; une phrase résume le compromis, par exemple « +43 % de distance par rapport au plus court, 50 % de risque en moins, évite 35 obstacles ouverts ».



(a) Carte avec l'itinéraire proposé et les obstacles

(b) Itinéraires proposés

Figure 4.15. Itinéraire recommandé, obstacles et tronçons à risque

4.6.7 Comparaison des itinéraires proposés

Chaque itinéraire est présenté sur une carte de synthèse avec sa distance, sa durée estimée, son risque cumulé et le nombre d'obstacles ouverts sur son parcours, répartis par niveau. L'opérateur compare ainsi d'un coup d'œil le gain de sécurité et le coût en distance. En mode prototype, la page ajoute le classement des **trajectoires manuelles** de la piste selon le même coût et un **éditeur de piste** : positions des points de passage en centimètres, tronçons, trajectoires, et téléchargement des marqueurs

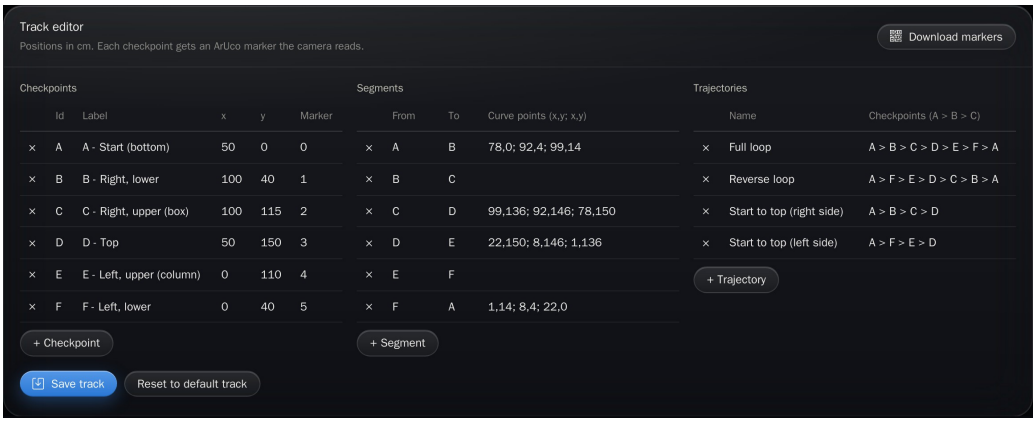
ArUco à imprimer (figure 4.16).



(a) Carte de la piste



(b) Trajectoires classées selon la priorité



(c) Éditeur de piste

Figure 4.16. Planification dans l'environnement prototype

4.7 Page Obstacles : gestion et priorisation des anomalies

La page *Obstacles* est le centre de gestion des anomalies. Elle présente, en haut, une barre de filtres et des indicateurs, puis la file de priorité, le détail de l'obstacle sélectionné, une carte, une répartition par classe et par niveau, et une explication du calcul de la priorité (figure 4.17).



Figure 4.17. Liste des anomalies détectées, triées par priorité

4.7.1 Affichage des obstacles détectés

La file de priorité est une table paginée qui présente pour chaque obstacle : son niveau, son score, la classe, la confiance, la distance estimée, sa position sur la trajectoire ou sur le côté, le nombre d'observations, la date de la dernière observation, le statut et le véhicule qui l'a détecté. Les obstacles vus plusieurs fois n'apparaissent qu'une fois, avec leur compteur d'observations.

4.7.2 Identification des nids-de-poule et autres anomalies

Les anomalies proviennent de deux sources : les détections réelles des modèles (caméra embarquée de CAR-01 et ESP32-CAM du prototype), accompagnées d'un instantané annoté, et les rencontres produites par le simulateur pour les autres véhicules de la flotte, signalées comme simulées. Le détail de l'obstacle précise sa source.

4.7.3 Classification des obstacles par catégorie

Chaque obstacle porte l'une des classes du vocabulaire commun : piéton, cycliste, animal, véhicule, nid-de-poule, cône, barrière, panneau de chantier, débris, et, pour le prototype, colonne et boîte. Un filtre permet de n'afficher que certaines classes, et le graphique *By object and level* montre leur répartition

(figure 4.18).

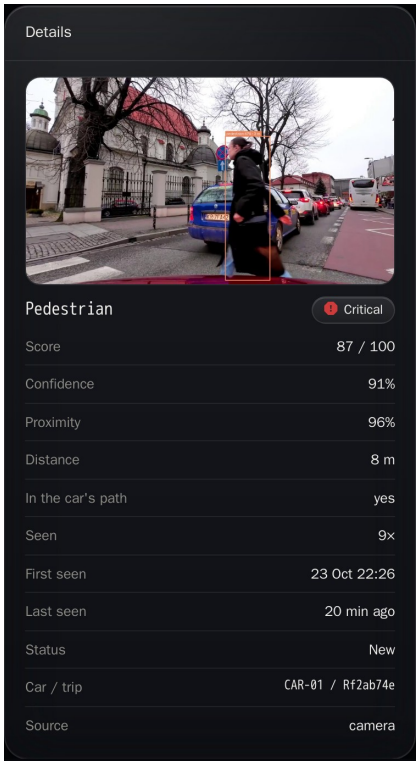


(a) Localisation des obstacles par niveau **(b)** Répartition par classe et par niveau

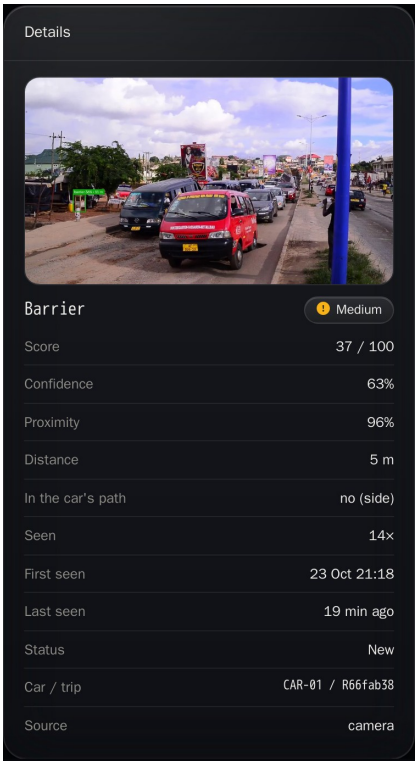
FIGURE 4.18. Carte et répartition des obstacles

4.7.4 Évaluation de la gravité

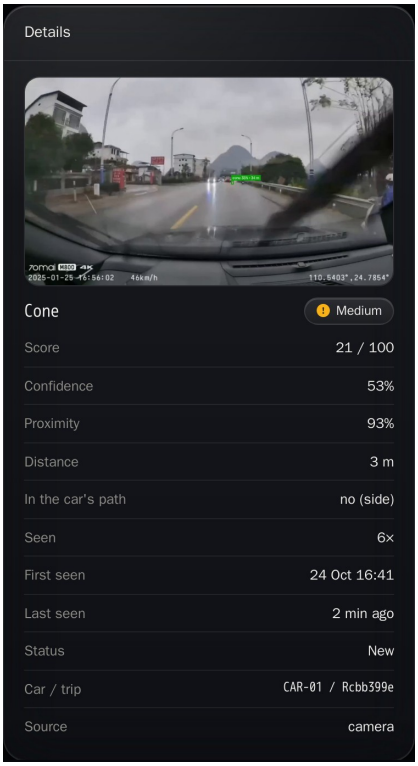
La gravité de chaque obstacle est le score explicable présenté au chapitre 3, qui combine le poids de danger de la classe, la proximité, la confiance et la présence sur la trajectoire. Le panneau *How priority is computed* rappelle la formule et les seuils des niveaux à l'opérateur, et le détail de l'obstacle en montre chaque composante (figure 4.19).



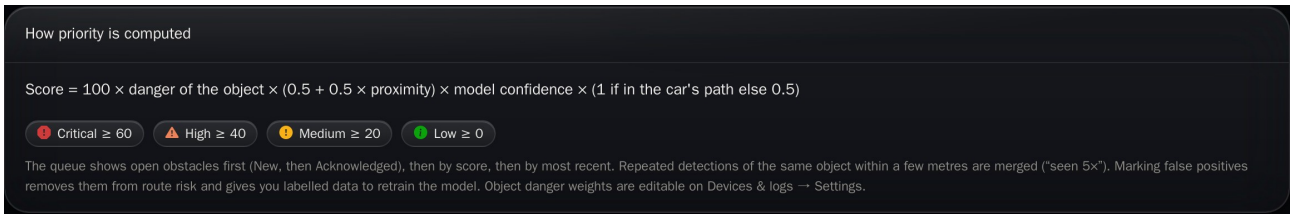
(a) Piéton sur la trajectoire : critique



(b) « Barrière » sur le côté : moyen



(c) Cône sur le côté : moyen



(d) Explication du calcul de la priorité

FIGURE 4.19. Niveaux de gravité et priorités : détail de trois obstacles détectés par la caméra embarquée

L'exemple de la « barrière » est instructif : l'instantané montre qu'il s'agit en réalité d'un poteau bleu. C'est exactement le type d'erreur que l'opérateur peut signaler comme faux positif.

4.7.5 Attribution des niveaux de priorité

Le score détermine le niveau (critique, élevé, moyen, faible) et la file est triée par niveau puis par score. Les indicateurs en tête de page comptent les obstacles ouverts de chaque niveau, le nombre d'obstacles revus et le taux de faux positifs parmi eux (figure 4.20). Les poids de danger étant réglables, l'opérateur peut adapter la priorisation à son contexte ; tous les scores sont alors recalculés.

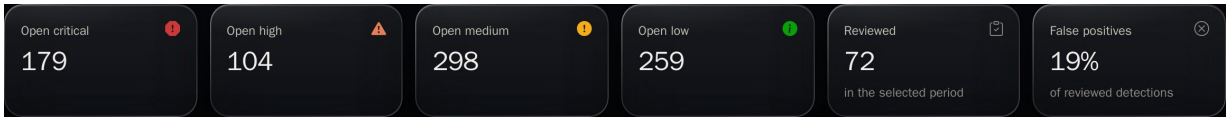


FIGURE 4.20. Indicateurs de la page Obstacles : obstacles ouverts par niveau, revues et faux positifs

4.7.6 Localisation et consultation des anomalies

La sélection d'un obstacle affiche son détail : instantané annoté, classe et niveau, score, confiance, proximité, distance, position par rapport à la trajectoire, nombre d'observations, dates, statut, véhicule, trajet et source. La carte *Where they are* situe tous les obstacles de la période. L'opérateur peut ensuite agir sur un ou plusieurs obstacles sélectionnés : **Acknowledge** (pris en compte), **Resolve** (traité), **False positive** (erreur du modèle) ou **Reopen**. Ces décisions modifient le risque utilisé par le planificateur et alimentent l'évaluation des modèles.

4.7.7 Filtrage et tri des obstacles

La barre de filtres permet de choisir l'onglet (*Needs review*, *Acknowledged*, *Closed*, *All*), les niveaux affichés, les classes et la période (dernière heure, 24 heures, 7 jours, 30 jours, tout l'historique) ; chaque colonne de la table peut être triée (figure 4.21).

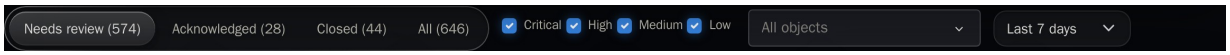


Figure 4.21. Barre de filtres de la page Obstacles

4.8 Page Live Stats : statistiques et indicateurs

La page *Live stats* rassemble les indicateurs et graphiques statistiques (figure 4.22).



Figure 4.22. Graphiques et indicateurs statistiques (monde réel)

4.8.1 Présentation des indicateurs statistiques

En monde réel, les indicateurs sont : les véhicules en ligne, la distance parcourue et le nombre de trajets en 24 heures, la vitesse moyenne, le nombre moyen d'obstacles par heure, les manœuvres d'évitement et la batterie moyenne. En prototype, ce sont l'état de la voiture, la source de données (caméra en direct ou démonstration), la vitesse estimée, la cadence de la caméra, les parcours et les obstacles des dernières 24 heures (figure 4.23).



(a) Monde réel

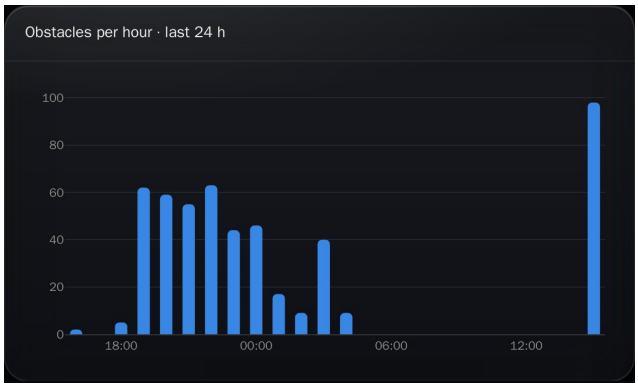


(b) Prototype

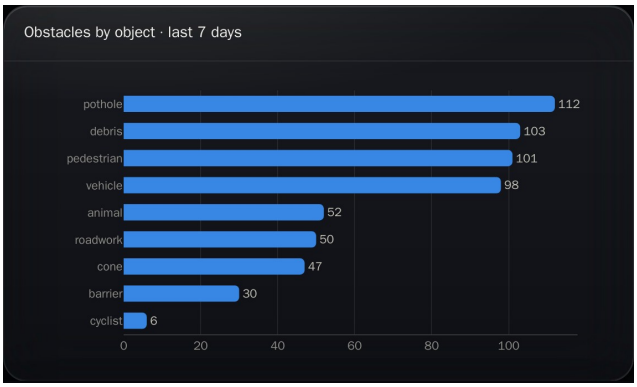
FIGURE 4.23. Indicateurs statistiques dans les deux environnements

4.8.2 Nombre et répartition des obstacles détectés

Le graphique *Obstacles per hour* montre le nombre d'obstacles détectés par heure sur les dernières 24 heures, ce qui fait apparaître les périodes chargées ; le graphique *Obstacles by object* donne leur répartition par classe sur 7 jours (figure 4.24).



(a) Obstacles par heure (24 h)



(b) Obstacles par classe (7 jours)

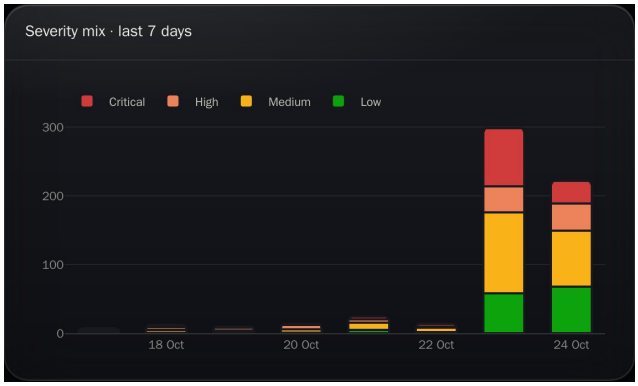
FIGURE 4.24. Nombre et répartition des obstacles détectés

4.8.3 Distribution des anomalies par gravité et priorité

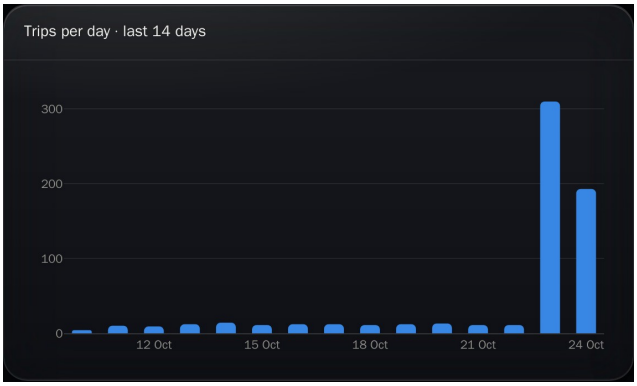
Le graphique *Severity mix* montre, pour chaque jour, la part de chaque niveau de gravité parmi les obstacles détectés (figure 4.25). Il permet de suivre l'évolution de la dangerosité et non seulement du volume.

4.8.4 Analyse des données de suivi des véhicules

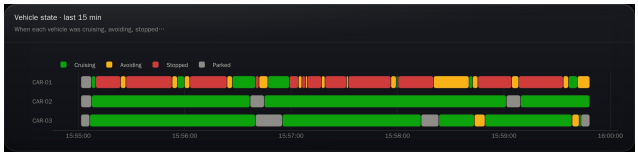
La chronologie *Vehicle state* montre l'état de chaque véhicule (en route, en évitement, arrêté, en charge) sur les 15 dernières minutes, le graphique *Speed* leur vitesse sur 5 minutes, et le graphique des trajets par jour l'activité de la flotte (figure 4.25).



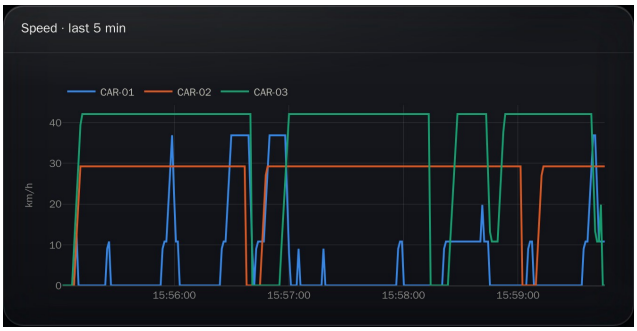
(a) Répartition par gravité



(b) Trajets par jour



(c) États des véhicules (15 min)



(d) Vitesse des véhicules (5 min)

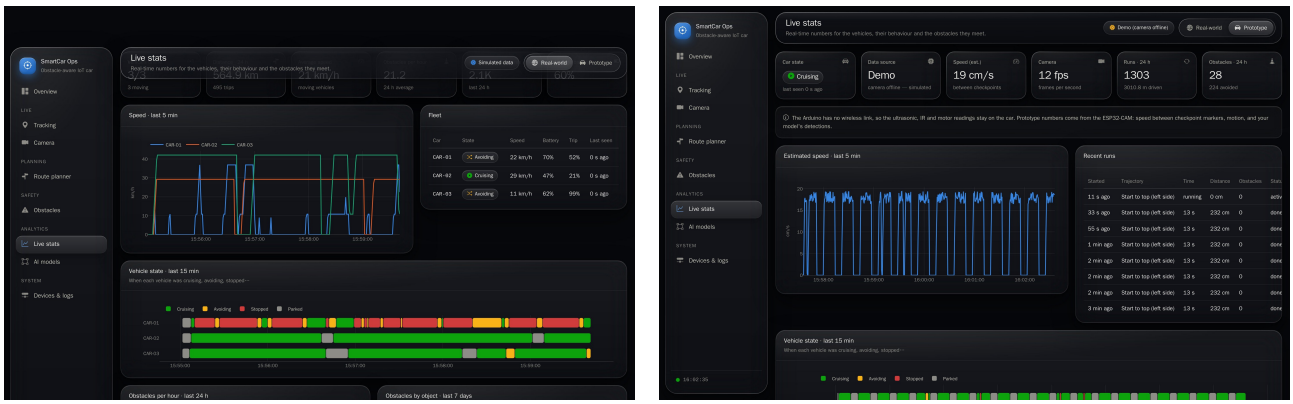
FIGURE 4.25. Gravité des anomalies et données de suivi des véhicules

4.8.5 Visualisation graphique des statistiques

Les graphiques sont construits avec Plotly et partagent une même charte : fond transparent sur les panneaux, quadrillage discret, palette de couleurs catégorielles distinctes pour les séries, couleurs de gravité réservées aux niveaux (rouge, orange, jaune, vert) et infobulles au survol. Les valeurs sont toujours accompagnées de leur unité.

4.8.6 Comparaison des résultats entre les deux environnements

Le changement d'environnement s'applique aussi aux statistiques : la même page présente les données de la flotte simulée ou celles de la voiture du prototype (figure 4.26). La comparaison doit tenir compte de la nature des données : les statistiques du monde réel décrivent une flotte simulée dont seul CAR-01 voit de vraies images, alors que celles du prototype décrivent des parcours réels sur la piste lorsque la caméra est connectée.



(a) Monde réel **(b)** Prototype

Figure 4.26. Page Live stats dans les deux environnements

4.9 Page AI Models : gestion des modèles d'intelligence artificielle

La page *AI models* présente, pour l'environnement choisi, chaque modèle avec son évaluation hors ligne, puis son comportement sur la route (figure 4.27).

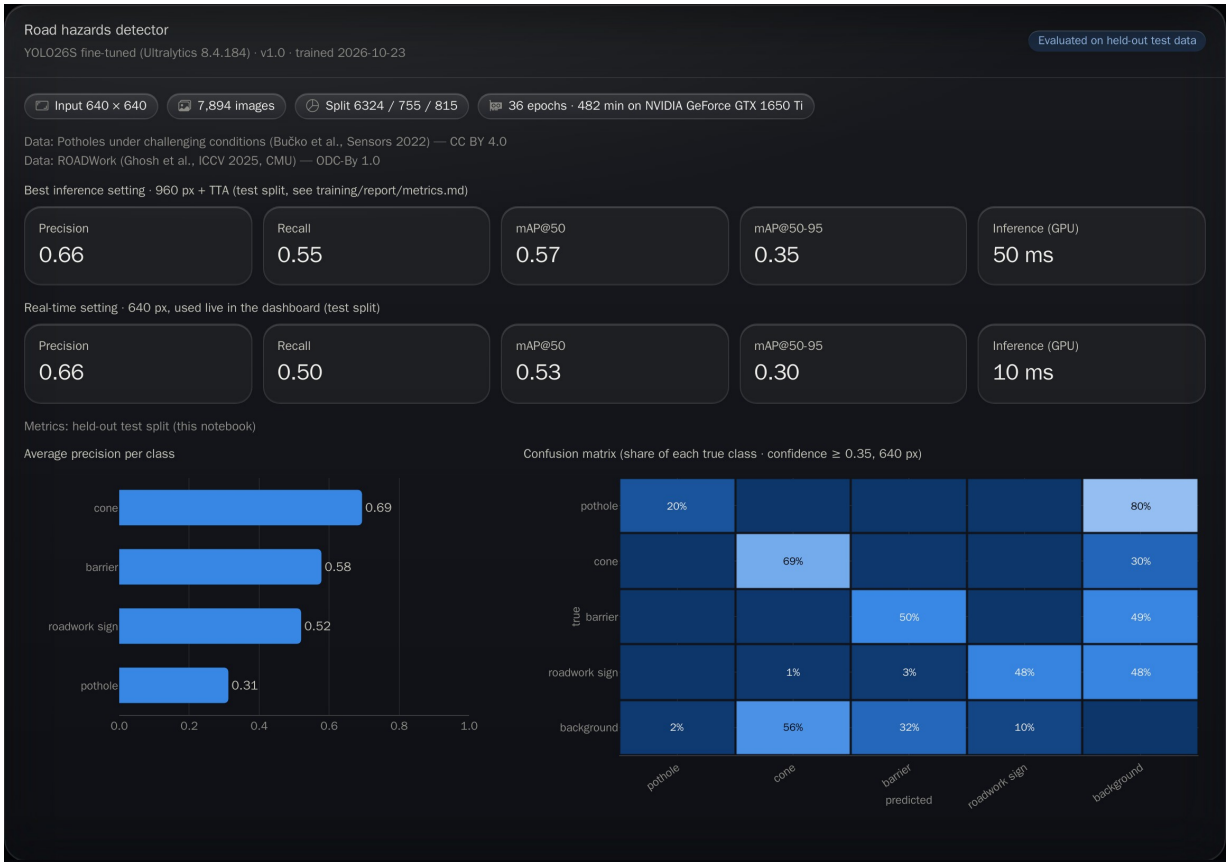


Figure 4.27. Page de présentation des modèles : détecteur d'anomalies routières

4.9.1 Présentation des modèles utilisés

Chaque modèle est présenté par une carte : nom, architecture, version, date d'entraînement, taille d'entrée, nombre d'images, répartition entraînement / validation / test, durée d'entraînement et

matériel, sources des données et badge indiquant l'origine des métriques (évaluation sur un ensemble de test, modèle préentraîné, entraînement en attente). Ces informations proviennent du registre `models/registry.json`, mis à jour par les notebooks d'entraînement.

4.9.2 Modèles YOLO26 pour les données réelles

En monde réel, deux cartes sont affichées : le détecteur des usagers de la route (YOLO26n préentraîné sur COCO, avec la liste des classes utilisées) et le détecteur d'anomalies routières (YOLO26s affiné). Pour ce dernier, deux rangées de métriques sont présentées et clairement libellées : le meilleur réglage hors ligne (960 px avec TTA) et le réglage temps réel utilisé dans l'application (640 px) (figure 4.28).

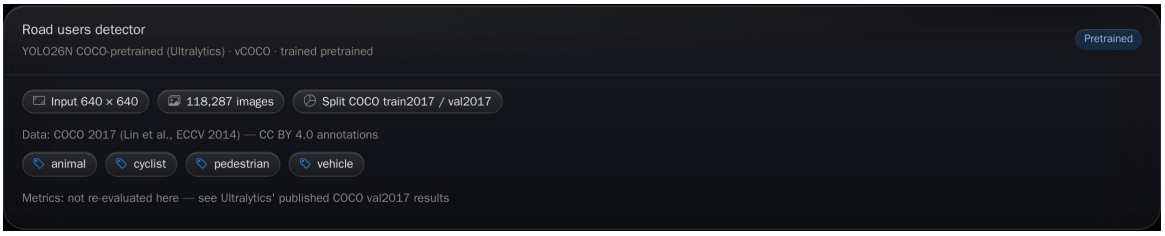


Figure 4.28. Carte du modèle des usagers de la route (préentraîné sur COCO)

4.9.3 Modèle YOLO26n pour le prototype

En mode prototype, la carte du modèle des obstacles de la piste présente ses métriques sur les images de test d'origine et sur leurs versions au style ESP32-CAM, la précision moyenne par classe et la matrice de confusion (figure 4.29).

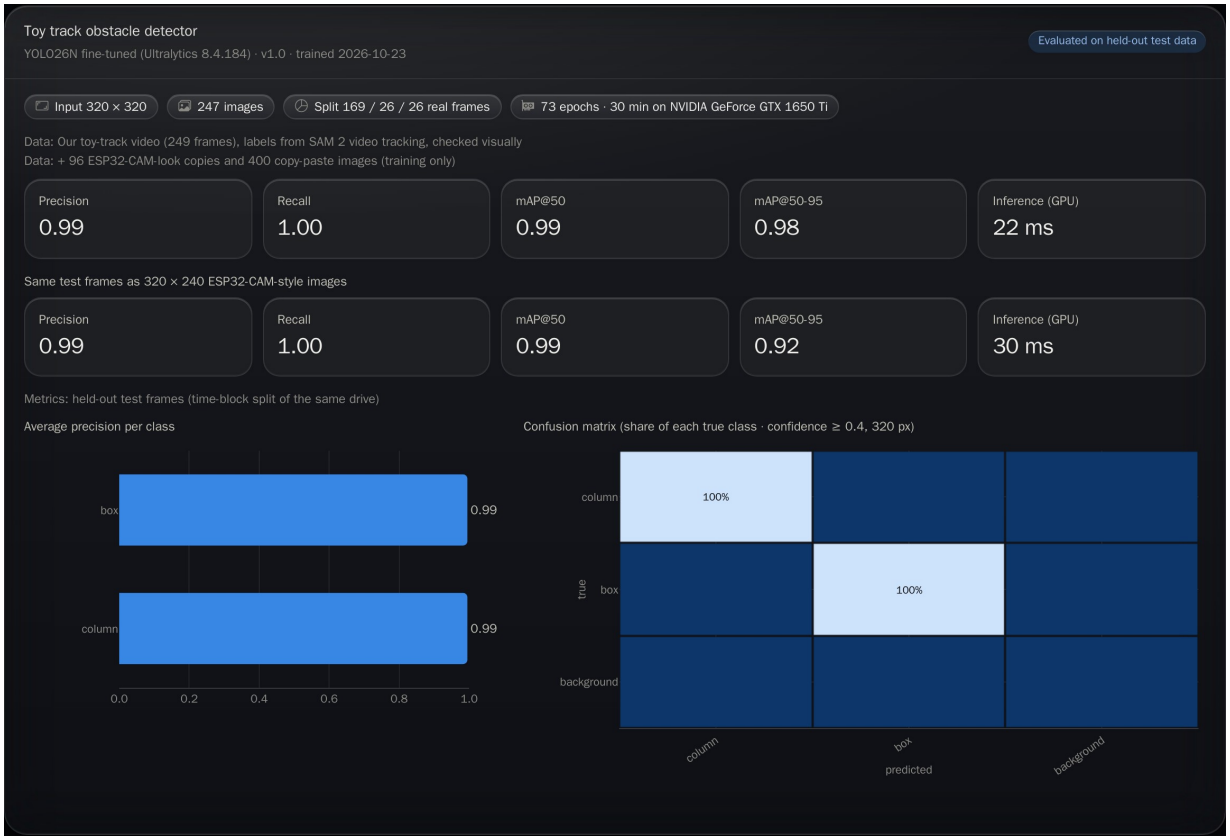


Figure 4.29. Carte du modèle du prototype

4.9.4 Configuration et chargement des modèles

La configuration est entièrement déclarative : chaque environnement dispose d'un profil qui liste ses modèles ONNX, leur taille d'entrée, leur seuil de confiance et la correspondance de leurs classes. Un nouveau modèle se déploie en copiant son fichier ONNX et en mettant à jour le profil, puis en redémarrant l'application ; un modèle manquant est signalé sans bloquer le reste.

4.9.5 Affichage des informations et des performances

Les métriques hors ligne (précision, rappel, mAP@50, mAP@50-95, temps d'inférence sur GPU) sont complétées par deux graphiques : la précision moyenne par classe et la matrice de confusion, normalisée par classe réelle et calculée au seuil de confiance utilisé dans l'application. Le panneau *On the road · last 7 days* mesure le comportement réel : nombre de détections, confiance moyenne, nombre d'obstacles revus par l'opérateur, taux de faux positifs par classe et modèle en cours d'utilisation (figure 4.30).



Figure 4.30. Comportement des modèles sur la route (7 derniers jours)

4.9.6 Comparaison des résultats des modèles

La page permet de confronter deux points de vue complémentaires : l'évaluation hors ligne, mesurée une fois sur un ensemble de test annoté, et l'évaluation en service, construite progressivement à partir des revues de l'opérateur. Un écart entre les deux (par exemple un taux de faux positifs élevé pour une classe) signale les classes à enrichir lors du prochain entraînement ; les obstacles marqués comme faux positifs constituent alors des exemples directement réutilisables.

4.10 Page Devices & Logs : périphériques et journaux du système

La page *Devices & logs* comporte trois onglets : *Devices*, *Event log* et *Settings* (figure 4.31).

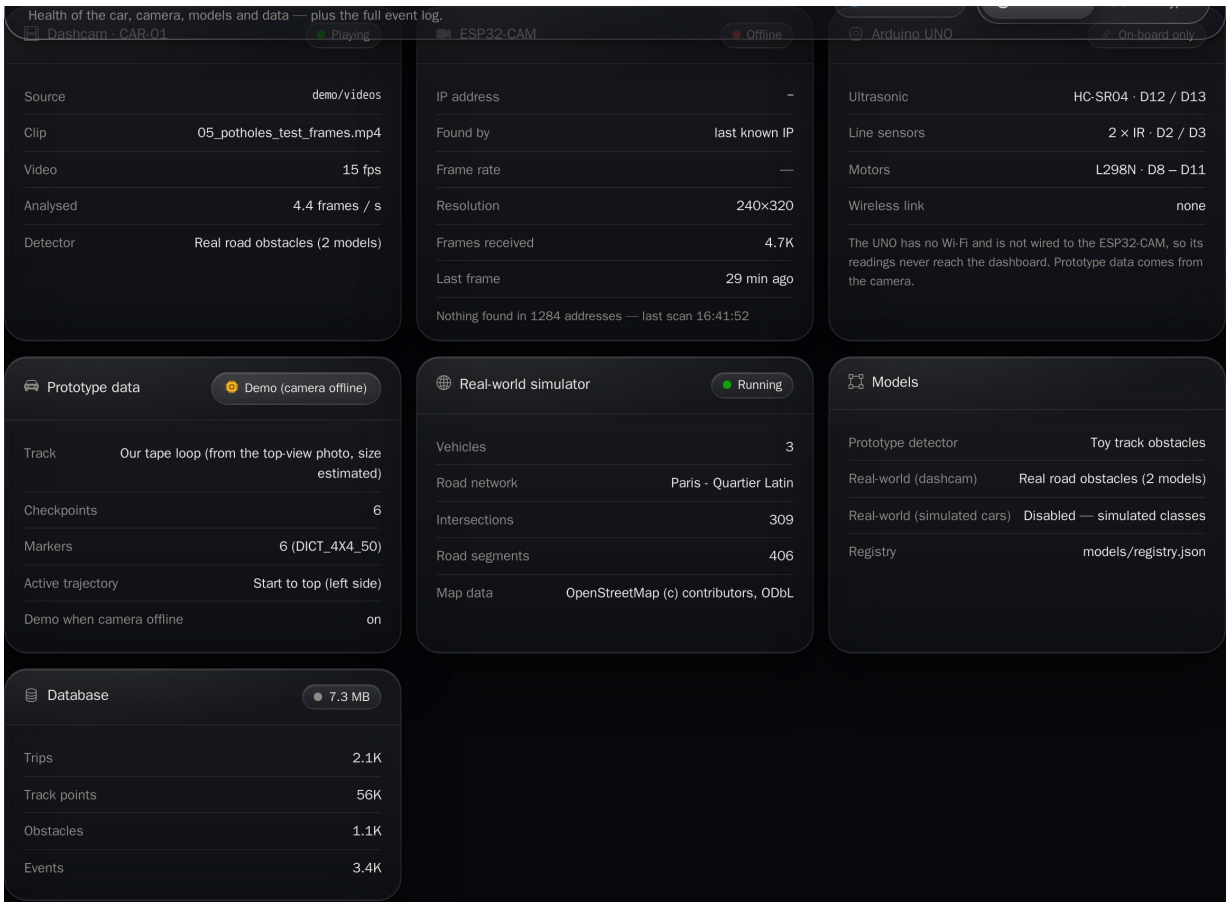


FIGURE 4.31. Périphériques et services du système

4.10.1 Présentation des périphériques connectés

L'onglet *Devices* présente une carte par équipement ou service : la caméra embarquée de CAR-01 (source, clip, cadence vidéo et cadence d'analyse), l'ESP32-CAM (adresse IP, méthode de découverte, cadence, résolution, dernière image), l'Arduino UNO (rappel de son rôle et de ses broches, sans liaison directe avec le serveur), les données du prototype (piste, trajectoire active, points de passage), le simulateur du monde réel (véhicules, réseau routier), les modèles chargés et la base de données (nombre de trajets, de points, d'obstacles et d'événements).

4.10.2 Suivi de l'état des composants et des dispositifs

Chaque carte porte un badge d'état (en ligne, hors ligne, en lecture, démonstration. . .) mis à jour en continu. Les problèmes sont ainsi visibles immédiatement : une caméra introuvable, un modèle non chargé ou une source vidéo vide.

4.10.3 Consultation des journaux d'exécution

L'onglet *Event log* affiche le journal des événements de l'environnement choisi : date, niveau (information, avertissement, critique), catégorie et message. Il peut être filtré par niveau et par catégorie (figure 4.32).

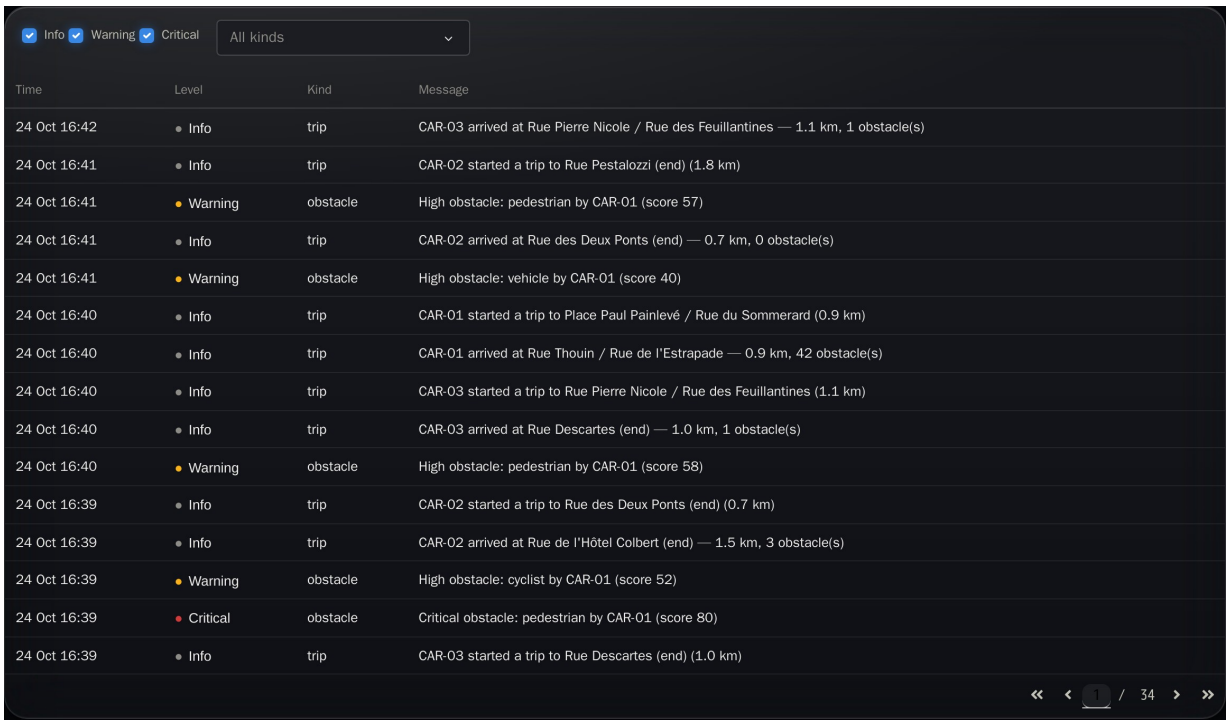


FIGURE 4.32. Journal des événements

4.10.4 Gestion des événements et des erreurs

Les événements sont enregistrés par tous les composants : début et fin de trajet, passage aux points de passage, obstacle de niveau élevé ou critique, connexion et perte de la caméra, changement de statut d'obstacles, erreurs du détecteur. Les erreurs ne bloquent pas l'application : une caméra perdue est recherchée à nouveau automatiquement, et une erreur de modèle est signalée une seule fois dans le journal puis affichée sur la page Camera.

4.10.5 Diagnostic et surveillance du système

L'onglet *Settings* permet de régler les poids de danger des classes, puis de recalculer tous les scores, et de régénérer les données de démonstration (figure 4.33). Associés à la page *Camera* (état de la connexion) et à la page *AI models* (modèles chargés), ces outils permettent de diagnostiquer rapidement l'état du système.

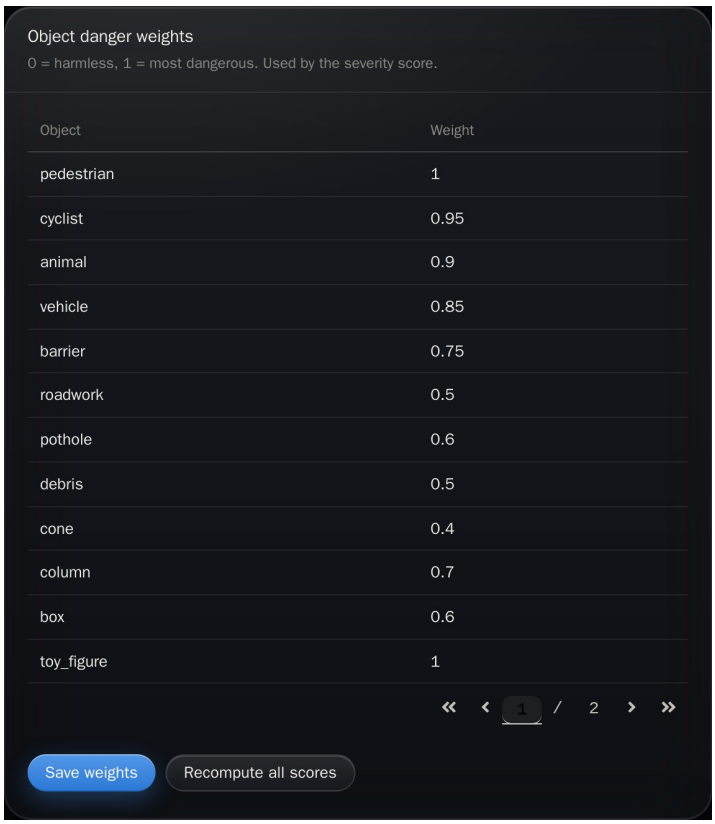


FIGURE 4.33. Réglage des poids de danger des classes

4.11 Synthèse des fonctionnalités de l'application

4.11.1 Interconnexion des différentes pages

Les pages ne sont pas indépendantes : elles partagent les mêmes données et les actions faites sur l'une se répercutent sur les autres (figure 4.34). Les détections de la page *Camera* deviennent des obstacles ; les décisions prises dans la page *Obstacles* modifient le risque utilisé par le *Route planner* et alimentent le taux de faux positifs de la page *AI models* ; les poids réglés dans *Devices & logs* changent la priorité de tous les obstacles ; la trajectoire active choisie dans le planificateur sert à la localisation de la voiture du prototype.

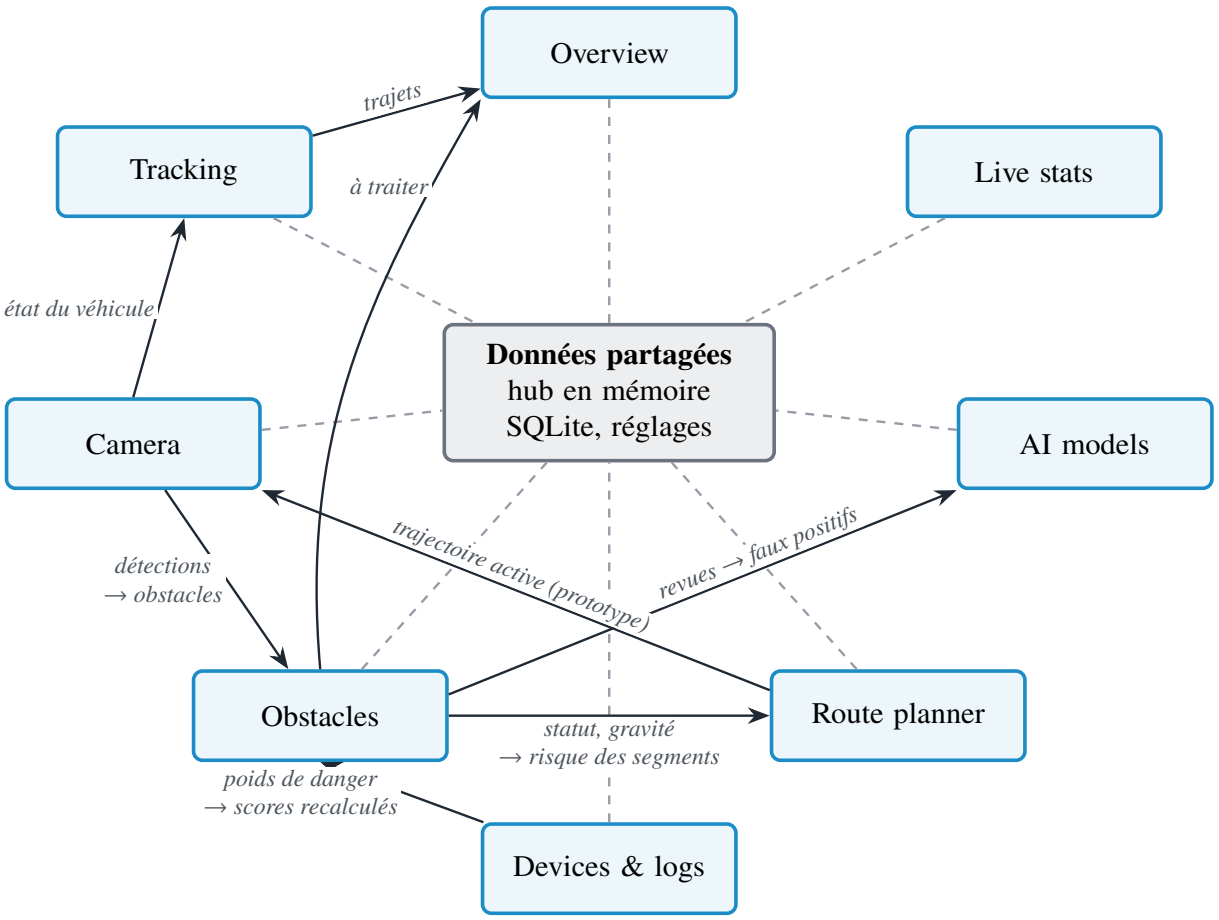


FIGURE 4.34. Interactions entre les modules de l'application

4.11.2 Circulation des données entre les composants

Les données suivent toujours le même chemin : acquisition par les services d'arrière-plan, analyse par les modèles, enregistrement dans le hub et la base de données, lecture par les pages à chaque rafraîchissement, puis retour des décisions de l'opérateur vers la base. Cette séparation entre acquisition et affichage garantit que l'interface reste fluide quelle que soit la vitesse des modèles, et que plusieurs opérateurs peuvent consulter l'application en même temps.

4.11.3 Apports de l'application au système intelligent

SmartCar Ops transforme les sorties brutes des modèles (des boîtes et des scores) en informations exploitables : des obstacles localisés, dédupliqués, hiérarchisés et suivis dans le temps, intégrés à la planification des itinéraires. Elle ferme la boucle entre la perception et la décision, et entre l'opérateur et les modèles, puisque ses revues préparent les prochains entraînements. Enfin, en gérant les deux environnements avec la même chaîne logicielle, elle montre que la solution validée sur le prototype est transposable à des données routières réelles.

Conclusion du chapitre

Ce chapitre a présenté l'application *SmartCar Ops* : ses choix technologiques (Dash, Plotly, Docker), son architecture en couches, la gestion de ses données et ses huit pages. L'application exécute les modèles en temps réel, priorise les obstacles selon un score explicable, en tient compte dans la planification des itinéraires et permet à l'opérateur de superviser aussi bien une flotte sur des routes réelles que la voiture du prototype.

Conclusion et perspectives

Ce projet avait pour objectif de concevoir, avec des composants à faible coût, un système IoT capable de détecter les obstacles sur le trajet d'un véhicule, d'en évaluer la gravité et d'exploiter ces informations en temps réel ; au terme de ce travail, une chaîne complète a été réalisée, du capteur jusqu'à la décision. Sur le plan matériel, nous avons construit une voiture miniature autour d'une carte Arduino UNO, d'un pont en H L298N, de deux moteurs à courant continu, de deux capteurs infrarouges et d'un capteur ultrasonique, qui suit de façon autonome une piste tracée au ruban adhésif et s'arrête devant un obstacle, tandis qu'une caméra ESP32-CAM diffuse en Wi-Fi les images prises de son point de vue. Sur le plan de l'intelligence artificielle, nous avons entraîné des détecteurs YOLO26 dans un environnement Docker reproductible : le détecteur d'anomalies routières, affiné pendant huit heures sur 7 894 images réelles, atteint un mAP@50 de 0,53 en temps réel (0,57 au meilleur réglage) sur des lieux absents de l'entraînement, et le modèle du prototype, entraîné sur notre propre jeu de données annoté automatiquement avec SAM 2, reconnaît tous les obstacles de son ensemble de test et détecte la boîte sur le flux réel de la caméra avec une confiance de 95 %. Sur le plan logiciel, l'application *SmartCar Ops*, développée avec Dash et Plotly, exécute ces modèles en temps réel, transforme leurs détections en obstacles localisés et priorisés selon un score explicable, les intègre à la planification des itinéraires et permet de superviser avec la même chaîne logicielle une flotte sur le réseau routier réel d'un quartier de Paris et la voiture du prototype. Ce projet nous a permis de mettre en pratique l'ensemble des notions du module (capteurs et actionneurs, microcontrôleurs, communication sans fil, traitement des données, intelligence artificielle et supervision) et d'apprendre à évaluer honnêtement un modèle en identifiant ses limites. Plusieurs perspectives permettraient de prolonger ce travail : enrichir les données du prototype avec des images enregistrées par l'ESP32-CAM elle-même, dans différents lieux et éclairages, pour supprimer les fausses détections sur des objets inconnus ; améliorer le modèle routier par un entraînement plus long et davantage d'exemples de nids-de-poule ; relier l'Arduino au système pour transmettre la distance ultrasonique et recevoir des consignes du serveur ; localiser précisément la voiture grâce aux marqueurs ArUco sur la piste et, dans le monde réel, à un module GPS ; exécuter les modèles sur un calculateur embarqué (Raspberry Pi, Jetson) pour réduire la latence ; et enfin déployer l'application sur un serveur accessible à distance, avec des notifications en cas d'obstacle critique.

Bibliographie

[1] Luigi Atzori, Antonio Iera et Giacomo Morabito. “The Internet of Things : A survey”. In : *Computer Networks* 54.15 (2010), p. 2787-2805.

[2] Ala Al-Fuqaha et al. “Internet of Things : A Survey on Enabling Technologies, Protocols, and Applications”. In : *IEEE Communications Surveys & Tutorials* 17.4 (2015), p. 2347-2376.

[3] Jayavardhana Gubbi et al. “Internet of Things (IoT) : A vision, architectural elements, and future directions”. In : *Future Generation Computer Systems* 29.7 (2013), p. 1645-1660.

[4] Jing Zhang et Dacheng Tao. “Empowering Things With Intelligence : A Survey of the Progress, Challenges, and Opportunities in Artificial Intelligence of Things”. In : *IEEE Internet of Things Journal* 8.10 (2021), p. 7789-7817.

[5] Arduino. *Arduino UNO R3 – Product Reference Manual*. url : `https://docs.arduino.cc/hardware/uno-rev3/` (visité le 24/10/2026).

[6] STMicroelectronics. *L298 – Dual Full-Bridge Driver, datasheet*. url : `https://www.st.com/resource/en/datasheet/l298.pdf` (visité le 24/10/2026).

[7] ElecFreaks. *Ultrasonic Ranging Module HC-SR04, datasheet*. url : `https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf` (visité le 24/10/2026).

[8] Espressif Systems. *ESP32 Series Datasheet, et pilote esp32-camera (exemple CameraWebServer)*. url : `https://github.com/espressif/esp32-camera` (visité le 24/10/2026).

[9] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu et al. *SAM 2 : Segment Anything in Images and Videos*. 2024. arXiv : `2408.00714`.

[10] Joseph Redmon et al. “You Only Look Once : Unified, Real-Time Object Detection”. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, p. 779-788.

[11] Alexey Bochkovskiy, Chien-Yao Wang et Hong-Yuan Mark Liao. *YOLOv4 : Optimal Speed and Accuracy of Object Detection*. 2020. arXiv : `2004.10934`.

[12] Ao Wang et al. “YOLOv10 : Real-Time End-to-End Object Detection”. In : *Advances in Neural Information Processing Systems (NeurIPS)*. 2024.

[13] Glenn Jocher et Jing Qiu. *Ultralytics YOLO26*. Ultralytics, 2025. url : `https://docs.ultralytics.com/models/yolo26/` (visité le 24/10/2026).

[14] Tsung-Yi Lin et al. “Feature Pyramid Networks for Object Detection”. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, p. 2117-2125.

[15] Shu Liu et al. “Path Aggregation Network for Instance Segmentation”. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, p. 8759-8768.

[16] Tsung-Yi Lin et al. “Microsoft COCO : Common Objects in Context”. In : *European Conference on Computer Vision (ECCV)*. 2014, p. 740-755.

[17] Boris Bučko et al. “Computer Vision Based Pothole Detection under Challenging Conditions”. In : *Sensors* 22.22 (2022), p. 8878.

[18] Anurag Ghosh, Shen Zheng, Robert Tamburo et al. “ROADWork Dataset : Learning to Recognize, Observe, Analyze and Drive Through Work Zones”. In : *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2025.

[19] Golnaz Ghiasi et al. “Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021, p. 2918-2928.

[20] Mark Everingham et al. “The PASCAL Visual Object Classes (VOC) Challenge”. In : *International Journal of Computer Vision* 88.2 (2010), p. 303-338.

[21] Rafael Padilla, Sergio L. Netto et Eduardo A. B. da Silva. “A Survey on Performance Metrics for Object-Detection Algorithms”. In : *International Conference on Systems, Signals and Image Processing (IWSSIP)*. 2020, p. 237-242.

[22] Plotly Technologies Inc. *Dash Python User Guide*. url : `https://dash.plotly.com/` (visité le 24/10/2026).

[23] Gary Bradski. “The OpenCV Library”. In : *Dr. Dobb’s Journal of Software Tools* (2000).

[24] ONNX Runtime developers. *ONNX Runtime*. 2021. url : `https://onnxruntime.ai/` (visité le 24/10/2026).

[25] Plotly Technologies Inc. *Plotly Python Graphing Library*. url : `https://plotly.com/python/` (visité le 24/10/2026).

[26] Dirk Merkel. “Docker : lightweight Linux containers for consistent development and deployment”. In : *Linux Journal* 2014.239 (2014).

[27] Sergio Garrido-Jurado et al. “Automatic generation and detection of highly reliable fiducial markers under occlusion”. In : *Pattern Recognition* 47.6 (2014), p. 2280-2292.

[28] OpenStreetMap contributors. *OpenStreetMap*. Données sous licence ODbL. url : `https://www.openstreetmap.org` (visité le 24/10/2026).

[29] D. Richard Hipp. *SQLite*. url : `https://www.sqlite.org` (visité le 24/10/2026).

[30] Edsger W. Dijkstra. “A note on two problems in connexion with graphs”. In : *Numerische Mathematik* 1 (1959), p. 269-271.

[31] Aric A. Hagberg, Daniel A. Schult et Pieter J. Swart. “Exploring Network Structure, Dynamics, and Function using NetworkX”. In : *Proceedings of the 7th Python in Science Conference (SciPy)*. 2008, p. 11-15.